



University of Macedonia

Department of Applied Informatics

Towards Optimizing Fuel Consumption in Railways: A Generic Greedy Approach in Continuous Space

Bachelor's Thesis of
Tzelalis Georgios

ics21032

Supervisor
Professor Refanidis Ioannis

Thessaloniki Greece, 2024

Dedications

To my parents, who were always there for me, even on the tough days.

To my brother, whose advice have guided me through my academic journey.

Acknowledgements

I would like to express my deepest gratitude to my advisor, Professor Ioannis Refanidis, for providing me the opportunity to work on this thesis, as well as for his support and guidance throughout my research. Their expertise and insights have been instrumental in shaping this work. Finally, I am profoundly grateful to my friends and family, whose consistent support, love, and patience have been a source of strength throughout this educational journey. This thesis is dedicated to all those who have supported me in my academic journey.

Περίληψη

Η παρούσα διατριβή ασχολείται με το πρόβλημα της παραγωγής προφίλ κίνησης τρένου με την ελάχιστη κατανάλωση καυσίμων ως μετατροπή του προφίλ κίνησης με την ελάχιστη διάρκεια. Το τρένο κινείται σε ένα τμηματοποιημένο μονοπάτι με διαφορετικά όρια ταχύτητας και στόχος μας είναι να τροποποιήσουμε το αρχικό προφίλ κίνησης αυξάνοντας τη διάρκειά του ώστε να παράγουμε το προφίλ κίνησης με την ελάχιστη κατανάλωση εντός του επεκταμένου χρονικού πλαισίου. Τα σιδηροδρομικά δίκτυα αποτελούν πυλώνα της οικονομίας και η ζήτησή τους συνεχώς αυξάνεται, όμως η αντιμετώπιση του αναφερόμενου προβλήματος για εξοικονόμηση καυσίμων δεν έχει ερευνηθεί. Η δική μας προσέγγιση τροποποιεί συγκεκριμένα τμήματα του αρχικού προφίλ κίνησης σύμφωνα με τον λόγο της κατανάλωσης ως προς το χρόνο. Ο επαναληπτικός αλγόριθμος που αναπτύξαμε αποτελείται από δύο ακόμη αλγόριθμους οι οποίοι τροποποιούν αυτά τα τμήματα του προφίλ κίνησης. Η μεθοδολογία μας ακολουθεί μία άπληστη προσέγγιση στην οποία οι βασικές μεταβλητές είναι ο χρόνος και η ταχύτητα. Τα αποτελέσματα της έρευνάς μας δείχνουν πως οι πιο αποδοτικές τροποποιήσεις επηρεάζουν αποκλειστικά τις ακολουθίες «Επιτάχυνση, Σταθερή Ταχύτητα και Επιβράδυνση» και «Επιτάχυνση και Επιβράδυνση» του προφίλ κίνησης, όπου η δεύτερη είναι η βέλτιστη επιλογή. Η προτεινόμενη λύση μας θα μπορούσε να αποτελέσει βάση στην κατασκευή ενός μοντέλου το οποίο αξιοποιεί το χρόνο που δημιουργείται από τις καθυστερήσεις των τρένων για την αποφυγή συγκρούσεων μεταξύ τους για να ελαχιστοποιήσει τη συνολική κατανάλωση.

Λέξεις κλειδιά: Τρένο, καύσιμα, κατανάλωση, βέλτιστο, άπληστο

Abstract

This thesis addresses the challenge of producing fuel-optimal speed profiles for locomotives by transforming time-optimal profiles. In the time-optimal speed profile, the train travels over a single segmented railway line with various speed limits for the least time. Our approach aims to adjust its speed profile by extending its total duration to produce the fuel-optimal speed profile within that extended time frame. Rail networks are crucial to the economy, and their demand has been increasing over the years. However, the minimization of fuel consumption for the described problem has not been studied. Our approach modifies specific instances of the given speed profile to minimize the fraction of fuel consumption over time.

The developed iterative algorithm consists of two sub-algorithms that address those instances, following a greedy approach where the main variables are time and speed. The study's results demonstrated that it is most effective to adjust only "Acceleration and Deceleration" and "Acceleration, Cruising and Deceleration" throttle sequences, with the former always being the best option. Our proposed solution could contribute to minimizing fuel consumption in railway networks by leveraging the delays created to avoid collisions at intersections as the basis of the model.

Keywords: Train, fuel, consumption, optimal, greedy

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.3	Contribution	3
1.4	Study's Structure	3
2	Related Work	4
2.1	Train Scheduling	4
2.1.1	Optimization of Stop Locations and Line Planning	4
2.1.2	Delay and Conflict Management	5
2.1.3	Speed Control and Routing Optimization	6
2.1.4	Genetic Algorithms	6
2.1.5	Reinforcement Learning	7
2.1.6	Mathematical and Heuristic Models	8
2.2	Fuel Consumption	8
2.2.1	Scale of the Input Variables-based Modelling	9
2.2.2	Formulation Approach-based Modelling	10
2.2.3	Main Input Variable-based Modelling	16
2.2.4	State Variable Value-based Modelling	21
2.2.5	Number of Dimensions-based Modelling	24
3	Problem Definition	26
3.1	Fuel Consumption Model	26
3.1.1	Instant Consumption	26
3.1.2	Consumption of Acceleration	27
3.1.3	Consumption of Deceleration	27
3.1.4	Total Fuel Consumption Model	27
3.2	Problem Formulation	27
4	Methodology	29
4.1	Algorithms	29
4.1.1	Acceleration, Cruising and Acceleration	30
4.1.2	Acceleration, Cruising and Deceleration	33
4.1.3	Acceleration and Deceleration	37
4.1.4	Deceleration, Cruising and Deceleration	42
4.2	Implementation	44

4.2.1	Basic Functions	45
4.2.2	Algorithms	47
5	Results	68
5.1	Example 1	68
5.1.1	Description	68
5.1.2	Analysis	70
5.2	Example 2	73
5.2.1	Description	73
5.2.2	Analysis	74
5.3	Example 3	76
5.3.1	Description	76
5.3.2	Analysis	77
5.4	Example 4	80
5.4.1	Description	80
5.4.2	Analysis	80
5.5	Example 5	84
5.5.1	Description	84
5.5.2	Analysis	85
5.6	Example 6	87
5.6.1	Description	87
5.6.2	Analysis	88
6	Conclusion and Future Work	93

List of Figures

4.1	Consumption of Time-Optimal Speed Profile for Problem 1	31
4.2	Time-Optimal Speed Profile for Problem 1	31
4.3	Acceleration, Cruising and Acceleration Method 1	32
4.4	Acceleration, Cruising and Acceleration Fuel Consumption of Method 1 .	32
4.5	Acceleration, Cruising and Acceleration Method 2	33
4.6	Acceleration, Cruising and Acceleration Fuel Consumption of Method 2 .	33
4.7	Time-Optimal Speed Profile for Problem 2	34
4.8	Consumption of Time-Optimal Speed Profile for Problem 2	35
4.9	Acceleration, Cruising and Deceleration Method 1	35
4.10	Acceleration, Cruising and Deceleration Fuel Consumption of Method 1 .	36
4.11	Acceleration, Cruising and Deceleration Method 1 Special Case	36
4.12	Acceleration, Cruising and Deceleration Method 2	37
4.13	Acceleration, Cruising and Deceleration Fuel Consumption of Method 2 .	37
4.14	Acceleration, Cruising and Deceleration Method 3	38
4.15	Acceleration, Cruising and Deceleration Fuel Consumption of Method 3 .	38
4.16	Time-Optimal Speed Profile of Problem 3	39
4.17	Consumption Time-Optimal Speed Profile of Problem 3	39
4.18	Acceleration and Deceleration Method 1	40
4.19	Acceleration Deceleration Fuel Consumption of Method 1	40
4.20	Acceleration and Deceleration Method 2	41
4.21	Acceleration and Deceleration Fuel Consumption of Method 2	41
4.22	Acceleration and Deceleration Method 3	42
4.23	Acceleration and Deceleration Fuel Consumption of Method 3	42
4.24	Time-Optimal Speed Profile of Problem 4	43
4.25	Fuel Consumption of Time-Optimal Speed Profile of Problem 4	44
4.26	Deceleration, Cruising and Deceleration Method 1	44
4.27	Deceleration, Cruising and Deceleration Fuel Consumption of Method 1 .	45
4.28	Deceleration, Cruising and Deceleration Method 2	45
4.29	Deceleration, Cruising and Deceleration Fuel Consumption of Method 2 .	46
5.1	Time-Optimal Speed Profile of Example 1	69
5.2	Fuel Consumption of Time-Optimal Speed Profile of Example 1	70
5.3	Fuel-Optimal Speed Profile of Example 1	72
5.4	Fuel Consumption of Fuel-Optimal Speed Profile of Example 1	73
5.5	Time-Optimal Speed Profile of Example 2	74
5.6	Fuel Consumption of Time-Optimal Speed Profile of Example 2	74
5.7	Fuel-Optimal Speed Profile of Example 2	76

5.8	Fuel Consumption of Fuel-Optimal Speed Profile of Example 2	77
5.9	Time-Optimal Speed Profile of Example 3	78
5.10	Fuel Consumption of Time-Optimal Speed Profile of Example 3	78
5.11	Fuel-Optimal Speed Profile of Example 3	79
5.12	Fuel Consumption of Fuel-Optimal Speed Profile of Example 3	80
5.13	Time-Optimal Speed Profile of Example 4	81
5.14	Fuel Consumption of Time-Optimal Speed Profile of Example 4	82
5.15	Fuel-Optimal Speed Profile of Example 4	84
5.16	Fuel Consumption of Fuel-Optimal Speed Profile of Example 4	84
5.17	Time-Optimal Speed Profile of Example 5	86
5.18	Fuel Consumption of Time-Optimal Speed Profile of Example 5	86
5.19	Fuel-Optimal Speed Profile of Example 5	88
5.20	Fuel Consumption of Fuel-Optimal Speed Profile of Example 5	88
5.21	Time-Optimal Speed Profile of Example 6	90
5.22	Fuel Consumption of Time-Optimal Speed Profile of Example 6	90
5.23	Fuel-Optimal Speed Profile of Example 6	91
5.24	Fuel Consumption of Fuel-Optimal Speed Profile of Example 6	91
5.25	Fuel-Optimal Speed Profile of Example 6	92

List of Algorithms

1	Determine the Fuel-Optimal Speed Profile	48
2	Addition of Cruising	49
3	Slower Constant Speed	50
4	Computations of Algorithm 2	51
5	Acquire States at the End of Cruising	52
6	Computations of Algorithm 3	53
7	Identify "Acceleration and Deceleration" Instances	54
8	Initialize the "Addition of Cruising" Method Computations	55
9	Construct the Speed Profile of "Addition of Cruising" Method	56
10	Identify the Start of an "Acceleration and Deceleration" Instance	57
11	Identify the State of an "Acceleration, Cruising and Deceleration" Instance	58
12	Construct the Speed Profile in Normal Case	59
13	Construct the Speed Profile in Alternative Case	60
14	Employ "Slower Constant Speed" Method Utilizing All Extra Time	62
15	Construct the Speed Profile for Decelerating Alternative Case	64
16	Construct the Speed Profile for Accelerating Alternative Case	65
17	Initialize the Computations of Algorithm 18	66
18	Computations of Algorithm 14	67

List of Tables

4.1	Problem 1	30
4.2	Train 1	30
4.3	Path 1 Layout	30
4.4	Problem 2	34
4.5	Path 2 Layout	34
4.6	Problem 3	38
4.7	Path 3 Layout	39
4.8	Problem 4	43
4.9	Path 4 Layout	43
5.1	Train 2	68
5.2	Problem 5	69
5.3	Path 5 Layout	69
5.4	Problem 6	73
5.5	Path 6 Layout	73
5.6	Problem 7	77
5.7	Path 7 Layout	77
5.8	Problem 8	80
5.9	Path 8 Layout	81
5.10	Problem 9	85
5.11	Path 9 Layout	85
5.12	Problem 10	89
5.13	Path 10 Layout	89

Chapter 1

Introduction

1.1 Motivation

Transport networks, including rail, have always been crucial to the economy, serving as both service and infrastructure providers (Economic and Papers, 2014). Researchers have suggested that rail travel can meet an individual's desire for high mobility while maintaining low negative environmental, social, economic, and financial impacts (Mian-doab et al., 2020). The demand for transport networks, particularly railways, has been increasing over the years. The European Commission aims for rail networks to handle most of the medium-distance passenger transport by 2050 (Commission et al., 2011).

Rail usage across the UK increased significantly between 2010 and 2020, with growth ranging from 102% in Wales to 247% in the West Midlands (Logan et al., 2020), while France experienced a 34% increase in tram and metro passenger-kilometers from 2020 to 2021 (Commission et al., 2022). Additionally, transport networks are essential for the integration and efficiency of the EU's internal market. Investment in cross-border energy infrastructure is also needed to improve the EU's energy security and the functioning of the market (Economic and Papers, 2014).

Asia is also expanding its railway networks. Since 2010, new metro systems have been introduced in 46 countries, 34 of which are in Asia (Lin et al., 2021). According to data from the National Bureau of Statistics of China, railway passenger volume increased from 5% in 2010 to 21% in 2019. On the other hand, even though automobile travel remains the most dominant mode of transport, the proportion of highway passengers decreased from 93% in 2010 to 74% in 2019 (X. Guo et al., 2020).

Beijing experienced rapid growth in its rail transit infrastructure between 2005 and 2015. In that period, the rail transit mileage of Beijing expanded from 114 km to 554 km, reflecting an average annual growth rate of 38.6% (S. Guo and Chen, 2019). By the end of 2019, 37 cities in mainland China had opened metro systems, totaling 5180.6 km in length (Lin et al., 2021), while, according to IEA, 2023, China's railway network reached 155000km by the end of 2022.

By the end of 2022, high-speed railway networks had globally become a prominent mode of transportation (Zhao et al., 2023; Xiang et al., 2023). The environmental impact of these diesel-powered freight locomotives has faced criticism as worldwide environmental concerns and regulatory constraints have increased (Abbey, 2022). According to Facts, 2021, freight trains alone consumed 3082 million gallons of fuel in 2021, which translates into an estimated 35.2 million tons of CO_2 emissions. Although studies of many countries have identified railways as one of the best environmentally friendly transport modes (Miandoab et al., 2020), González-Gil et al., 2014 are concerned that urban rail may lose its competitive edge in terms of environmental emissions if it does not reduce its energy usage while maintaining or enhancing its service quality and capacity. X. Guo et al., 2020 reported that a 10% increase in the number of people transported by railways each year resulted in a 1% decrease in CO and a 2% decrease in NO_x emissions. Their study was based on inter-city railway and air pollution data in Germany.

Logan et al., 2020 estimated that by 2050, at 100% capacity, conventionally fueled trains (i.e., petrol and diesel power) in the UK will produce a fifth of the emissions of conventionally fueled vehicles per kilometer per person. They suggested that between 2017 and 2050, for the UK, total diesel rail emissions will increase by 5.2%. This indicates that despite technological advancements in fuel consumption and emission factors for diesel trains, the emission levels remain relatively constant, with regional and intercity rail operating emissions increasing slightly over the given period. According to S. Guo and Chen, 2019, the increasing proportion of rail transit in the urban traffic system in Beijing from 2005 to 2015 eased ground traffic congestion while the air pollution indices, including concentrations of SO_2 , NO_2 and PM_{10} , showed a downward trend.

1.2 Objectives

In this study, we present an approach to producing the fuel-optimal speed profile of a train for a railway line given the time-optimal speed profile. Our method transforms the time-optimal speed profile to fuel-optimal by utilizing extra time. This method considers a time-optimal speed profile and various train and path characteristics such as maximum speed, acceleration and deceleration rates, acceleration and deceleration durations, and the maximum allowed speed per railway segment. Additionally, this study focuses only to single railway lines, rather than railway networks.

Our approach operates in continuous domains, allowing a train to be located anywhere on the railway line with any speed and to accelerate or decelerate while adhering to physical constraints. The goal is to minimize fuel consumption under real-world conditions on railway networks, thereby aiding the development of a practical fuel-efficient application for real-world railway operations.

The approach's structure and originality allow for numerous extensions, such as incorporating additional train features (e.g., load; engine torque; variable maximum acceleration/deceleration depending on speed), environment factors (e.g., slopes of the ground; wind), alternative optimization criteria (e.g., passenger comfort) or a more complex fuel consumption model.

1.3 Contribution

This study provides a comprehensive algorithm designed to compute the fuel-optimal speed profile of a train given its time-optimal speed profile. The algorithm takes into account a variety of crucial factors, ensuring the locomotive operates efficiently while adhering to the physical constraints inherent to its design and capabilities. Additionally, it ensures compliance with the maximum allowed speed for any segment of the railway. Its main contribution is filling the literature gap regarding the generation of fuel-optimal speed profiles over a segmented railway with different speed limits. Another possible contribution is its use as the groundwork of a fuel-optimal railway network scheduling system that utilizes the extra time created by the locomotives' deceleration to avoid collisions. The implemented code along with multiple problems can be found at GitHub¹.

1.4 Study's Structure

The rest of the dissertation is organized as follows.

- Chapter 2 presents a review of the background literature, regarding both train scheduling and fuel consumption models,
- Chapter 3 introduces the fuel consumption model design to satisfy the needs of this study and demonstrates the problem's formulation,
- Chapter 4 discusses all the existing approaches to alter a speed profile and details the algorithms which are implemented, to compute the fuel-optimal speed profile,
- Chapter 5 depicts our algorithm's behaviour against various problems, and
- Chapter 6 concludes the dissertation and poses future challenges.

¹Link for the GitHub repository: <https://github.com/g-tzel/Towards-Optimizing-Fuel-Consumption-in-Railways-A-Generic-Greedy-Approach-in-Continuous-Space>

Chapter 2

Related Work

In this chapter, the literature review is presented, organized into two distinct parts. The first part delves into Train Scheduling approaches, while the second section provides an in-depth report on Fuel Consumption techniques.

2.1 Train Scheduling

There have been multiple approaches to develop a model that provides the optimal scheduling over either a single line or complete railway network. The following subsections investigate six categories of classification these approaches can be classified into and present a number of examples. The six categories of approaches are:

1. optimization of stop locations and line planning,
2. delay and conflict management,
3. speed control and routing optimization,
4. genetic algorithms,
5. reinforcement learning and
6. mathematical and heuristic models.

2.1.1 Optimization of Stop Locations and Line Planning

The first category emphasizes on optimizing various aspects of railway planning, including the placement of stops, the creation of efficient line plans, and the scheduling of trains on single-track railways. The goal is to enhance overall system efficiency, minimize passenger travel time, and address specific constraints associated with single-track operations. Harbering et al., 2015 develop a dynamic programming approach to schedule trains on single-track railways, where trains can only pass each other at designated stations. This problem is akin to job shop scheduling with two opposing routes and no preemption.

Albeit their model efficiently solves the problem, offering valuable insights for optimizing train schedules on single-track railways, it regards only specific cases and only a fixed number of blocks.

Gattermann et al., 2015 proposed an iterative algorithm to address the line pool generation problem for public transportation systems. This problem involves selecting a subset of potential lines to optimize line planning outcomes. Their model balances the need for flexibility in line plans against computational complexity. The study found that the quality of line plans heavily depends on the initial line pool, underscoring the importance of a strategic approach to line pool generation for efficient public transportation planning. However, their model needs to be adjusted for each different scenario to ensure its effectiveness.

Carrizosa Priego et al., 2016 investigated the optimization of stop locations in public transportation networks to minimize passenger travel time. They developed a model that evaluates the impact of adding stops on total travel time, differing from traditional models that focus only on the number of stops. They proved the problem is NP-hard and introduced integer programming formulations to address it.

Harbering, 2017 introduces a line planning model designed to improve the delay resistance of public transportation systems by minimizing passenger transfers. The basic planning problem solved in this work is the line planning problem, or more precisely the frequency setting problem. The model integrates passenger routing into the line planning stage to lessen the effects of delays. Using a column generation approach, the study showcases that the resulting line plans significantly improve system reliability and punctuality. Yet, it does not directly consider real-time delay data.

2.1.2 Delay and Conflict Management

This subsection addresses methodologies for detecting and managing delays and conflicts in railway systems. By leveraging network-based techniques and advanced algorithms, these studies aim to identify delay causes, optimize real-time scheduling, and resolve conflicts efficiently. For instance, D'ariano et al., 2007 developed a branch and bound algorithm for real-time management of train schedules in a busy railway network. They designed a detailed mathematical model incorporating railway signaling aspects, safety rules, and no-store constraints. Trains are represented as jobs, tracks as machines, and an alternate graph formulation is used to manage the no-wait and no-store constraints. A branch and bound algorithm was employed to find conflict-free schedules, aiming to minimize the maximum secondary delay for trains. The algorithm demonstrated notable improvements over traditional dispatching rules, reducing both maximum and average delays, and demonstrating the advantages of global optimization for real-time applications. Yet, the model's high computational complexity making it unsuitable for real-world applications.

Corman et al., 2010 developed a tabu search algorithm to reroute trains during rail operations. Their study addresses train conflict detection and resolution. Their algorithm is part of the real-time traffic management system Railway traffic Optimization by Means of Alternative (ROMA) graphs. Its performance was validated using practical instances

from the Dutch railway network, including complex disturbances with multiple late trains and blocked tracks. The results demonstrated that their approach outperformed previous versions, achieving up to 15% better solutions in significantly shorter times.

Manitz et al., 2015 proposed network-based techniques to identify the causes of delays in railway systems. Based on a general approach for source detection in complex networks, they developed an enhanced model which was applied to real railway city systems in Germany and Greece. The network-based approach proved to be reliable and efficient in identifying the causes of delays.

2.1.3 Speed Control and Routing Optimization

This category explores approaches that optimize train speed control and routing, aiming to improve operational efficiency and passenger comfort. Y. H. Yoon and Bang, 2011 proposed an apparatus and method for controlling speed in automatic train operation under the patent “US 2011/0046827 A1”. Their method estimates and controls the speed of a train at every location to satisfy a restrictive speed profile while ensuring passenger comfort by observing acceleration and jerk (i.e. the rate of change of acceleration) limits. The proposed system aims to improve train operation efficiency and safety by automatically performing functions such as acceleration, deceleration, and stopping. The method allows for flexible planning and correction of the running schedule to accommodate for inconsistent speed limits and conditions. However, the train’s length is not considered in the problem formulation significantly reducing the complexity since for example, a train cannot be in multiple segments with overlapping speed limits.

Samà et al., 2016 presented a mathematical approach to optimize real-time train routing using an ant colony optimization algorithm. The routing selection problem was formulated as a linear programming model and solved with an adaptation of the ant colony optimization algorithm. The model efficiently handles the combinatorial nature of the problem, providing better quality solutions in a shorter computation time. Yet, it cannot scale to larger networks.

Eaton et al., 2017 presented a model that also employs ant colony optimization to tackle the intricate issue of train rescheduling, especially in situations that are dynamic and involve multiple objectives. The main objective is to reduce energy consumption and train delays in a simulated railway junction. Key findings suggest that maintaining non-dominated solution archives and updating pheromone trails between changes greatly improve algorithm performance in high-frequency, high-magnitude change scenarios. Yet, the study does not account real-time applicability.

2.1.4 Genetic Algorithms

Genetic algorithms are used to explore large search spaces and generate feasible schedules, addressing various operational constraints and real-world challenges. Tormos et al., 2008 developed a genetic algorithm for optimizing train timetabling on highly utilized railway lines. Their model aims to efficiently generate feasible and optimized train schedules considering various operational constraints, such as track capacity and train priorities. The model was tested with real-world from the Spanish Manager of Railway

Infrastructure and it demonstrated that it can explore large search spaces and produce good solutions within reasonable computational times. Yet, further refinement is needed to improve the model's performance near local optima.

Dündar and Şahin, 2013 developed a genetic algorithm and an artificial neural network model to address train re-scheduling challenges in single-track railways. Their approach aimed to resolve inter-train conflicts caused by deviations in normal operations. These conflicts are traditionally handled by train dispatchers using simplifying rules. The genetic algorithm was designed to minimize total delays by generating feasible schedules encoded as binary strings representing meeting and passing conflicts. On the other hand, the neural network was trained on real conflict resolution data from Turkish State Railways to emulate dispatcher decisions. Their results showed that the genetic algorithm outperformed the artificial neural network in most scenarios. Yet, the former algorithm does not always provide optimal solutions.

Nirmala and Ramprasad, 2014 introduced an optimization method for train scheduling to address the transit network design problem. Their model utilizes genetic algorithms to further minimize the overall number of locomotives by considering all routes together that was evaluated individually for every route in the first step of the model. The paper showcases the benefits of genetic algorithms for global optimization and addresses real-world constraints.

M. Wang et al., 2019 developed a genetic algorithm-based particle swarm approach to reschedule high-speed railway timetables. They formulated a mixed integer programming model incorporating strategies like retiming, reordering, and adjusting stop patterns. Their algorithm improves upon traditional methods by reconstructing position vectors and genetic evolution operators based on train departure and arrival times. When compared to alternative strategies like first-come, first-served, the methodology showed a decrease in objective value. Furthermore, all delay instances are addressed within 1.5 minutes. Yet, the model is not suitable for managing serious disruptions.

2.1.5 Reinforcement Learning

In this subsection we discuss the use of reinforcement learning techniques for railway scheduling and rescheduling. By modeling the railway environment and employing learning algorithms like Q-learning and deep Q-networks, these studies aim to optimize train operations and minimize delays.

Šemrov et al., 2016 introduced a reinforcement learning-based method for train rescheduling on a single-track railway. The authors specifically utilize Q-learning in their method. Their approach models the railway environment, defining the possible states and actions of a learning agent, and uses a reward function designed to minimize delays. Their method was validated using real-world data of a railway network in Slovenia. The empirical results indicate that Q-learning can produce rescheduling solutions that are often superior, or at least equivalent, to traditional methods like FIFO and random walk. However, the model has high computational complexity.

Khadilkar, 2018 developed a scalable reinforcement learning algorithm for schedul-

ing railway lines, based on Q-learning. The study examined both single- and multi-track railway lines with the primary goal of optimizing track allocations and train arrival/departure times. Their goal should be met while also minimizing total priority-weighted delay. The algorithm's efficiency was demonstrated on two lines from the Indian rail network, where it showed superior performance compared to two previous heuristic-based approaches. Yet, this approach demands considerable time.

Gong et al., 2020 investigated the use of deep reinforcement learning for train scheduling on a single-track unidirectional railway. Their focus was on determining the stations where express trains overtake preceding local trains. The problem was formulated as a Markov decision process whose Q-value functions were estimated using a deep Q-network. They validated their model's effectiveness in generating feasible schedules through numerical experiments. Yet, the model cannot scale appropriately to larger problems.

Agasucci et al., 2023 investigated the application of deep reinforcement learning techniques to address the train dispatching problem. The paper proposes a decentralized and a centralized deep Q-learning technique. Their research demonstrates that these deep Q-learning approaches outperform traditional linear Q-learning methods based on matrices. Yet, their model does not provide explainable mathematical trends.

2.1.6 Mathematical and Heuristic Models

The last category presents mathematical and heuristic models designed to optimize various aspects of railway scheduling, such as passenger train routing and freight transportation. For instance, W. Zhou and Teng, 2016 developed a mathematical model for simultaneous passenger train routing and timetabling on a rail network. They introduced an integer linear programming model, which was then reformulated as a train path-choice model. This reformulation allowed for a train-based Lagrangian relaxation decomposition that used fewer Lagrangian multipliers. Additionally, the authors designed a solving framework that included a heuristic algorithm to optimize both dual and feasible solutions. Their model outperformed earlier techniques in numerical testing. Yet, the model fails to scale efficiently on more complex problems.

Bożejko et al., 2017 developed a mathematical model that finds the fastest routes for rail freight transportation. Their approach adapts the Dijkstra algorithm and accounts for real-world factors, such as passenger train timetables and safety intervals. Experimental results demonstrated that their model can efficiently find optimal routes in extensive railway networks.

2.2 Fuel Consumption

There have been multiple approaches to develop a model that estimates the fuel consumption rate of vehicles. The following subsections investigate the five categories of classification these approaches can be classified into and present several examples based on their applicability. The five categories of approaches are:

1. scale of the input variables-based modelling,
2. formulation approach-based modelling,
3. main input variable-based modelling,
4. state variable value-based modelling and
5. number of dimensions-based modelling.

2.2.1 Scale of the Input Variables-based Modelling

The first category is scale of the input variables-based modelling, which was proposed by Yue, 2008. Scale of the input variables-based modelling can be divided into microscopic models, mesoscopic models, and macroscopic models. These three categories can all be called modal models if they account for different operating modes.

Microscopic Models

Microscopic models estimate instantaneous vehicle fuel consumption and emission rates which are derived from a relationship between fuel consumption rates and instantaneous vehicle power. VT-microscopic model (VT-Micro) is a microscopic model regarding instantaneous fuel consumption and emission rates based on the instantaneous speed and acceleration levels of individual vehicles (Ahn, 1998; Rakha et al., 2000; Ahn, 2002). The instantaneous fuel consumption and emission rates of individual vehicles that the model predicts given the instantaneous speed and acceleration were calculated by equation 2.1 (Yue, 2008).

$$MOE_e = \begin{cases} \sum_{i=0}^3 \sum_{j=0}^3 \exp(k_{i,j}^e \cdot v_T^i \cdot a^j), & \text{for } a \geq 0 \\ \sum_{i=0}^3 \sum_{j=0}^3 \exp(l_{i,j}^e \cdot v_T^i \cdot a^j), & \text{for } a < 0 \end{cases} \quad (2.1)$$

where MOE represents the Measure of Effectiveness; MOE_e represents the instantaneous fuel consumption or emission rate; $k_{i,j}^e$ represents the model regression coefficient for MOE_e at speed i ; and acceleration power j ; $l_{i,j}^e$ represents the model regression coefficient for MOE_e at speed i ; and acceleration power j for positive accelerations; v represents the instantaneous vehicle speed; and a represents the instantaneous vehicle acceleration. In order to account for the sensitivity of emission rate to speed between the different acceleration levels of the vehicle, the VT-Micro distinguished four driving modes: accelerating, idling, cruising and decelerating. According to the study, VT-Micro is able to model the vehicles fuel consumption with higher speed and sharper acceleration with a precision of 2.5% compared to the measured field values. However, the model's predictions provide unexplainable mathematical trends due to its empirical formulae.

Macroscopic Models

Rather than different driving modes macroscopic models use average aggregate network parameters to estimate network-wide energy consumption and emission rates. Watson et al., 1980 the model represented in equation 2.2 which incorporates the changes in the

positive kinetic energy during acceleration as a predictor variable.

$$F = K_1 + \frac{K_2}{V_S} + K_3V_S + K_4PKE \quad (2.2)$$

The term PKE of equation 2.2 represents the sum of the positive kinetic energy changes during acceleration in m/s^2 , and is calculated using equation 2.3.

$$PKE = \sum \frac{V_f^2 - V_i^2}{12.960X_S} \quad (2.3)$$

When the average speeds are over 55 km/h the aerodynamics effects on fuel consumption become significant (Evans et al., 1976). The model estimates the fuel consumption given the initial speed, the average speed, the final speed, and the total section length. Even though the Watson model is good for estimating fuel consumption, the accuracy of its estimation is inferior to the models based on instantaneous speed.

Mesoscopic Models

The mesoscopic models are based on a middle ground between macroscopic and microscopic models since their input variables are neither fully disaggregated nor completely aggregated. They tend to use a few explanatory variables to estimate vehicle fuel consumption and emissions (Yue, 2008). Herman (M.-F. Chang and Herman, 1981; Evans and Herman, 1978) proposed a model based on average speed, named the elemental model. The elemental model is a simple theoretical model expressing fuel consumption in urban conditions as a linear function of the average trip time per unit distance, i.e., reciprocal of average speed. This model is presented in equation 2.4.

$$\Phi_E = K_1 + K_2T, \text{ for } v < 55 \text{ km/hr} \quad (2.4)$$

where K_1 represents the gallons per mile which is a vehicle-depended parameter; K_2 represents the gallons per hour which is a vehicle-depended parameter; and T represents the hours per mile. The model takes into account the trip time, the trip distance and the average speed and estimates the fuel consumption of the vehicle. However, it does not consider aerodynamic drag at high speeds, so the elemental model, and others models alike mainly use the average speed of the vehicle, in most cases, they should not be used for speeds higher than 55 km/h (Akcelik, 1985). SIDRA INTERSECTION (Akcelik, 1985; Richardson and Akcelik, 1981) is a software that is widely used in application in Australia and has adopted the elemental model. This software estimates fuel consumption for each driving cycle separately. Similar to the VT-Micro the cycles are namely cruising, idling and acceleration-deceleration. The main disadvantage of this model is that it does not take into account the transient changes in vehicle's speed and acceleration. Ding, 2000 analyzed the vehicle fuel consumption and emission rates based on modelling and found out that the times the vehicle stops is proportional to the fuel consumption especially at high cruise speed.

2.2.2 Formulation Approach-based Modelling

Apart from modal models there are also formulation approach-based modelling, which is the development of the vehicle model. The four categories engulfing these models are analytical, empirical, statistical, and graphical modelling.

Analytical Models

The analytical modelling of vehicle models is utterly derived from the principles of physics in order to represent the relationship amongst the vehicle's subsystems in terms of mathematical formulae. This approach has been followed by many researchers such as Lavoie and Blumberg, 1980 who proposed a thermodynamics-based model to predict fuel consumption, and emissions. The model was a function of engine design and operating conditions. Although the model proved to be reliable in predicting fuel consumption, it is suitable only for spark ignition engines. Instead of constructing a new formula to calculate fuel consumption they used the Indicated Specific Fuel Consumption (ISFC) as shown in equation 2.5.

$$ISFC = \frac{\text{Fuel Mass Flow Rate}}{\text{Indicated Power Output}} \quad (2.5)$$

This formula represents the fuel mass flow rate per unit of indicated power output. The indicated power output is derived from the Indicated Mean Effective Pressure (IMEP) and the displacement volume of the engine. Equation 2.6 represents the IMEP.

$$IMEP = IMEP \times \text{Displacement Volume} \times \text{Engine Speed} \quad (2.6)$$

Harris and Pearce, 1990 dedicated their study to a governed diesel engine and developed a simple mathematical model of its performance on the engine using the following concept. At every throttle setting, as torque rises, the speed of a diesel engine will decrease from its initial value to zero torque. With further torque increase, the engine operates along a curve where fuel delivery is maximized. Equations 2.7 and 2.8 represent the fuel consumption model.

$$F = (b_{11}Z + b_{22}Z^2 + b_{13}Z^3) + (b_{21}Z + b_{22}Z^2 + b_{23}Z^3) \cdot R + (b_{31}Z + b_{32}Z^2 + b_{33}Z^3) \cdot R^2 \quad \text{ml/s} \quad (2.7)$$

$$F = b_1 + b_2R + b_3R^2 \quad \text{ml/s at constant } Z \quad (2.8)$$

where Z represents the engine speed at zero torque, commonly known as idle speed while R on the other hand, represents the depression of engine speed below Z . In other words, it is the difference between the current engine speed and the idle speed (Z). The implementation of the model is simple, and it can indirectly measure the tractor engine performance, but it is constrained by the type of engine and its concept of use.

In order to optimize the driving power and manage the overall fuel consumption for a single internal combustion engine vehicle Khayyam et al., 2008 designed an analytical model of fuel consumption (AMFC). The model they developed assesses various loads applied on the vehicle such as wind drag, mechanical losses, accessories, road slope and road friction. The formula they constructed to calculate the vehicle's fuel consumption is represented by equation 2.9.

$$m_{\text{fuel}} = \int_0^T MFR dt \quad (2.9)$$

where T represents the number of steps in the simulation and MFR the mass fuel rate consumption which is calculated as a function of numerous parameters including fuel road friction power loss, road slope power loss, a constant accessory power loss, otto cycle efficiency and mechanical efficiency. Despite its effectiveness, the model requires extensive calculations.

D. J. Chang and Morlok, 2005 developed a mathematical formula to calculate the fuel consumption in order to answer the question of what speed profile will minimize vehicle fuel consumption. Equation 2.10 represents the fuel consumption model of a single straight segment with constant speed for locomotives.

$$z = r(1.5T + 0.03Tv + CAv^2)s \quad (2.10)$$

where z represents the fuel consumption of the section; T represents the vehicle's weight; v represents the velocity of the vehicle; C represents the air drag coefficient; A represents the frontal cross-sectional area of the vehicle; and s represents the length of the section. Equation 2.11 represents the fuel consumption of a single curved path with constant speed for locomotives.

$$z = r(1.5T + 0.03Tv + CAv^2 + 0.8Td)s \quad (2.11)$$

where d represents the degree of curvature. Respectively, equation 2.12 represents the fuel consumption of a single graded path with constant speed for locomotives.

$$z = r(1.5T + 0.03Tv + CAv^2 + 20Tq)s \quad (2.12)$$

where q represents the percent grade (i.e. the vertical rise/distance along track). The fuel consumption for paths with variable speed where the train either accelerates or decelerates is the integral of the formulas for constant speed. The authors proved that operating the vehicle engine at a steady speed approximately minimizes its fuel consumption. Even though the results are useful while the vehicle is cruising, they are not practical because the model does not take into consideration transient conditions. Although the aforementioned study belongs to analytical modelling, it has significant empirical validation which could affiliate it with empirical modelling.

Empirical Models

The next category of formulation approach-based modelling that will be discussed is called phenomenological modelling which is mostly known as empirical modelling. Empirical modelling heavily relies on data collection and analysis from real-world observations and the trial-and-error method. In other words the researches do not tend to use any formulas for the calculation of fuel consumption rate. In the literature it is mainly used in the field of powertrain modelling and control. Hrovat and Sun, 1997 published a survey for idle speed control (ISC) application regarding linear internal combustion engine models and control design methodologies. In their report they stated that the application of ISC techniques, for instance sensitivity tuning controls, led to improved engine performance at idle speed. Furthermore, they highlighted the importance of controlling the transition both from and to the ISC mode. Although, the models that are presented in the study better define the relationship amongst idle speed and the rate of fuel consumption, they are applicable only when the vehicle is idling.

A non-linear model of the dynamics of the SI engine under various operating circumstances was presented by P. Yoon et al., 2000. Intake manifold dynamics, fuel film dynamics, and engine rotational dynamics with transport delays inherent in four-stroke engine cycles were all incorporated in the model. Albeit it does not take into consideration compression ignition engines and is valid only for SI engines.

In order to predict the dynamic features that are impacted by mixture components, such as changes in air flow, manifold pressure, and the AFR based on the water vapor and gaseous fuel in the mixture, Sim and Sunwoo, 2004 proposed a model of the dynamics of a gaseous-fuel engine. While the model finds correlations between many dynamic features of the internal combustion engine and fuel consumption rate, it does not offer tenable mathematical trends.

A model-based approach for the real-time load control of parallel hybrid vehicles was presented by Sciarretta et al., 2004. The goal of the model is to establish a fuel-optimal control. Yet, the model is only applicable for the set of data on which it was built.

Pu and Yin, 2007 introduced a mathematical approach for the best possible regulation of vehicle fuel economy. They presented a cost function regarding the impact of frequent gear changes and engine stop-starts on driving comfort and fuel efficiency. An algorithm that limits the explorable region was proposed as a solution to the dimensionality issue of numerical dynamic programming (DP). This approach minimizes computing complexity. MATLAB/RealTimeWorld was used to transform the system model into a real-time simulation code. Albeit the model does not offer explainable mathematical trends. Equation 2.13 represents the accumulated fuel consumption over the given driving cycle.

$$J = \int_0^f \mathcal{L}[\mathbf{x}(t), \mathbf{u}(t), t] dt \quad (2.13)$$

where $\mathcal{L}[\mathbf{x}(t), \mathbf{u}(t), t]$ is the fuel consumption rate of the internal combustion engine (ECI) at time t which is a function the vector of state variables of the system ($\mathbf{x}(t)$) and the vector of control variables at time t ($\mathbf{u}(t)$).

Ni and Henclewood, 2008 proposed two different empirical engine with acceptable accuracy, simplicity and computational efficiency models for vehicle infrastructure integration-enabled in-vehicle applications. Those two models, which are the Genta's polynomial model and the Genta's parabolic model originally proposed by Genta, 1997, were validated successfully using empirical data. Equation 2.14 represents the fuel consumption formula these models use.

$$Q = \frac{qP_r}{\eta_t V \rho_f} \quad (2.14)$$

where Q represents the amount of fuel consumed to produce a unit of power over a unit of time; P_r represents the power needed to overcome resistive forces, such as aerodynamic drag; η_t represents the efficiency of the drivetrain at time t ; V represents the vehicle's speed and ρ_f represents the density of the fuel. Although these two empirical models provide information about the relation between engine power and the fuel consumption

rate, they need calibration on peak power, peak torque, and their associated engine speeds.

Rakha et al., 2010 introduced a straightforward vehicle driveline model that can be integrated with car-following models within microscopic traffic simulation software. The proposed empirical model is calibrated using publicly available data regarding engine and driveline parameters without a necessity for field data collection. The researchers observed that the model was able to produce vehicle acceleration, speed, position, and fuel consumption estimates that align with field observations. Equation 2.15 represents the formula used to calculate fuel consumption.

$$\delta(\omega_e) = P \cdot \hat{\delta}_{\max}(\omega_e) \quad (2.15)$$

where $\delta(\omega_e)$ represents the fuel consumption; P represents the proportion of pedal signal ($P \in [0, 1]$); and $\hat{\delta}_{\max}(\omega_e)$ represents the maximum fuel consumption rate.

Brandstätter et al., 1985 in order to study the effect of engine configurations including valve profile, inlet manifold, piston bowl, pent roof piston, manifold configurations, duct length etc. conducted experimental work on flow modelling which they managed to successfully validate. Even though the model contributes to literature regarding the influence of the engine's configuration on the engine's performance and on the rate of fuel consumption, it is only applicable to the dataset its development was based on.

Powell et al., 1998 discussed the mathematical modelling, analysis, and simulation of a dynamic automatic transmission and manual dry clutch combination powertrain model. Experimental data were used for both powertrain models to simulate and validate them. The formula for fuel consumption used by the authors in the context of a dynamic automatic transmission and manual dry clutch combination powertrain is based on the air/fuel system equations. These equations involve the manifold fuel flow rate and injector fuel rate commands, considering various dynamic factors such as fuel condensation and evaporation. Equation 2.16 represents the manifold fuel flow rate.

$$m_f = (1 - \eta)\dot{m}_{fi} + \dot{m}_p \quad (2.16)$$

where m_f represents the manifold fuel flow rate; m_{fi} represents the injected fuel rate; m_p represents the fuel evaporation rate and η represents the fuel accumulation parameter.

Nilsson et al., 2008 proposed a model of a variable compression diesel engine to capture the effects of ignition and compression ratio on the diesel engine torque. The main goal of the proposed model is to establish control signals to achieve fuel consumption minimization for each given engine torque and speed. Equation 2.17 represents the mass fuel.

$$m_{\text{fuel}} = \frac{n_r}{N} \cdot \frac{\lambda}{(A/F)_s} m_{\text{air}} \quad (2.17)$$

where m_{fuel} represents the mass of fuel; n_r represents the revolutions per cycle; N represents the engine speed; λ represents the normalized air/fuel ratio; $(A/F)_s$ represents the stoichiometric air/fuel ratio and m_{air} represents the air mass flow. The model's error percentage was 1.2% according to validation based on experimental data. Yet, it does not provide explainable mathematical trends.

Watson et al., 1980 proposed an empirical correlation simulating the turbocharged diesel engine's combustion process (heat release) via a mathematical expression whose governing parameters are linked to in-cylinder conditions. According to Kim et al., 2002 the developed model became essential for modelling combustion in diesel engines as it enlightens what influences the fuel consumption rate. Equation 2.18 represents the rate of fuel consumption.

$$\dot{m}_t = \dot{m}_p + \dot{m}_d \quad (2.18)$$

where $\dot{m}$ represents the fuel burning rate with respect to crank angle and the subscripts t, p and d denote total, pre-mixed and diffusion burning, respectively. The model also provides the combustion information at operating point. Albeit the model is only applicable for the dataset its development was based on.

Filipi and Assanis, 2001 introduced a transient, non-linear, single-cylinder turbocharged diesel engine model for predictions of instantaneous engine speed and torque. The model is inspired by Assanis and Heywood, 1986 as it is the transient version of the seminal semi-empirical, thermodynamic, steady state turbocharged diesel engine model they presented, and it uses the fuel burning rate correlation proposed by Watson et al., 1980. This modified model represents the diesel engine as a non-linear, dynamic system and gives an insight into the relation between rate of fuel consumption, engine speed, and torque. However, the model is valid only for the dataset on which its development was based on.

Killingsworth et al., 2006 developed a simple model that produces an estimate of the combustion timing on a cycle-by-cycle basis for the HCCI's combustion process. The authors for the prediction of the start of the combustion, which influences fuel consumption and emission rates, set an ignition threshold as a function of the in-cylinder temperature and pressure. The study fails to explain the calculation of the fuel consumption rate, however, the generic formula that could be applied in this study, as it combines its concepts, is represented by equation 2.19.

$$F = \frac{\text{Mass of Air}}{\text{AFR}} \quad (2.19)$$

In the context of this paper, the air-to-fuel ratio in the cylinder (AFR_{cyl}) is calculated according to equation 2.20.

$$AFR_{cyl}(k) = (1 - x_{cyl}) (1 + AFR_{man}(k)) \frac{m_{cyl}(k)}{m_{cyl}(k) - m_{res}(k)} - 1 \quad (2.20)$$

where AFR_{man} represents the air to fuel ratio of the intake charge; x_{cyl} represents the inert gas fraction in the cylinder; m_{res} represents the mass of residual gas trapped from the previous cycle; m_{cyl} represents the total mass of gas in the cylinder and k represents the cycle index.

Statistical Models

The third category of formulation approach-based modelling is statistical models. Statistical modelling is an approach that models a system solely using data and statistical methodologies. It provides beneficial features, such as the capability of quantifying uncertainty, as a result a number of researchers have preferred it for vehicle powertrains modelling.

Biggs and Akcelik, 1987 evaluated the relation of vehicle features to fuel consumption using field data and a basic energy-related model of instantaneous fuel consumption. Equation 2.21 represents the formula of the fuel consumption rate.

$$F = \frac{f_i}{V} + 20.7 + 0.0443M \quad (2.21)$$

where f_i represents the idle fuel rate; V represents the average travel speed; and M represents the vehicle's mass. The authors illustrated the effects engine mass and capacity have on the parameters of the model. Yet, a recalibration of the model is essential for every different dataset.

Ouladsine et al., 2005 developed a model of a turbocharged diesel engine based on a neural network. The model is constructed of three interconnected neural sub-models, each of them constituting a non-linear multi-input single-output output error model. Three interconnected neural sub-models of separate non-linear multi-input single-output output error models are the components of the model. The field data are collected from a real diesel engine or on static mapping and used by the model to estimate the parameters. The study found that the model provides information into what influences the rate of fuel consumption. Equation 2.22 represents the formula for the injected fuel flow.

$$m_f(k) = f(R(k), T(k)) \quad (2.22)$$

where R represents the engine speed; T represents the injection pump position; k represents the discrete time of the engine speed; and f is the state of the "Injection" of the turbocharged diesel engine modeled by the neural network. Yet, a recalibration of the model is essential for every different dataset.

Graphical Models

The last category of formulation approach-based modelling is graphical modelling. Graphical modelling is an approach that makes use of graphical tools and graphical techniques to represent a system's subsystems as cohesive, interconnected objects. It mostly does not require the development of a formula for the rate of fuel consumption as it is predicted by the tools used by the tools the researchers utilize. An example of that is Gao et al., 2007 who proposed a physics principles-based resistive companion form technique and bond graph method with a powertrain component. Furthermore, they illustrated the modelling and simulation competence of existing tools including power electronics and motor drive software (PSIM), advanced vehicle simulator (ADVISOR), powertrain system analysis toolkit (PSAT), and virtual test bed. The aforementioned modelling methods and tools can predict the fuel consumption and emission rates. Yet, those tool's models do not give explainable mathematical trends.

2.2.3 Main Input Variable-based Modelling

Another way of modelling the rate of fuel consumption of a vehicle is based on the primary input independent variable. There are three distinct categories of main input in main input variable-based modelling, Main average speed, instantaneous speed, and specific power.

Average Speed Models

The first category to discuss will be average speed which is the most widely used independent input variable in vehicle fuel consumption and emission rates modelling. There are many research papers which favor average speed as the main independent input variable in vehicle fuel consumption modelling. Evans and Herman, 1978 which was previously introduced as a mesoscopic model, can be also considered modelling based on average speed as the main input independent variable.

Ding and Rakha, 2002 established a practice for estimating vehicle emissions based only on average speed, which at the time of the study was considered state-of-the-art. The authors discovered that in the literature EPA MOBILE5 model indicates that slowing of traffic typically increases emissions, while in many instances empirical research papers indicate the opposite. Additionally, they proposed statistical models that use average speed as a crucial input variable to estimate emissions and fuel consumption. Equation 2.23 represents the proposed formula for calculating fuel consumption which is a slight modification of equation 2.1.

$$\ln(MOE_e) = \sum_{i=0}^3 \sum_{j=0}^3 \exp(k_{i,j}^e \cdot v_T^i \cdot a^j) \quad (2.23)$$

Even though these models were successfully validated, they require recalibration with each dataset.

Taylor, 2003 introduced CONTRAM, a computer model designed to simulate time-varying traffic in road networks. This model uses the network definition and varying travel demand between origin and destination zones as inputs to produce as outputs network flows, routes, and travel times. The paper described the primary method employed in this model, which is time-dependent queuing. Additionally, it included an empirical model for fuel consumption and emissions based on the function by Everall, 1968, which relates average speed to fuel consumption. Equation 2.24 represents the fuel consumption per unit of distance.

$$Q = \left(a_0 + \frac{a_1}{V} + a_2 V^2 \right) D \quad (2.24)$$

where Q represents the quantity of the fuel consumption; V represents the average speed and D represents the distance. The coefficients a_0 , a_1 , a_2 are vehicle-depended and have been derived by empirical data. While this average speed model is useful for estimating aggregate emissions inventories and is highly relevant to road traffic, it does not account for the impact of transient changes in vehicle speed and acceleration.

Liao and Machemehl, 1998 proposed an analytical fuel consumption model (AFCM) based on average speed to assess the impact of signal timing on fuel consumption at signalized intersections. The AFCM outlines fuel consumption across three street segments, namely inbound approach, intersection itself, and outbound leg, during three signal cycle stages (effective red time, queue departure time, and remaining green time). Equations 2.25, 2.26 and 2.27 represent the fuel consumption models of the three cycle stages for inbound approach.

$$TF_{ib-1} = \int_0^r (qt f_0 + qT_{11} f r_0) dt \quad [\text{Effective Red Time Cycle}] \quad (2.25)$$

where q represents the average flow rate on the approach; f_0 represents the idle fuel consumption rate; r represents the effective red time; T_{11} represents the free-flow travel time; and f_{r0} represents the fuel consumption rate for vehicles moving from speed V_r to idle V_0 .

$$TF_{ib-2} = \int_0^{t_0} (qv f_{02} + av f_{r3}) dt \quad [\text{Queue Departure Time Cycle}] \quad (2.26)$$

where qv represents the queued vehicles; f_{02} represents the average fuel consumption rate for vehicles moving from speed V_0 to V_2 ; av represents the arriving vehicles; and f_{r3} represents the average fuel consumption rate for vehicles moving from speed V_r to V_3 .

$$TF_{ib-3} = qT_{13}f_r(g - t_0) \quad [\text{Remaining Green Time Cycle}] \quad (2.27)$$

where T_{13} represents the average travel time for vehicles moving on inbound approach during the remaining green time cycle; f_r represents the vehicle fuel consumption rate for desired speed V_r and g represents the remaining green time. Moving on the intersection street segment during the effective red time cycle, the fuel consumption is assumed to be zero as the vehicle cannot enter the intersection. Equations 2.28 and 2.29 represent the fuel consumption models of the other two signal cycle stages for the intersection street segment.

$$TF_{int-2} = \int_0^{t_0} (qv f_{24} + mv f_{35}) dt \quad [\text{Queue Departure Time Cycle}] \quad (2.28)$$

where f_{24} represents the average fuel consumption rate for vehicles moving from speed V_0 to V_4 ; mv represents the moving vehicles; and f_{35} represents the average fuel consumption rate for vehicles moving from speed V_3 to V_5 .

$$TF_{int-3} = qk(g - t_0)f_r \quad [\text{Remaining Green Time Cycle}] \quad (2.29)$$

where k represents the average travel time across the intersection. Regarding outbound leg street segment, equations 2.30, 2.31 and 2.32 represent the signal cycle stages, respectively.

$$TF_{ob-1} = \int_g^{g+r_1} (mva f_{5r} + mvd f_r) dt \quad [\text{Effective Red Time Cycle}] \quad (2.30)$$

where mva represents the moving vehicles with acceleration; f_{5r} represents the average fuel consumption rate for vehicles moving from speed V_5 to V_r ; and mvd represents the vehicles moving with the desired speed.

$$TF_{ob-2} = \int_k^{t_0} (qv f_{4r} + mv f_{5r}) dt \quad [\text{Queue Departure Time Cycle}] \quad (2.31)$$

where the values of f_{4r} and f_{5r} represent two different fuel consumption rates, one for vehicles that have been stopped and the other for vehicles that are delayed by signals.

$$TF_{ob-3} = \int_{t_0}^g (mva f_{5r} + mvd f_r) dt \quad [\text{Remaining Green Time Cycle}] \quad (2.32)$$

The fuel consumption rates in the AFCM were estimated using vehicle speed and acceleration/deceleration profile models along with corresponding Environmental Protection

Agency (EPA) fuel consumption data, while the total fuel consumption is the addition of all the equations from 2.25 to 2.32. The study compared numerical experiment results produced using the AFCM with those from the TEXAS simulation model, showing a 10% error. Yet, the AFCM does not consider the significant effect of speed variability on vehicle fuel consumption and emission rates.

Newman et al., 1989 introduced a fuel consumption model for petrol and diesel trucks. It predicts fuel consumption based on statistical data, utilizing average speed, engine displacement, and vehicle mass. Equation 2.33 represents the fuel consumption formula used in this paper.

$$f = k_1 + k_2 M + k_3 \frac{EC}{V} \quad (2.33)$$

where M represents the mass of the vehicle and EC represents the engine displacement while the constants k_1 , k_2 and k_3 are obtained by regression. They discovered that vehicle mass has the most significant impact on fuel consumption. The model they developed has a moderate accuracy level with a 20% error margin.

Instantaneous Speed Models

Instantaneous speed is the second most widely used independent input variable in vehicle fuel consumption modelling. There are several researchers that have implemented instantaneous speed as the main independent input variable in vehicle fuel consumption modelling in their respective research papers. Multiple studies, including Rakha and Ahn, 2004; Ahn et al., 2002, adopted variations of the VT-Micro model presented by Ahn, 2002 which can detect short-term fluctuations in a vehicle's speed and acceleration as it moves through a highway network. Yet, in the literature, instantaneous speed is mostly used to estimate emission rates.

Zweiri et al., 2001 proposed a comprehensive non-linear dynamic model for diesel single-cylinder engines. The model operates in the crank angle domain rather than the time domain, which allows for a more precise and detailed representation of the engine's dynamic behaviour. It satisfactorily illustrates the dynamic behaviour and interactions between engine speed and fueling. Equation 2.34 represents the formula used for calculating the fuel burning rate in this paper.

$$\dot{m}_{f_norm} = f_t \dot{m}_{f_pre} + (1 - f_t) \dot{m}_{f_diff} \quad (2.34)$$

where f_t represents the fraction of the total fuel that goes to the premixed burning rate; $\dot{m}_{f_pre}$ represents the normalized premixed burning rate; and $\dot{m}_{f_diff}$ represents the normalized diffusion burning rate. Although the model captures the variations in the engine's speed is accurate to the second, it is considered a prolonged procedure.

Hung et al., 2005 developed a data-based model of vehicle fuel consumption and emissions as a function of driving mode and instantaneous speed. They proposed piecewise interpolation functions derived from the collected data for each non-idling driving mode in that model (accelerating, cruising and decelerating). Equation 2.35 represents the piecewise interpolation function regarding the non-idle driving modes.

$$f(x) = \frac{\sum_{j=1}^N w(x_j) e_j}{\sum_{j=1}^N w(x_j)} \quad (2.35)$$

where x represents the distance between the data point and the center point of the corresponding speed class; $w(x_j)$ represents the weighting function that is equal to $(1 - x)$ and e_j represents the fuel consumption rate of the data point located at x_j . To estimate the idling fuel consumption, they proposed the formula represented by equation 2.36.

$$F_I(t_I) = \begin{cases} a \exp(-bt_I) & \text{if } t_I < T \\ e & \text{if } t_I \geq T \end{cases} \quad (2.36)$$

where E represents the instantaneous fuel consumption rate; t_I represents the idle time. The constants a and b are the regression coefficients. Albeit the described approach can capture short-term variations in a vehicle's speed as it travels through a highway network, the conduction of the inventory analysis is highly laborious.

Specific Power Models

The third independent input variable that will be discussed is specific power. Many research papers adopted specific power as the main independent input variable in vehicle fuel consumption modelling. Frey et al., 2007 introduced a vehicle-specific power (VSP) approach for modeling fuel consumption in diesel and hydrogen fuel cell buses. Equation 2.37 represents the formula used for the estimation of the trip-based fuel consumption.

$$E = \sum_{j=1}^J FR_j \cdot TVSP_j \quad (2.37)$$

where E represents the total trip fuel consumption; j represents the Vehicle Specific Power (VSP) mode index; J represents the number of VSP modes (specifically $J = 8$ for diesel buses and $J = 6$ for hydrogen buses as explained in the paper); FR_j represents the fuel consumption rate for VSP mode j ; and $TVSP_j$ represents the bus trip time spent in VSP mode j . The VSP is calculation is based on the instantaneous speed v as its represented by equation 2.38.

$$VSP = v(a + g \sin(\phi) + 0.092) + 0.00021 \cdot v^3 \quad (2.38)$$

where a represents the instantaneous acceleration of the vehicle; ϕ represents the instantaneous road grade; 0.092 is the rolling resistance term coefficient; and 0.00021 is the drag term coefficient. The research's results concluded that the relative errors between estimated and actual trip fuel consumption were typically less than 10% for all observations. The study suggested using the VSP-based modeling approach when the necessary data are available.

H. Wang et al., 2008 proposed a data-driven VSP-based model of vehicle fuel consumption that used a portable emissions monitoring tool to evaluate the impact of driving patterns on fuel consumption. In this paper, the fuel consumption rate is estimated according to equation 2.37. Its variation is on the constants used in the calculation of VSP. They discovered that as acceleration increases, fuel consumption increases dramatically. Albeit this specific power model is reasonably accurate, it is only valid for the set of data on which it was developed.

Song et al., 2009 presented a model to assess the impact of traffic management on the fuel economy of light-duty vehicles. This model illustrates the correlation between

VSP, real-world driving activities, and the associated fuel consumption. The instantaneous fuel consumption is calculated from emissions of CO₂, HC, and CO using the carbon balance method, as shown in equation 2.39.

$$FR = \left(ER_{CO_2} \cdot \frac{12}{44} + ER_{CO} \cdot \frac{12}{28} + ER_{HC} \cdot \frac{12}{13} \right) \cdot \frac{1}{\%C} \quad (2.39)$$

where FR represents the fuel consumption rate; ER_x represents the emission rate of x emission; and $\%C$ represents the carbon percentage of gasoline by weight ($\approx 86.4\%$). Yet, this approach requires the availability of precise power data.

2.2.4 State Variable Value-based Modelling

The state variable value-based modelling is another classification of modelling of vehicles' fuel consumption. Guzzella and Amstutz, 1998, as well as Cook et al., 2006, proposed the crank-angle resolution-based models and the mean value-based models as two subcategories of this classification method.

Crank-angle Resolution-based Models

Watson, 1984 discussed the cylinder-by-cylinder modelling, its crank angle resolution characteristic, and its explainable trends. Using the current crank-angle resolution, it forecasts the effects of mechanical and/or control system modifications on vehicle powertrain output torque, fuel consumption, and emissions. Equation 2.40 represents the fuel burning rate model of Watson.

$$\frac{dm_f}{dt} = \beta \left[\frac{dm_f}{dt} \right]_p + (1 - \beta) \left[\frac{dm_f}{dt} \right]_d \quad (2.40)$$

with

$$(m_f)_p = 1 - (1 - \tau_f^p) C_p^2 \quad (2.41)$$

$$(m_f)_d = 1 - \exp(-C_d \cdot r \tau_f C_d^2) \quad (2.42)$$

where equation 2.41 represents the premixed fuel burning rate formula; equation 2.42 represents the diffusion fuel burning rate formula; and β denotes that fraction of the fuel injected that burns in the premixed combustion mode and integration.

Streit and Borman, 1971 introduced a crankshaft resolution model for multi-cylinder turbocharged diesel engines. The cylinder model incorporates instantaneous heat transfer, homogeneous combustion burning rates that explain how they fuel consumption rates, and a scavenging model allowing any intermediate state between perfect scavenging and complete mixing. Equations 2.43, 2.44, 2.45 and 2.46 represent the estimation of fuel consumption.

$$\frac{\dot{M}_F}{M} = \frac{\frac{RT\dot{V}}{V} - p \frac{\partial u}{\partial p} + \frac{1}{M} \sum_j \dot{Q}_j - C \cdot B}{u - h_F + D \frac{\partial u}{\partial F} - C \left(1 + \frac{D \partial R}{R \partial F} \right)} \quad (2.43)$$

with

$$B = \frac{\dot{p}}{p} - \frac{\dot{p}}{R} \frac{\partial R}{\partial p} + \frac{\dot{V}}{V} \quad (2.44)$$

$$C = T \frac{\partial u}{\partial T} \left(1 + \frac{T}{R} \frac{\partial R}{\partial T} \right) \quad (2.45)$$

$$D = \frac{(1 + f_s \cdot F_o)M}{f_s M_o} \quad (2.46)$$

where F represents the equivalence ratio; f_s represents the stoichiometric fuel/air ratio; h_F represents the absolute fuel enthalpy; M represents the instantaneous mass in the cylinder; $\dot{M}_F$ represents the mass of burning rate; $\dot{Q}_j$ represents the instantaneous heat transfer rate through surface i ; p represents the pressure; $\dot{p}$ represents the rate of pressure change; R represents the gas constant; T represents the temperature; u represents the absolute internal energy; V represents the volume; $\dot{V}$ represents the rate of volume change; and o represents the condition prior to injection.

Winterbone et al., 1977 proposed a crank angle resolution model for turbocharged diesel engines using the 'filling-and-emptying' method and empirical feedback to estimate transient responses. The developed model, which was experimentally validated, was employed to assess the linearized transfer function of the diesel engine for control studies, including fuel consumption control. Equation 2.47 represents the mass of fuel injected per cycle and can be derived from the paper's constant pressure combustion formula.

$$m_f = (m_f)_{cp} + (m_f)_{cv} \quad (2.47)$$

where m_f is the mass of fuel injected per cycle; $(m_f)_{cv}$ represents the mass of fuel burned at constant volume; and $(m_f)_{cp}$ represents the mass of fuel burned at constant pressure. Yet, the model requires simplification to be applicable for vehicle control applications.

Mean Value-Based Models

Mean value-based modelling is the other subcategory of the state variable value-based modelling classification. Mean value-based models produce the same outputs as crank-angle resolution models but they are based on the mean value of the state variables within the vehicle powertrain system. Kao and Moskwa, 1995 discussed the mean value modelling of diesel engines, its average value of states characteristic, as well as its simplicity and ease of operation.

Hendricks, 1997 proposed a mean value diesel engine's dynamics model as a further development of his previous model MVEP (Hendricks, 1986). This model outlines the interactions between inputs (such as crankshaft angular speed and throttle valve angle) and outputs (such as gasoline mass flow). In his first paper the author presented the formula represented by equation 2.48 for fuel mass flow.

$$\dot{m}_f = k_{mf} Y \Omega \quad (2.48)$$

where Y represents the index or fuel rack position; ω represents the engine shaft speed; and k_{mf} is a constant. Equation 2.49 represents the formula of fuel mass flow into intake port used in his latter paper.

$$\dot{m}_f = \dot{m}_{fv} + \dot{m}_{ff} \quad (2.49)$$

where $\dot{m}_{fv}$ represents the fuel vapour mass flow and $\dot{m}_{ff}$ represents the fuel film mass flow. It is based partly on fundamental physical and thermodynamic principles and aims to predict the mean values of heavy-duty diesel engine variables, including volumetric efficiency and exhaust manifold pressure. Hendricks' studies were made as an alternative to analytical modelling i.e. as crank-angle resolution models, as they exhibit long computational time and a large numbers of fitting parameters.

Karmiggelt, 1998 presented a hybrid control technique based on online estimation of engine output torque using a MVEM. The model requires engine speed, throttle valve position, and intake manifold pressure as inputs, and provides engine output torque and fuel consumption as outputs. In this study, Karmiggelt estimates fuel consumption based on his proposed fuel consumption formula represented by equation 2.50.

$$Q_{fc} = \frac{Q_{ac}}{\lambda_{ECU} L_{st}} \quad (2.50)$$

where Q_{fc} represents the fuel consumption; Q_{ac} represents the air mass flow into cylinders; λ_{ECU} represents the air/fuel equivalence determined by the Electronic Control Unit; and L_{st} represents the stoichiometric air/fuel ratio. It was validated satisfactorily, demonstrating reasonable computational efficiency.

Hendricks and Sorenson, 1990 developed a groundbreaking mean value non-linear dynamic model encompassing all subsystems of a four-cycle SI engine, including the fuel injection and exhaust systems. The authors utilized the same formula to calculate fuel mass flow represented by equation 2.49. The model demonstrated approximately identical steady-state accuracy as much as for quick transients with a 2% error. Yet, it applies solely to SI engines. Hendricks, 2001 introduced a seminal adiabatic mean value model for both SI and diesel engines. The suggested adiabatic mean value model accurately captured the performance of Exhaust Gas Recirculation (EGR) engines, such as fuel consumption rate and emission rates, as well as its dynamics. The estimation of the fuel consumption rate is based on the formula of the fuel mass flow represented by equation 2.51.

$$m_f = \frac{m_{ap} m_{EGR}}{\lambda L_{th}} \quad (2.51)$$

where m_{ap} represents the port air mass flow; m_{EGR} represents the EGR mass flow; λ is equal to one and represents the air/fuel ratio; and L_{th} represents the stoichiometric air/fuel ratio. Although the results of the model are more accurate than those of conventional isothermal MVEMs, it does not consider delays.

Wahlström, 2006 proposed a mean value model for a turbocharged diesel engine to reduce emissions from heavy-duty cars by coordinating the control of the EGR valve and VGT position. They noted that all the major aims of control through modelling of this study were achieved. Specifically, the study's goals were to meet the legal pollution

standards, minimize fuel consumption, and ensure the turbocharger's safe functioning. The formula used to calculate the mass flow of fuel is presented by equation 2.52.

$$W_f = \frac{10^{-6}}{120} u_\delta n_e n_{cyl} \quad (2.52)$$

where W_f represents the mass flow of fuel; u_δ represents the injected amount of fuel; n_e represents the rotational engine speed; and n_{cyl} represents the number of cylinders. Although the model was successfully validated, it is applicable only to diesel engines.

2.2.5 Number of Dimensions-based Modelling

The last category of vehicle fuel consumption modelling is the number of dimensions-based modelling. Jung and Assanis, 2001 proposed zero/one dimensional/single zone, quasi dimensional, and multi-dimensional/multi-zone modelling, as the subcategories of this classification.

Zero/One Dimensional/Single Zone Models

According to Foster, 1985, the zero/one dimensional/single zone modelling implies that the cylinder remains uniform in both composition and temperature throughout the cycle, leading to high computational efficiency. Thus, in case high computational efficiency is considered a major priority, researchers tend to prefer zero/one dimensional/single zone modelling to other types of modelling. This type of modelling is regularly used to model vehicle powertrain fuel consumption rates.

Schmitt et al., 2009 demonstrated how a zero-dimensional engine model can enhance the engine control law design for a turbocharged diesel engine equipped with an EGR system. The simplicity of the zero-dimensional model allows it to accurately estimate the start of combustion with minimal computational time, a conclusion also reinforced by Aceves et al., 2001. This model aids in the development of diesel engine controllers that comply with the Euro VI pollutant criteria while preserving the fuel consumption advantage of diesel engines over SI engines. The combustion process is often described with the Mass Fraction of Burned fuel (MFB) to the total injected fuel mass. The formula that estimates the MFB is represented by equation 2.53.

$$MFB = X_1 \left(\frac{2(X_2 + (1 - X_2)u)}{1 - \exp(X_3(-u + X_4))} \right) \quad (2.53)$$

where u describes the Crank Angle evolution between IVC and EVO and X_1 , X_2 , X_3 and X_4 are learnable parameters of an Artificial Neural Network.

Liu and Chen, 2009 proposed a zero-dimensional combustion model for optimizing SI engine knock. The model aids in optimizing fuel consumption rates and fuel injection timing using a three-zone approach (unburned, burning, and burned zones). The authors of this paper also utilize MFB as represented by equation 2.54.

$$x_b = x_{\text{flame}} + x_{\text{auto-ignition}} \quad (2.54)$$

where x_{flame} represents the mass burned by flame propagation in the model, which is calculated by the Weibe function (Heywood, 1988) and $x_{\text{auto-ignition}}$ represents the burned

mass in the unburned zone is due to auto ignition only. A modified version of Tanaka's reduced chemical kinetic model for commercial gasoline fuel was used in both burned and unburned zones, integrated with the Loughborough University Chemical Kinetics Simulation (LUCKS) code. However, the model's accuracy is relatively low.

Quasi Dimensional Models

Quasi dimensional modelling is the next subcategory of the number of dimensions-based modelling classification to report. It is an intermediate step between zero-dimensional and multi-dimensional models, which is the following subcategory, as it achieves balance between accuracy and computational efficiency. According to Austen and Lyn, 1962 and Whitehouse and Sareen, 1974, the quasi-dimensional modelling can offer the spatial information needed to predict emission products and requires substantially fewer computing resources than multidimensional models, yet some researchers have adopted it to calculate fuel consumption amongst others.

Bi et al., 1999 proposed a quasi-dimensional multi-zone model for diesel spray combustion, incorporating most of the physical processes which affect fuel consumption and emissions. The multi-zone, which will be better discussed in the following section divides the fuel spray into various zones. As a result, the fuel amount in the entire spray is the sum of zone fuel amount. Equation 2.55 represents the fuel amount contained within each zone is calculated with the fuel pressure, cylinder pressure and specifications of the injection nozzle.

$$m_f = C_d A \rho_l \sqrt{\frac{2\Delta p}{\rho_l}} \Delta t \quad (2.55)$$

where A represents the across area of nozzle hole; ρ_l represents the density of liquid fuel; Δ_p represents the pressure difference across the injector hole; Δ_t represents the calculation time step; and C_d represents the nozzle coefficient of discharge which is assumed to remain constant throughout the injection period. The model balances accuracy and computational efficiency but fails to account for significant spatial variations in mixture composition and temperature within the cylinder charge. Similarly, P. Zhou et al., 2006 developed an experimentally validated quasi-dimensional multi-zone combustion model for DI diesel engines, which concerns the fuel evaporation process.

Multi-Dimensional/Multi-zone Models

The last subcategory of the number of dimensions-based modelling classification is multi-dimensional modelling which is better known as multi-zone modelling. Multi-zone modelling is preferred for its emphasis on accuracy, making it a popular choice among researchers for modelling vehicle powertrain fuel consumption and emissions rates. According to Bracco, 1985, Amsden et al., 1985 and Varnavas and Assanis, 1996 multi-zone models consider spatial variations in mixture composition and temperature, both of which are crucial for predicting exhaust emissions. Albeit there are papers that have adopted multi-zone modelling and concluded about what influences the rate of fuel consumption, such as Delhaye and Taklanti, 1999, Daw et al., 1998 and Komninou et al., 2005, they do not deal with the calculation of the fuel consumption or any substitute like fuel mass and air-to-fuel ratio.

Chapter 3

Problem Definition

In this chapter, we delve into the fundamentals of both our fuel consumption model and the problem's formulation. The purpose is to establish a clear framework regarding the principles of our fuel consumption model and to lay the foundation for a comprehensive understanding of the problem at hand. The matters discussed in this chapter will be crucial for subsequent sections, where we present the possible modifications and the behaviour of our algorithm.

3.1 Fuel Consumption Model

In this section, we present our fuel consumption model. Although our approach can be applied with any fuel consumption model, its behaviour greatly depends on the chosen model. We developed a straightforward analytical formula to calculate the locomotive's fuel usage over a specific time period using basic kinematics. Our model accounts for three categories of fuel consumption: instant consumption, acceleration consumption, and deceleration consumption. Each category is represented by a distinct formula, while the total fuel consumption integrates all three.

3.1.1 Instant Consumption

Instant consumption refers to the fuel consumed by the locomotive during a specific time period. Equation 3.1 represents the locomotive's instant consumption.

$$W_{t_1:t_2}^v = \int_{t_1}^{t_2} K_v dt \quad (3.1)$$

where $W_{t_1:t_2}^v$ represents the fuel consumption from t_1 to t_2 ; t_1 represents the initial time point of the locomotive's travel; t_2 represents the final time point of the locomotive's travel; and K_v represents the consumption's weight at constant speed. For simplicity, in our study, we assume K_v to be 1. Equation 3.1 applies regardless of the train's throttle setting (whether it is cruising, accelerating, or decelerating). Equation 3.1 simplifies to equation 3.2 for constant speed.

$$W_{t_1:t_2}^v = K_v \cdot (t_2 - t_1) \quad (3.2)$$

3.1.2 Consumption of Acceleration

This category of consumption describes the fuel usage of the locomotive during a specific time period when the locomotive is positively accelerating. The locomotive's acceleration consumption is represented by equation 3.3.

$$W_{t_1:t_2}^a = K_a \cdot a \cdot (t_2 - t_1) \quad (3.3)$$

where $W_{t_1:t_2}^a$ represents the fuel consumption from t_1 to t_2 when the train is accelerating; a represents the acceleration rate; and K_a represents the weight of consumption during acceleration, set to 50. Its value is significantly greater than K_v to reflect the increased fuel consumption rate during acceleration.

3.1.3 Consumption of Deceleration

On the other hand, consumption of deceleration describes the fuel usage of the locomotive over a given time frame when the locomotive is decelerating. Equation 3.4 represents the locomotive's fuel consumption during deceleration.

$$W_{t_1:t_2}^b = K_b \cdot b \cdot (t_2 - t_1) \quad (3.4)$$

where $W_{t_1:t_2}^b$ represents the fuel consumption from t_1 to t_2 during deceleration; b represents the deceleration rate; and K_b represents the consumption weight when the train is decelerating, with $K_b = 0.5$. In comparison to K_a , K_b is less than K_v to simulate the train's reduced fuel consumption rate during deceleration.

3.1.4 Total Fuel Consumption Model

The total fuel consumption model is based on the equation for instant speed consumption, to which we add the equation for acceleration consumption and subtract the equation for deceleration consumption. In the rare instance where the value from equation 3.4 is negative, we consider it to be zero. In this scenario, the locomotive does not store any fuel; instead, it does not use any fuel while decelerating.

3.2 Problem Formulation

In this section of the study, we will discuss the formulation of the problem. Specifically, we examine the characteristics of railways, trains, speed profiles, problems, and solutions. The innovation of this study derives from the unique representation of the railways. We consider that a railway consists of multiple consecutive segments. Each track segment is described by its length and its speed limit (i.e., the maximum allowed speed). Additionally, we assume that there are no cycles in the railways, meaning that no segment is connected to a prior segment of the railway. The terms "railway" and "path" will be used interchangeably.

A train is defined by specific key parameters: its length, the maximum speed it can reach, its maximum acceleration rate, and its maximum deceleration rate. The train's speed is non-negative at all times, meaning that it cannot travel backwards. Furthermore,

we assume that a train has three distinct throttle settings, namely acceleration, cruising, and deceleration. The terms "throttle setting" and "mode" will be used interchangeably.

A speed profile is a list of consecutive states. A state refers to the status of a train and is described by the distance of the locomotive's front to the next segment, the speed it has reached at that moment, and the list of the segments it is on at the end of the state. For simplicity, a state can be described as (Distance, Speed, Segments), which is a tuple with its respective attributes.

An original speed profile is a speed profile produced by the algorithm described by Manolakis and Refanidis, 2023 over a single path. It is noteworthy that the authors proved that in a time-optimal the locomotive accelerates and decelerates at its maximum rates. Additionally, we assume that every original speed profile is valid (Manolakis and Refanidis, 2023). The terms "original" and "time-optimal" speed profile are used interchangeably.

A problem is described by six components: a train, a path, an initial state, a goal state, an original speed profile, and a positive amount of extra time. Given a problem, a solution is a fuel-optimal speed profile, which is a modification of the time-optimal speed profile. The modification is made according to the algorithms discussed in the next section of the study, utilizing the extra time.

Chapter 4

Methodology

This chapter outlines the methodology of our study, including a discussion of all the possible algorithms that can alter a speed profile and the implementation of our final approach. The algorithms discussed adjust instances of speed profiles rather than the entirety of them, while our implementation consists of only those approaches that minimize fuel consumption for each instance.

4.1 Algorithms

In this section of our study, we outline the algorithms we developed to determine the fuel-optimal speed profile along a designated path. Additionally, we provide various examples that demonstrate how the algorithms function in basic scenarios. The main algorithm consists of case-specific methods. These methods can alter up to three throttle settings at a time with respect to the conditions at the start and end of their respective scenarios. We will analyze the best response for every possible category of scenario. These scenarios are described as two or three continuous throttle settings. There are four categories of scenarios, as listed below.

1. Acceleration, Cruising and Acceleration,
2. Acceleration, Cruising and Deceleration,
3. Acceleration and Deceleration, and
4. Deceleration, Cruising and Deceleration

It is noteworthy that scenarios in which the locomotive initially decelerates and later accelerates, with or without an intervening cruising mode, cannot be modified. Specifically, the "Deceleration, Cruising and Acceleration" scenario cannot be changed while adhering to the speed limits. If the acceleration starts sooner, the rear of the train will violate the speed limit of the respective railway segment. Conceptually, the deceleration cannot end at a later time point because the front of the train will violate the speed limit of the segment it is on. It is also apparent that the start of the deceleration and

the end of the acceleration cannot be altered. On the other hand, the "Deceleration and Acceleration" scenario is not feasible. If the locomotive decelerates, it must first cruise before accelerating; otherwise, the rear part of the train would not adhere to the speed limit of the previous segment.

4.1.1 Acceleration, Cruising and Acceleration

In this scenario, the train initially accelerates, then cruises at a steady pace, and finally accelerates once more. There is no correlation between the two rates of acceleration. The evaluation of the different methods for addressing this scenario will be based on their solutions to the problem described in table 4.1. Figure 4.1 illustrates the fuel consumption over time for the time-optimal speed profile of the aforementioned problem. The total fuel consumption of the speed profile is 3872.5 units. This scenario can be addressed by two techniques, as described in the following subsections.

Parameter	Value
Train	Train 1 (Table 4.2)
Path Speed	Path 1 (Table 4.3)
Initial State	(1250, 0, [1]))
Goal State	(0, 75, [3]))
Original Speed Profile	Speed Profile 1 (Figure 4.2)
Extra Time	10 sec

Table 4.1: Problem 1

Parameter	Value
Length	150 m
Maximum Speed	75 m/s
Maximum Acceleration Rate	1 m/s ²
Maximum Deceleration Rate	1 m/s ²

Table 4.2: Train 1

Segment	Length (m)	Speed Limit (m/s)
1	1400	60
2	2225	50
3	1712.5	75

Table 4.3: Path 1 Layout

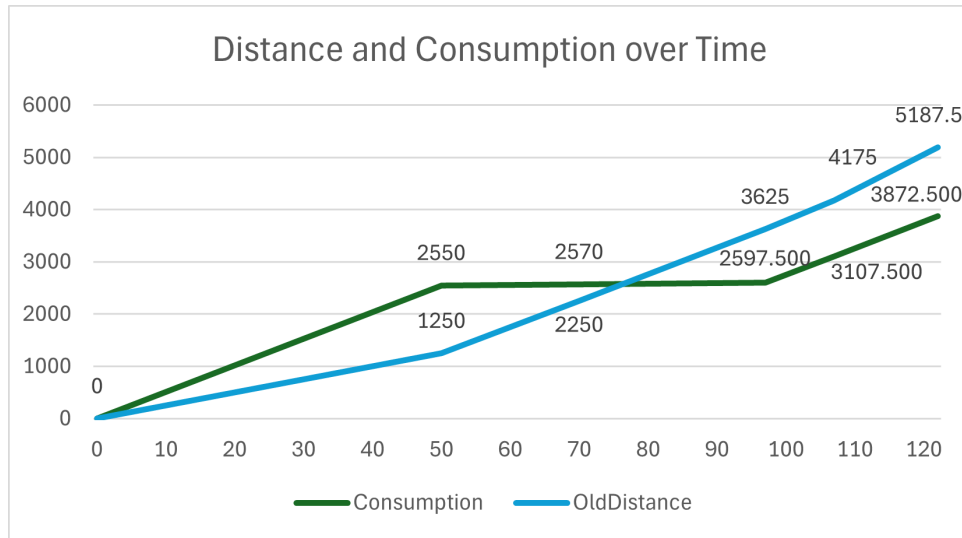


Figure 4.1: Consumption of Time-Optimal Speed Profile for Problem 1

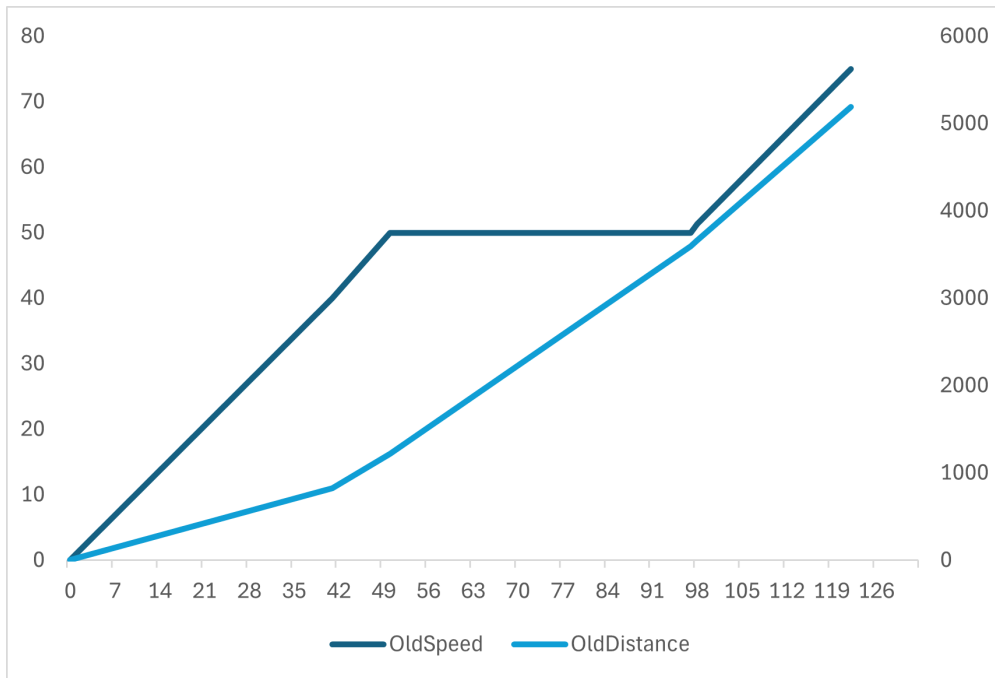


Figure 4.2: Time-Optimal Speed Profile for Problem 1

Slower Constant Speed

This technique reduces the speed of the cruising. Essentially, the locomotive starts cruising earlier at a lower speed than the time-optimal speed profile, shortening the first acceleration. The start of the second acceleration is delayed as much as possible to reach the same speed and travel the same distance as in the time-optimal speed profile, thereby extending the second acceleration. The acceleration rates are not modified in this method. Figure 4.3 illustrates a comparative graph of the speed and distance traveled over time for the original and altered speed profiles.

As demonstrated in figure 4.4, which depicts the fuel consumption and the distance covered over time for the adjusted speed profile, the "Slower Constant Speed" method is less efficient than the original, as it yields a speed profile with higher fuel consumption compared to the latter.

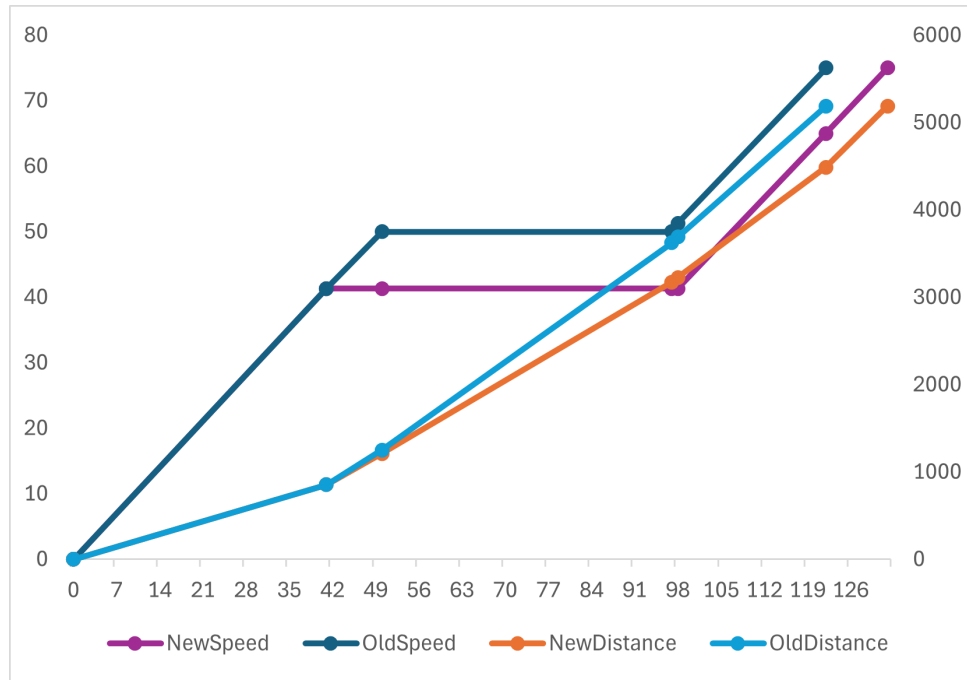
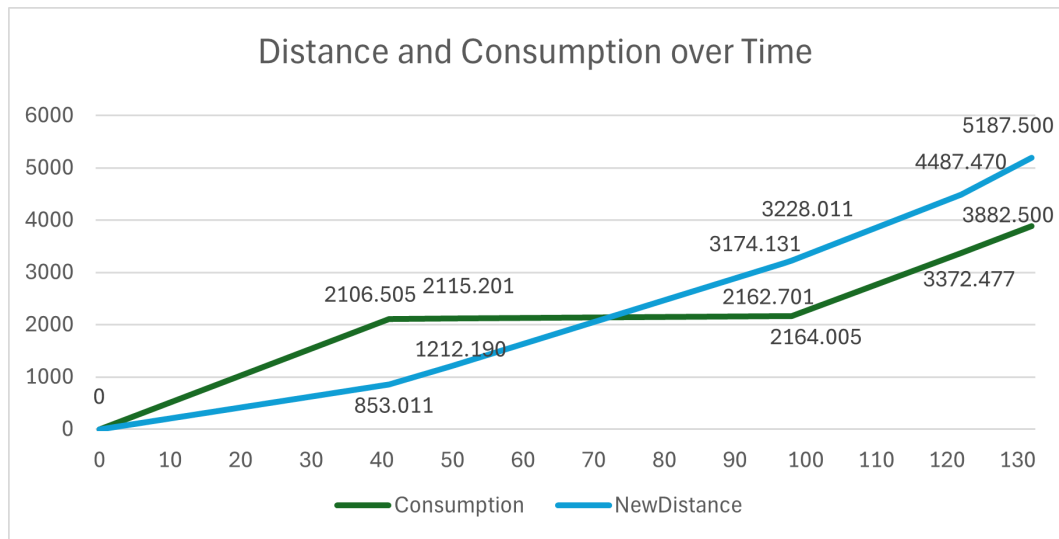


Figure 4.3: Acceleration, Cruising and Acceleration Method 1

Figure 4.4: Acceleration, Cruising and Acceleration Fuel Consumption of Method 1
Smoother Acceleration

By "Smoother Acceleration", we refer to a reduced acceleration rate. We can only modify the initial acceleration. Adjusting the second acceleration would mean starting the acceleration sooner; however, in the original speed profile, the acceleration starts at the last possible moment to adhere to the speed limit of the segments. Put another way, by adjusting the second acceleration, the locomotive would violate at least one segment's speed limit. By modifying the first throttle setting, the train accelerates more gradually until it reaches the cruising speed. Consequently, the cruising duration decreases, potentially to zero seconds. The train must not travel farther than it does at the start of the second acceleration in the time-optimal speed profile. If the cruising duration decreases to zero seconds, the speed profile will consist of two continuous accelerations with no cruising. Figure 4.5 shows a comparative graph of the speed and the traveled distance for the original and new speed profiles over time.

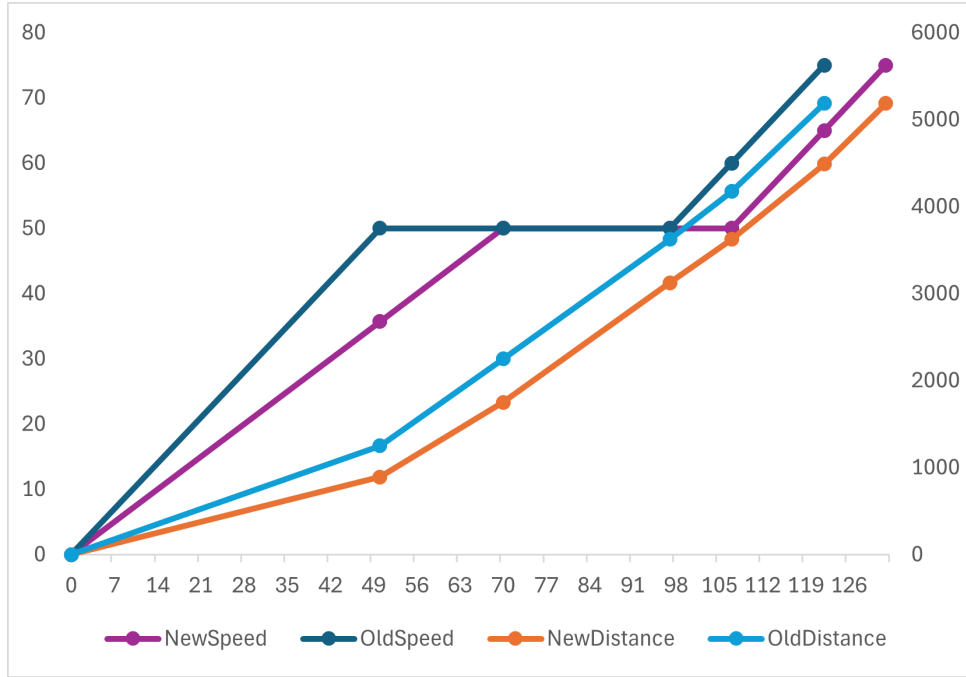


Figure 4.5: Acceleration, Cruising and Acceleration Method 2

Figure 4.6 illustrates the locomotive's covered distance and fuel consumption over time. It is apparent that the "Smoother Acceleration" technique is less fuel-efficient compared to the original speed profile. As a result, it is not preferred in this scenario.

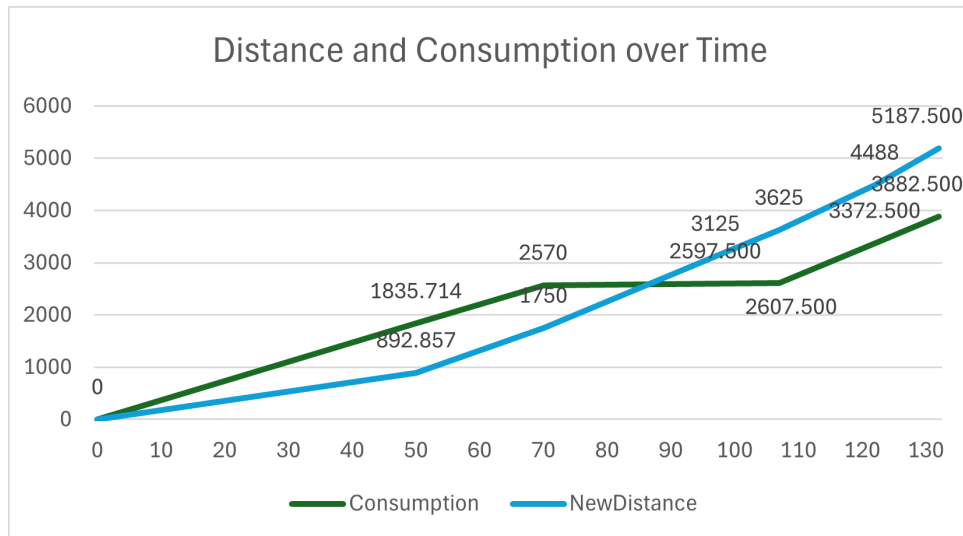


Figure 4.6: Acceleration, Cruising and Acceleration Fuel Consumption of Method 2

4.1.2 Acceleration, Cruising and Deceleration

The locomotive in this scenario initially accelerates, then cruises at a constant speed, and finally decelerates. The three techniques for addressing this scenario, outlined in the following subsections, are evaluated based on their solutions to the problem detailed in table 4.4. Figure 4.8 shows the fuel consumption over time for the time-optimal speed profile of the example. The overall fuel consumption of the time-optimal speed profile is 2617.5 units.

Parameter	Value
Train	Train 1 (Table 4.2)
Path Speed	Path 2 (Table 4.5)
Initial State	(1500, 0, [1]))
Goal State	(0, 10, [3]))
Original Speed Profile	Speed Profile 2 (Figure 4.7)
Extra Time	14.375 sec

Table 4.4: Problem 2

Segment	Length (m)	Speed Limit (m/s)
1	1400	70
2	2225	50
3	1350	60

Table 4.5: Path 2 Layout

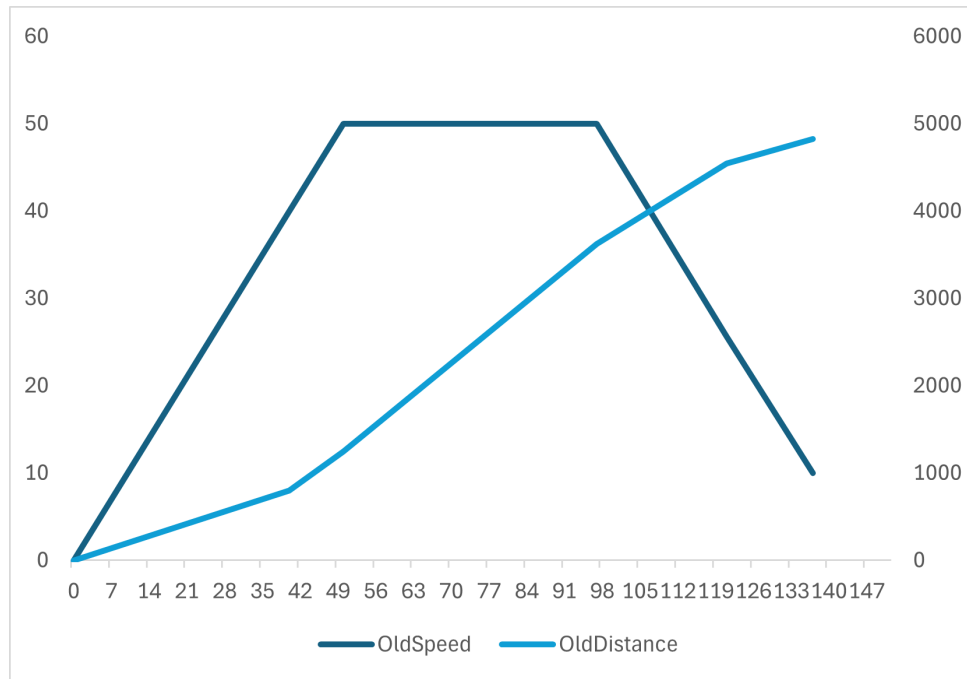


Figure 4.7: Time-Optimal Speed Profile for Problem 2

Smoother Acceleration

The train accelerates more smoothly until it reaches the cruising speed later than in the original speed profile. Consequently, the cruising duration decreases, potentially to zero seconds. Once the locomotive attains the cruising speed, the remainder of the speed profile is the same, albeit delayed. Figure 4.9 showcases the outcome of this method.

Figures 4.10 demonstrates the train's traveled distance and fuel consumption over

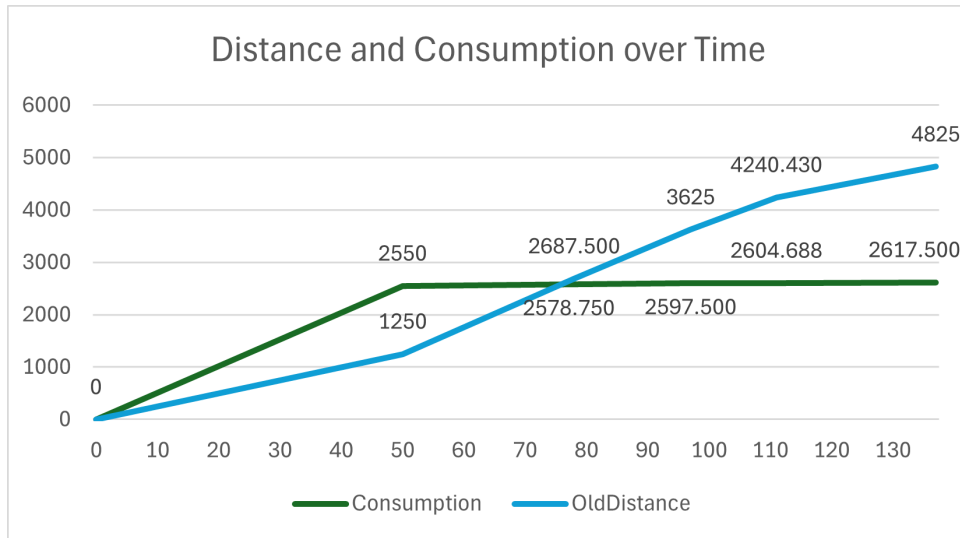


Figure 4.8: Consumption of Time-Optimal Speed Profile for Problem 2

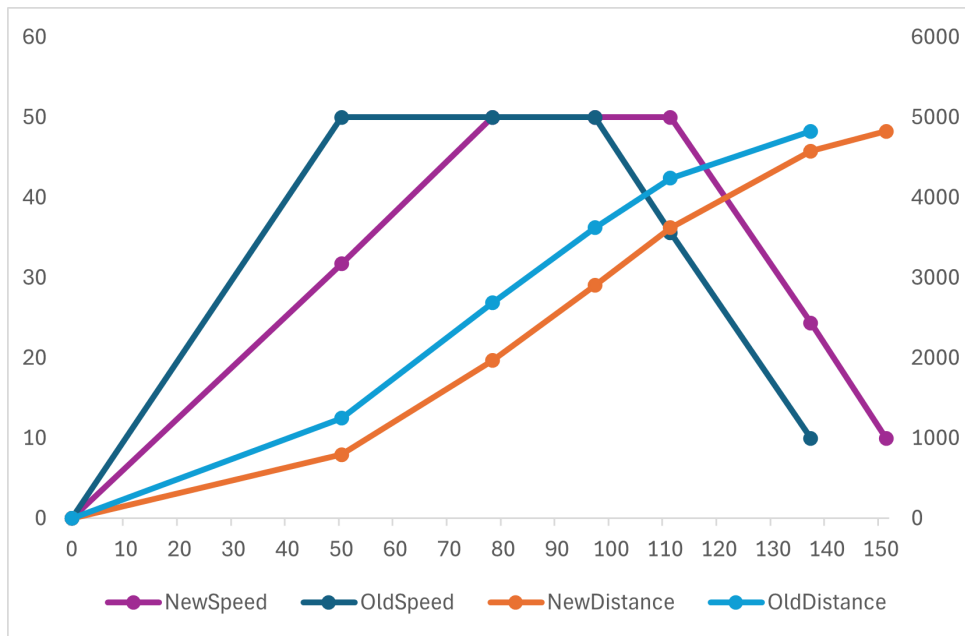


Figure 4.9: Acceleration, Cruising and Deceleration Method 1

time. The "Smoother Acceleration" method produces a more fuel-efficient speed profile than the original speed profile, yet "Slower Constant Speed" proves to be the best.

Given sufficient time, the locomotive can reach the required speed after covering the same distance as at the start of the deceleration in the original speed profile. In this instance, the modified speed profile has no cruising, and the acceleration and deceleration are continuous. This scenario is better described by figure 4.11.

Slower Constant Speed

In this technique, the locomotive begins cruising earlier, at a lower speed than the time-optimal speed profile, and delays the deceleration as much as possible to reach the same distance and speed as at the start of the deceleration in the original speed profile. The acceleration and deceleration rates remain unmodified. Figure 4.12 illustrates the functionality of this method.

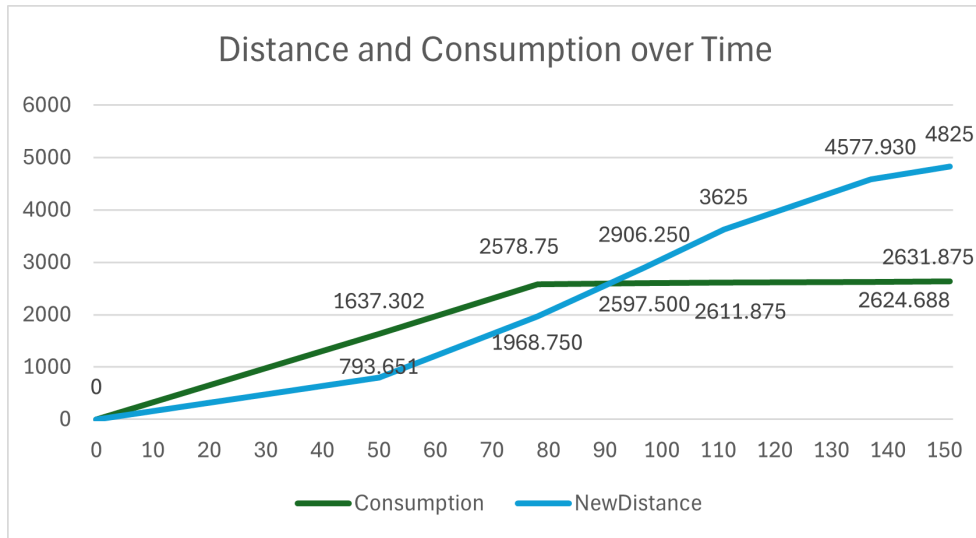


Figure 4.10: Acceleration, Cruising and Deceleration Fuel Consumption of Method 1

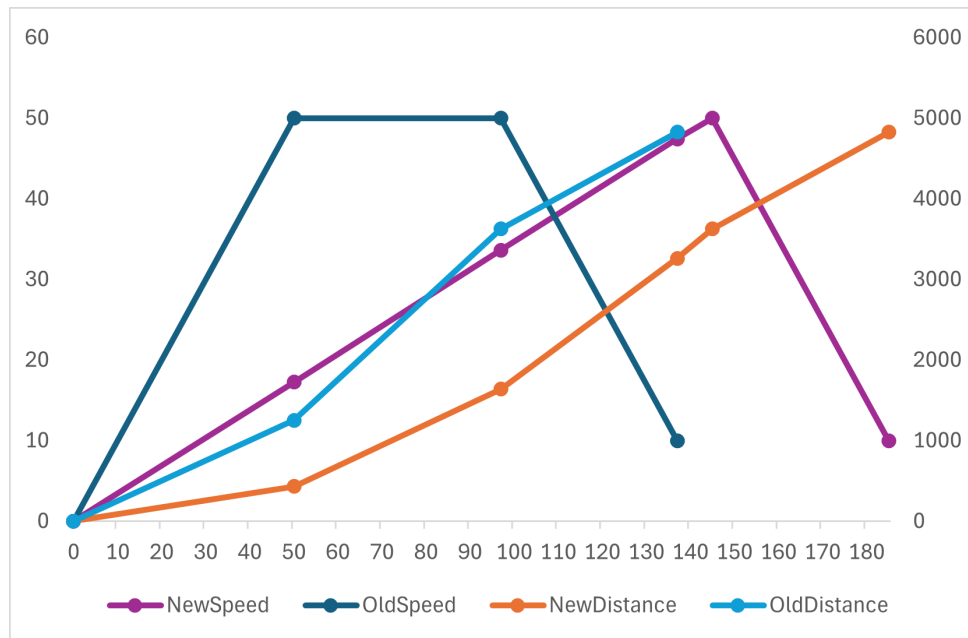


Figure 4.11: Acceleration, Cruising and Deceleration Method 1 Special Case

Figure 4.13 illustrates the train's covered distance and fuel consumption over time. It is evident that the "Slower Constant Speed" method is the best technique in this scenario, as it produces the speed profile with the least fuel consumption.

Smoother Deceleration

This method adjusts the deceleration by reducing its rate. The locomotive decelerates more smoothly until it reaches the final speed of the deceleration at a later time than in the time-optimal speed profile. Consequently, the duration of the cruising decreases, potentially to zero seconds. In such a scenario, there would be no cruising, and the two other throttle settings would be continuous. Figure 4.14 illustrates a comparative graph of the speed and the traveled distance for the original and time-optimal speed profiles over time.

Figure 4.15 presents the locomotive's traveled distance and fuel consumption over

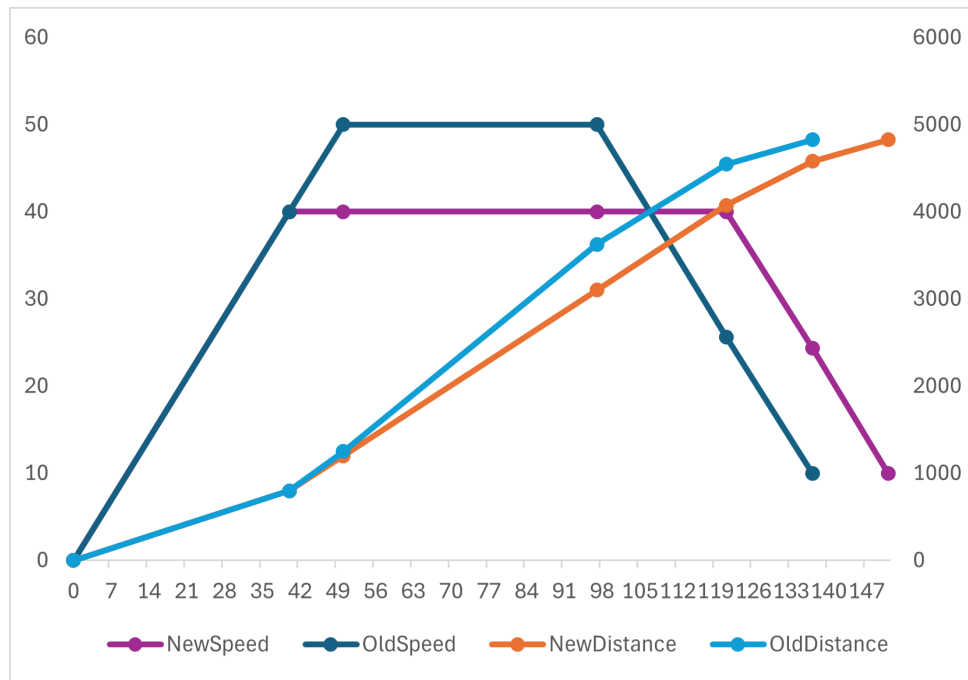


Figure 4.12: Acceleration, Cruising and Deceleration Method 2

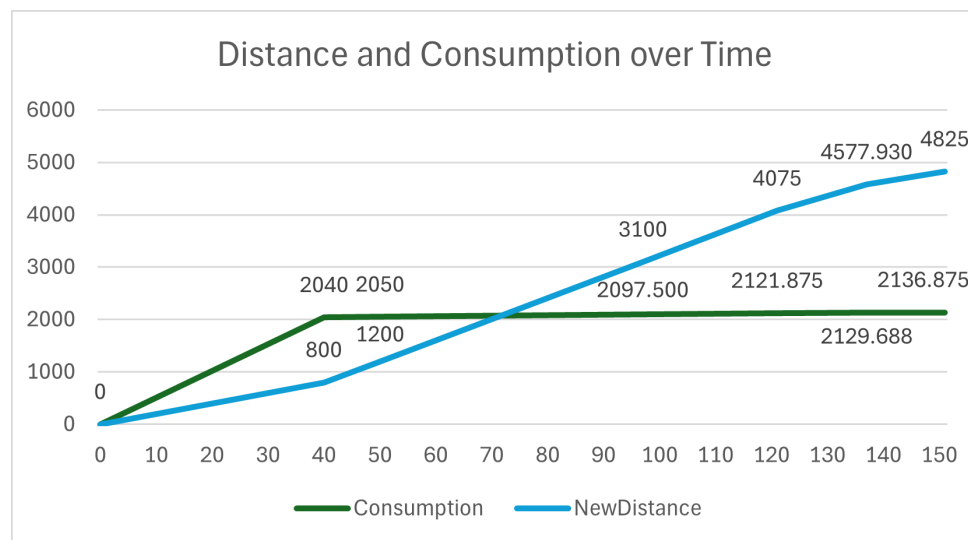


Figure 4.13: Acceleration, Cruising and Deceleration Fuel Consumption of Method 2 time. It is apparent that the "Smoother Deceleration" technique produces a less fuel-efficient speed profile compared to the original speed profile.

4.1.3 Acceleration and Deceleration

This scenario assumes the locomotive accelerates and decelerates without any intervening throttle settings. There are three approaches to handling this scenario, which are detailed in the following subsections. The evaluation of these methods will be based on their solutions to the problem in table 4.6. Figure 4.17 demonstrates the fuel consumption over time for the time-optimal speed profile of the example. The speed profile consumes a total of 2570 units of fuel.

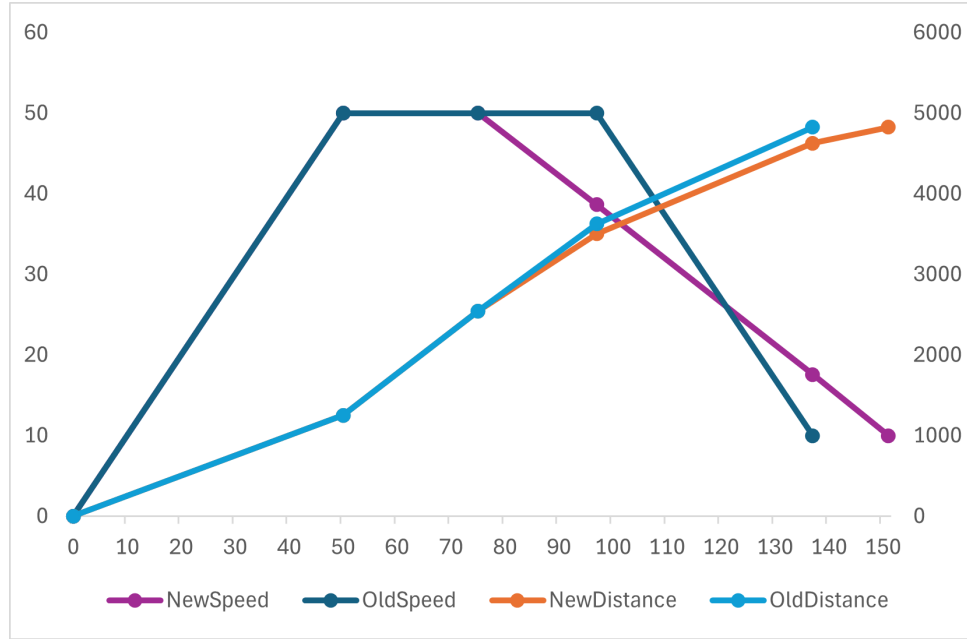


Figure 4.14: Acceleration, Cruising and Deceleration Method 3

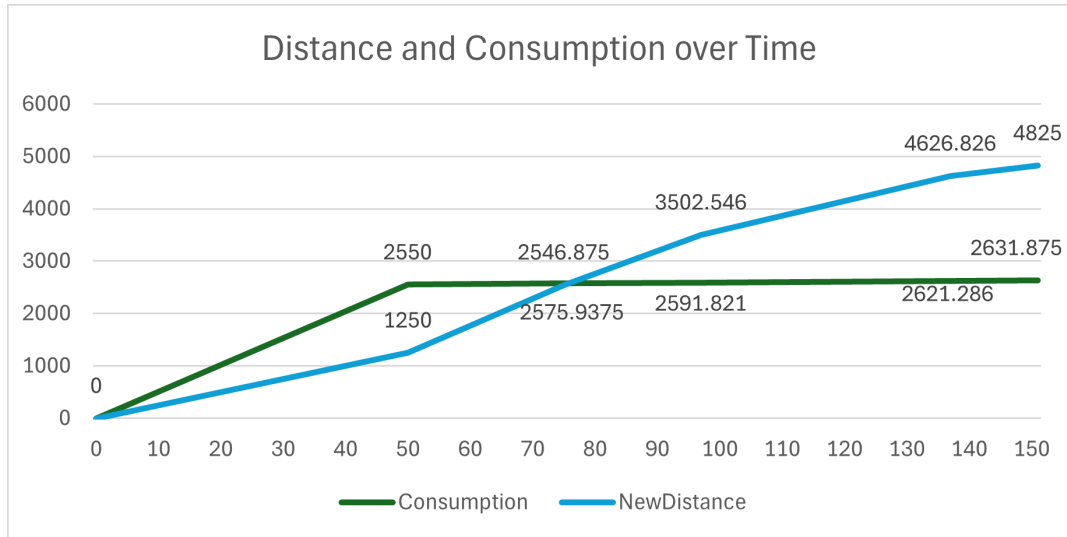


Figure 4.15: Acceleration, Cruising and Deceleration Fuel Consumption of Method 3

Parameter	Value
Train	Train 1 (Table 4.2)
Path Speed	Path 3 (Table 4.7)
Initial State	(2450, 0, [1]))
Goal State	(0, 10, [1]))
Original Speed Profile	Speed Profile 3 (Figure 4.16)
Extra Time	10 sec

Table 4.6: Problem 3

Segment	Length (m)	Speed Limit (m/s)
1	2600	100

Table 4.7: Path 3 Layout

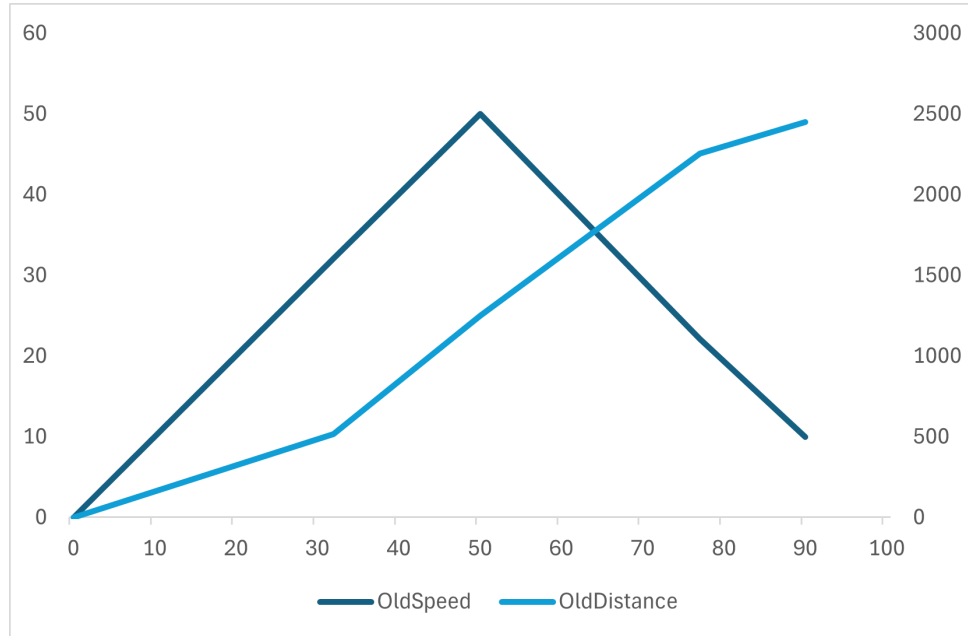


Figure 4.16: Time-Optimal Speed Profile of Problem 3

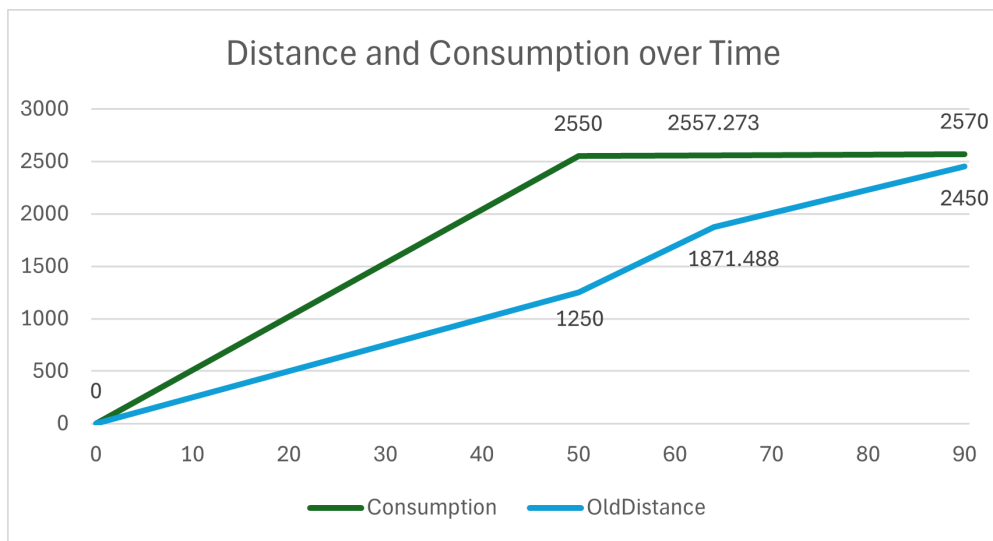


Figure 4.17: Consumption Time-Optimal Speed Profile of Problem 3

Smoother Acceleration

The train gradually accelerates until it travels a distance that is also covered in the time-optimal speed profile during deceleration. Simultaneously, it must reach the speed at which this distance was covered in the original speed profile. If there is a significant amount of extra time, the acceleration may transform into a deceleration; however, it is unlikely. The remaining part of the speed profile, once the locomotive reaches the target speed and distance, is identical to that of the time-optimal speed profile with delay. Figure 4.18 illustrates this method's result.

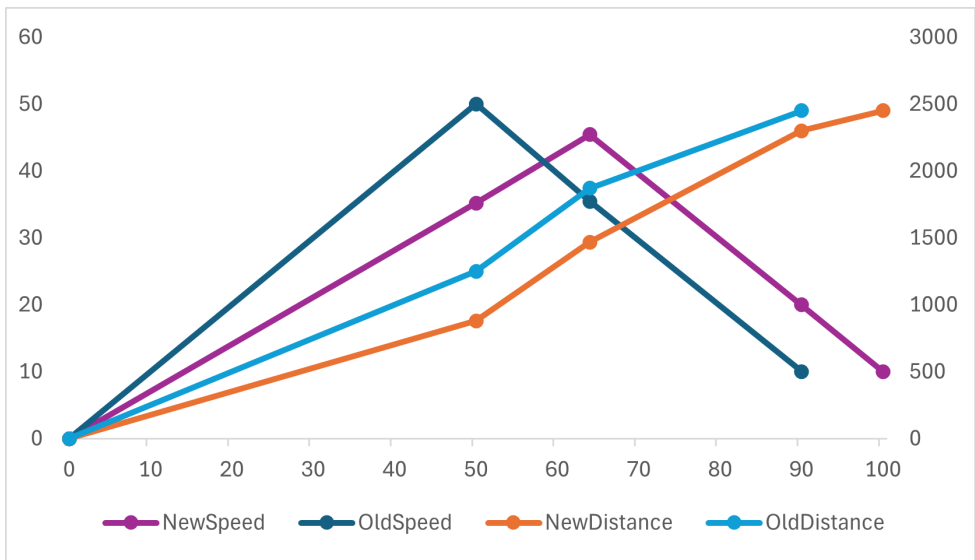


Figure 4.18: Acceleration and Deceleration Method 1

Figure 4.19 illustrates the locomotive's distance traveled and fuel consumption over time. The "Smoother Acceleration" technique produces a better speed profile than the original speed profile, in terms of fuel consumption. Yet, the "Addition of Cruising" method, described in the following section, proves to be even better.

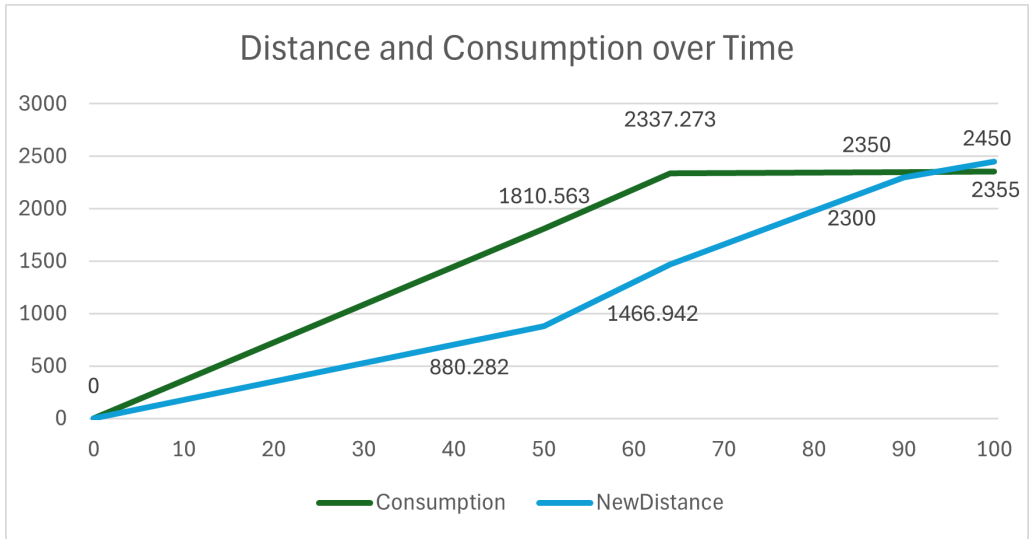


Figure 4.19: Acceleration Deceleration Fuel Consumption of Method 1

Addition of Cruising

In this technique, the train starts cruising at a speed lower than the peak speed of the original speed profile and continues until it covers the same distance and reaches the same speed as in the time-optimal speed profile where it decelerates. In the scenario of abundant extra time and positive initial and goal speed, at least one of the throttle settings may be replaced by a single cruising. Figure 4.20 illustrates the method's outcome.

Figure 4.21 depicts the locomotive's distance traveled and fuel consumption over time. The "Addition of Cruising" method is the best technique for this scenario, as it produces the speed profile with the least fuel consumption among all the approaches.

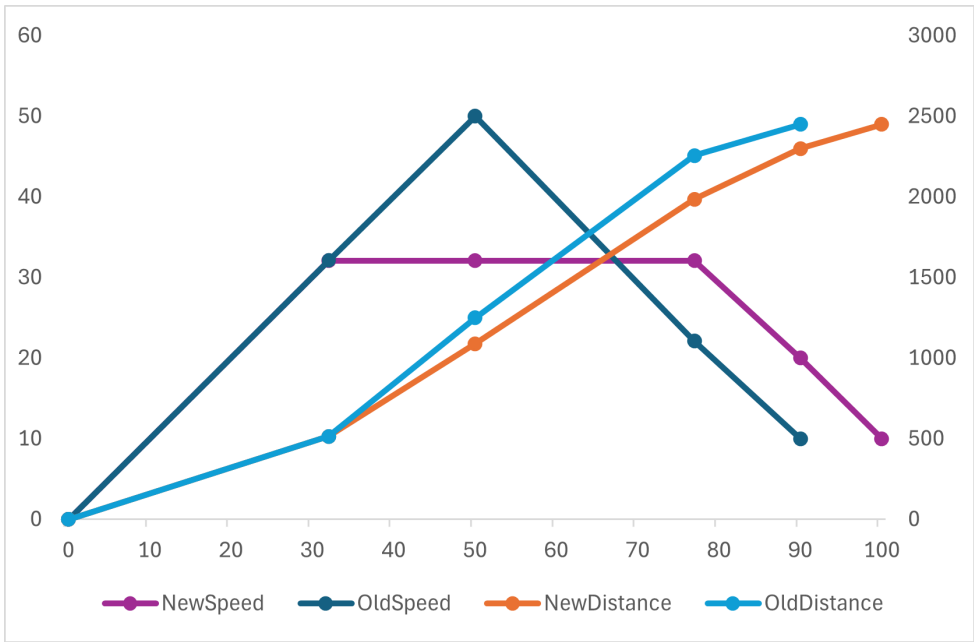


Figure 4.20: Acceleration and Deceleration Method 2

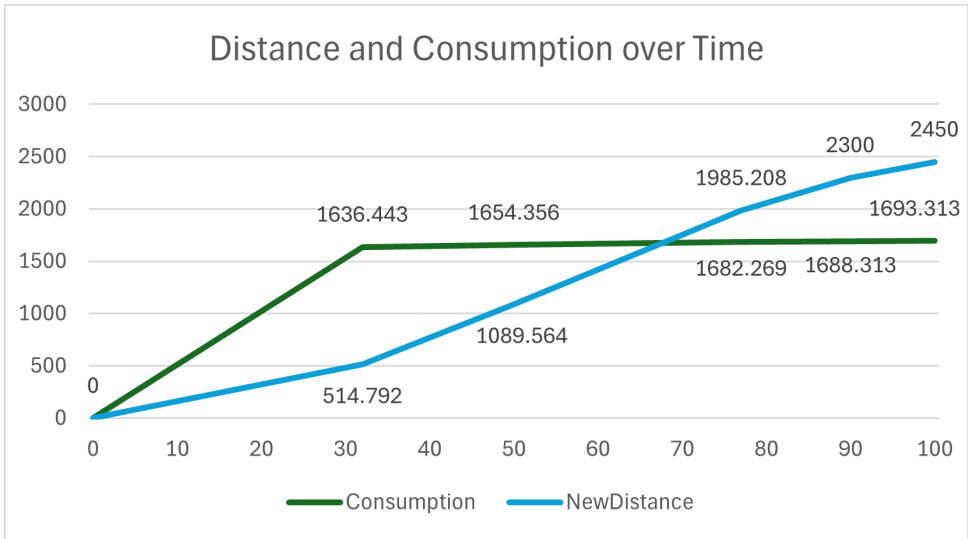


Figure 4.21: Acceleration and Deceleration Fuel Consumption of Method 2

Smoother Deceleration

The locomotive decelerates more smoothly until it reaches the final speed of the deceleration in the original speed profile, but at a later time. Consequently, the deceleration begins sooner than before, shortening the acceleration. In the scenario of abundant extra time, both throttle settings may be replaced by a single cruising. Additionally, this scenario requires the initial and final speeds to be both positive and equal. Figure 4.22 illustrates the discussed technique.

Figure 4.23 illustrates the locomotive's traveled distance and fuel consumption over time. The "Smoother Deceleration" technique yields a better speed profile than the original in terms of fuel efficiency. However, it is less efficient than the "Addition of Cruising" method.

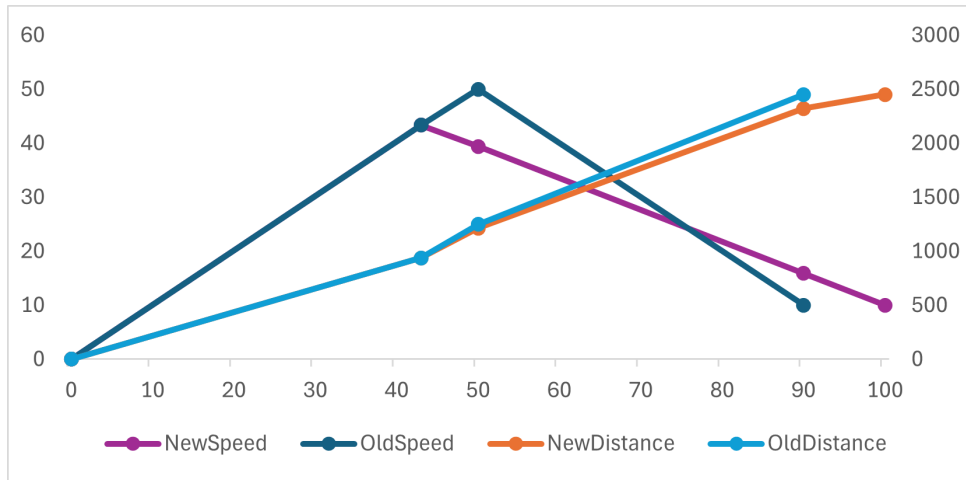


Figure 4.22: Acceleration and Deceleration Method 3

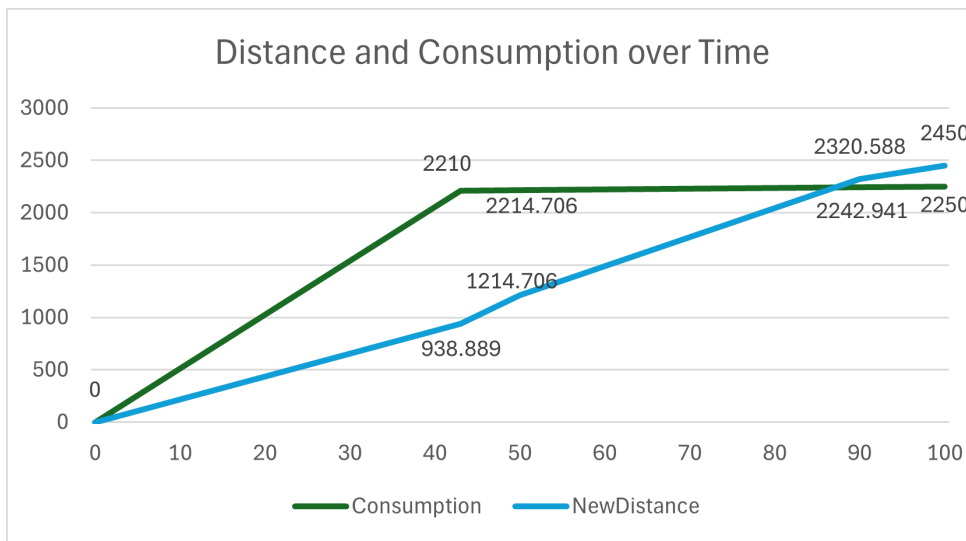


Figure 4.23: Acceleration and Deceleration Fuel Consumption of Method 3

4.1.4 Deceleration, Cruising and Deceleration

This scenario assumes that the train decelerates, then cruises at a constant speed, and finally decelerates again. The two deceleration rates are not correlated. The evaluation of the methods for addressing this scenario is based on their solutions to the problem described in table 4.8. Figure 4.25 demonstrates the fuel consumption over time for the time-optimal speed profile of the example. The total fuel consumption of the speed profile is 105 units. The following subsections outline the two techniques for addressing this scenario.

Smoother Deceleration

This method pertains only to the last deceleration, since altering the first deceleration would cause speed limit violations as already explained. The locomotive decelerates more gradually until it reaches the final speed of the deceleration in the original speed profile at a later time. At that moment, the distance covered must be equal to the total distance traveled in the time-optimal speed profile. Figure 4.26 depicts the behaviour of the "Smooth Deceleration" method for this scenario.

Parameter	Value
Train	Train 1 (Table 4.2)
Path Speed	Path 3 (Table 4.9)
Initial State	(1562.5, 0, [1]))
Goal State	(0, 10, [2]))
Original Speed Profile	Speed Profile 4 (Figure 4.24)
Extra Time	5 sec

Table 4.8: Problem 4

Segment	Length (m)	Speed Limit (m/s)
1	1712.5	90
2	6475	50

Table 4.9: Path 4 Layout

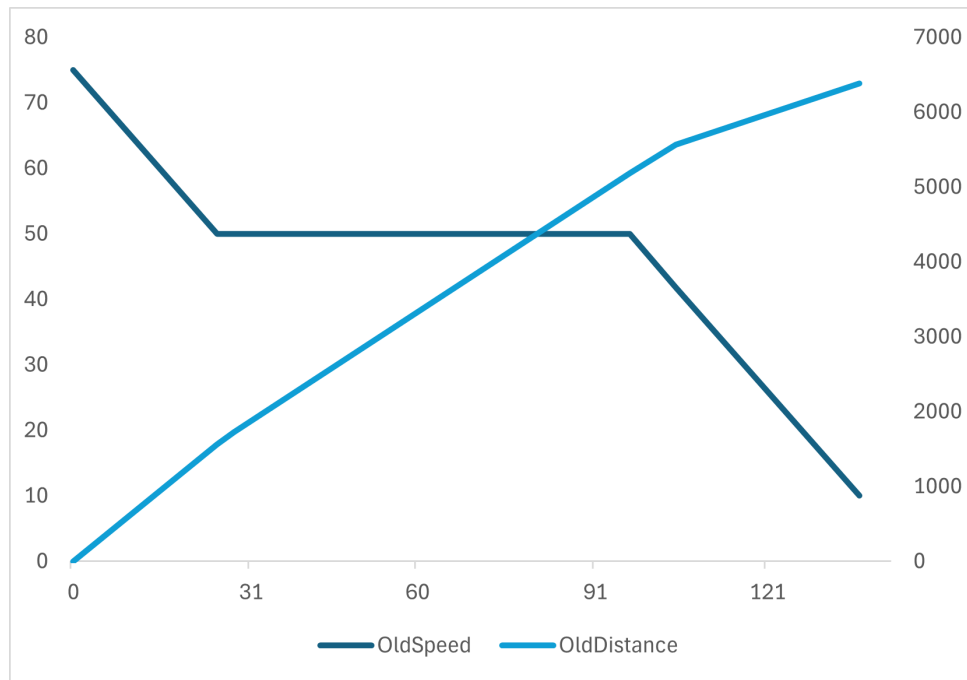


Figure 4.24: Time-Optimal Speed Profile of Problem 4

Figure 4.27 presents the fuel consumption and the speed of the locomotive over time for the adjusted speed profile. It is evident that the "Smoother Deceleration" method produces a less fuel-efficient speed profile, as the train consumes more fuel than in the time-optimal.

Slower Constant Speed

In this technique, the locomotive starts cruising later at a slower speed than in the time-optimal speed profile by prolonging the duration of the initial deceleration. Once the locomotive covers the same distance as in the original speed profile at the new cruising

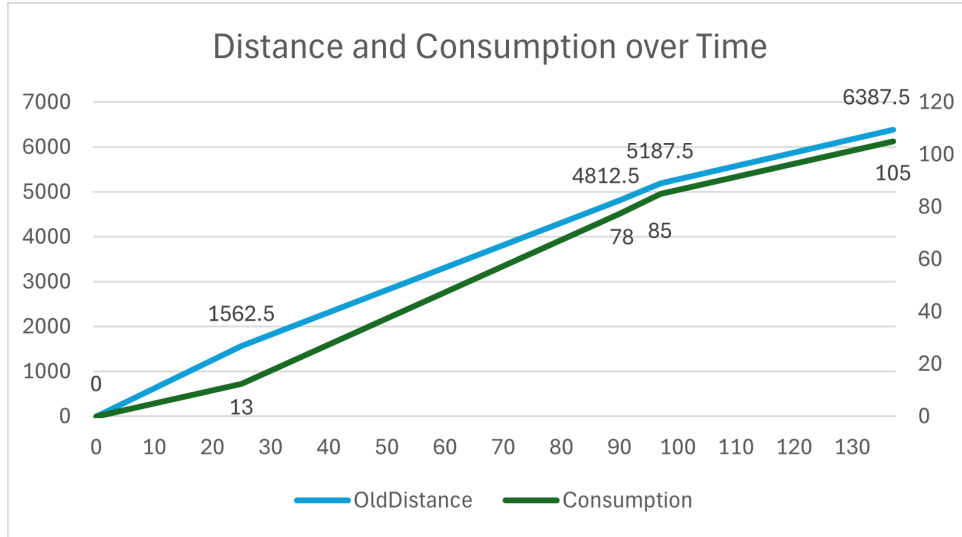


Figure 4.25: Fuel Consumption of Time-Optimal Speed Profile of Problem 4

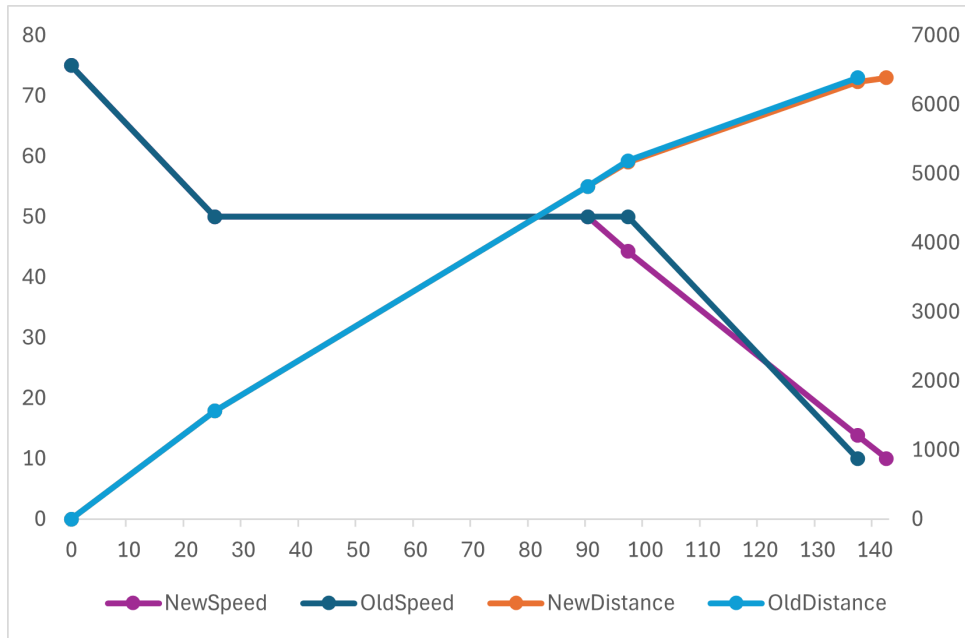


Figure 4.26: Deceleration, Cruising and Deceleration Method 1

speed, it decelerates to the final speed. The remaining modified speed profile is identical to the time-optimal speed profile but with delay. Figure 4.28 illustrates the method's outcome.

Figure 4.29 presents the fuel consumption and the speed of the locomotive over time for the modified speed profile. Similar to the "Slower Constant Speed" technique, this method produces a speed profile with higher fuel consumption than the time-optimal speed profile's.

4.2 Implementation

This section follows up on the previous discussion by detailing the code that implements the algorithms that minimize fuel consumption. As we showcased in the previous section, the only speed profile scenarios that can be modified to be more fuel-efficient are the

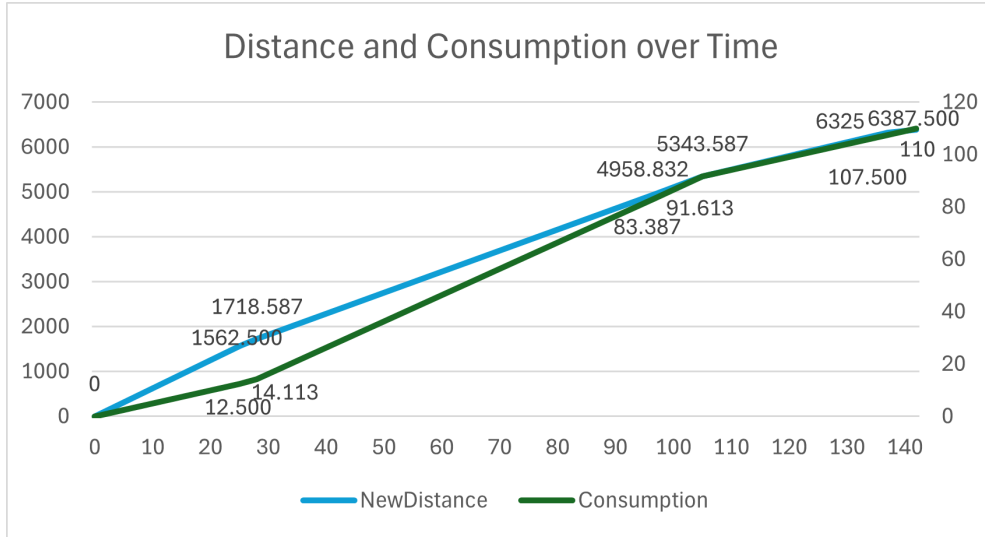


Figure 4.27: Deceleration, Cruising and Deceleration Fuel Consumption of Method 1

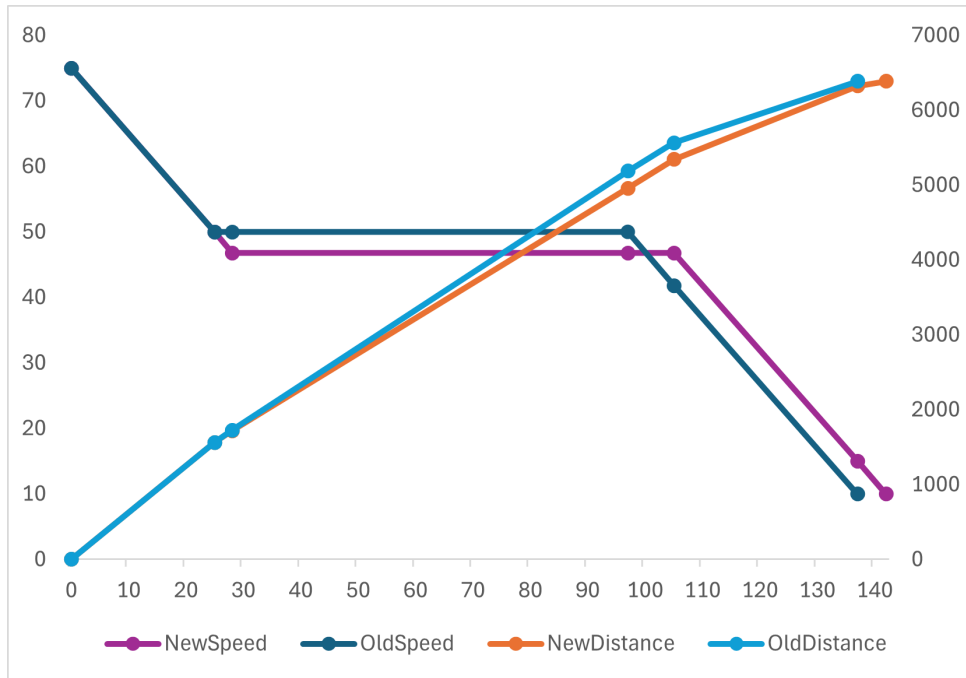


Figure 4.28: Deceleration, Cruising and Deceleration Method 2

"Acceleration, Cruising and Deceleration" and the "Acceleration and Deceleration". The implementation of our solution was developed in Java Version 8. The developed program's calculations consider over ten decimal points; however, we guarantee an accuracy of four decimal points.

4.2.1 Basic Functions

Before we proceed to explain the implementation of our algorithm, we present some key aspects of our implementation. Specifically, we describe the functionality of the methods for each entity in our problem. The execution of the following methods will be implied to simplify the demonstration of the main algorithms.

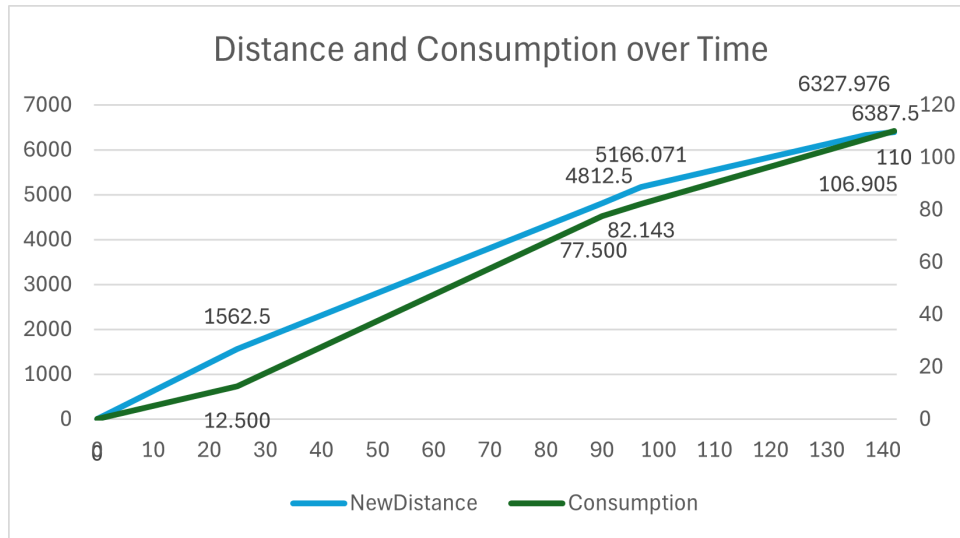


Figure 4.29: Deceleration, Cruising and Deceleration Fuel Consumption of Method 2 Train

This entity includes the simplest functions, as they are limited to basic getter methods. There is one getter for each characteristic of the train (i.e., length, maxSpeed, maxAcc, and maxDec).

Path

Moving on to the Path entity, it mostly includes methods regarding its segments. Some of its getters are more complex since they return a characteristic derived from the entity's attributes rather than just an attribute. The methods "getHSgm" and "getTSgm" return the head and tail of the path, respectively. The head is considered the path's last segment, while the tail is the path's first segment. The method "getTotalLength" returns the total distance of the path, which is equal to the sum of the segments' distances. The "getSegments(tSgm, hSgm)" method returns a list containing all the path's segments from *tSgm* to *hSgm*.

State

The State entity also provides various methods. Similar to the Path entity, it includes "getHSgm" and "getTSgm", which return the head and tail of the path that the train is currently on, respectively. In other words, they return the segments which correspond to the front and rear of the locomotive. The method "getTotalLength" returns the total distance of the path occupied by the locomotive. The "getSegments" method returns a list of the segments the train occupies in the current state. Additionally, "getSpeed" returns the train's speed, while "getDist" returns the distance from the front of the train to the end of the segment it is currently on.

Speed Profile

This entity encompasses the most methods. The method "nOfEntries" returns the number of states in the current speed profile. "addState" is a void method that appends the input state at the end of the speed profile with an ending time equal to the time input. "getState" and "getIndexof" are two complementary methods: "getState"'s input is an

integer referring to a state and its output is that state; "getIndexOf"'s input is a state and its output is the state's position in the speed profile. The "getSpeedOf(index)" method returns the speed of the state at position *index*. Both "getStartTimeAtState" and "getEndTimeAtState" require an integer referring to a state and return the start and end time of that state, respectively. The last method, "calcDistanceTraveled(index)", calculates the distance the locomotive has covered until the state at position *index*.

4.2.2 Algorithms

Our main algorithm consists of two sub-algorithms that address separate instances of a speed profile. Both of them follow a greedy approach to optimally modify the given speed profile. Algorithm 2 tackles "Acceleration and Deceleration" instances based on their time-fuel trade-off when they are modified by slightly extending the total duration, resulting in a speed reduction. On the other hand, algorithm 3 addresses "Acceleration, Cruising and Deceleration" instances based on their time-fuel trade-off when they are modified by slightly reducing the cruising speed, resulting in an extension of the total duration.

Algorithm 1 computes the fuel-optimal speed profile as a modification of the original speed profile. Its inputs are the SpeedProfile *originalSp* and the extra available time (*time*) and its output is the SpeedProfile *fuelOptimalSp*. First, the algorithm initializes the variables *newSp*, *accToDecSp*, and *crSp* to *originalSp*, and the variables *accToDecDuration* and *crDuration* to zero. The variable *newSp* denotes a temporary speed profile used in the computations of algorithms 2 and 3, detailed later. The variables *accToDecSp* and *crSp* are the speed profiles generated by algorithms 2 and 3, respectively. The variables *accToDecDuration* and *crDuration* denote the overtime of *accToDecSp* and *crSp* over *originalSp*, respectively. To utilize all the extra available time possible, the computations are nested in a loop that iterates while *time* is positive. Inside the loop, the algorithm invokes algorithms 2 and 3 to acquire both the aforementioned variables and the trade-off of their respective speed profiles. On each iteration, these algorithms, detailed thoroughly later in this paper, implement slight modifications to *originalSp*. The variable *accToDecTradeOff* represents the trade-off of *accToDecSp*, while the variable *crTradeOff* represents the trade-off of *crSp*. These variables have been initialized to -1 . If both variables are unchanged after the execution of the algorithms, then *originalSp* does not contain instances of either "Acceleration, Cruising and Deceleration" or "Acceleration and Deceleration" scenarios. In this event, since there is no beneficial modification of the original speed profile, the algorithm returns *originalSp*. If only one of the variables is altered and a modification was successfully applied, then both *fuelOptimalSp* and *originalSp* are set equal to the modified speed profile corresponding to the updated trade-off variable and *time* is reduced by the duration of the adjusted speed profile; otherwise, the algorithm returns *originalSp*, since there are no applicable modifications. If both *crTradeOff* and *accToDecTradeOff* are altered and any modification was successfully applied to either one of them, then the variables *fuelOptimalSp*, *originalSp* and *time* are updated to the results of the algorithm corresponding to the lower trade-off; otherwise, the algorithm returns *originalSp*, since there are no applicable modifications. When *time* gets reduced to zero, the loop terminates and the algorithm returns *fuelOptimalSp*.

Algorithm 1 Determine the Fuel-Optimal Speed Profile**Require:** SpeedProfile *originalSp*, double *time***Ensure:** SpeedProfile *fuelOptimalSp*

```

1: newSp  $\leftarrow$  originalSp
2: accToDecSp  $\leftarrow$  originalSp
3: crSp  $\leftarrow$  originalSp
4: fuelOptimalSp  $\leftarrow$  originalSp
5: accToDecDuration  $\leftarrow$  0
6: crDuration  $\leftarrow$  0
7: while time > 0 do
8:   Execute Algorithm 2
9:   Execute Algorithm 3
10:  if accToDecTradeOff = -1 and crTradeOff = -1 then
11:    return originalSp
12:  else if accToDecTradeOff = -1 then
13:    if crSp = originalSp then
14:      return originalSp
15:    end if
16:    fuelOptimalSp  $\leftarrow$  crSp
17:    originalSp  $\leftarrow$  fuelOptimalSp
18:    time  $\leftarrow$  time - crDuration
19:  else if crTradeOff = -1 then
20:    if accToDecSp = originalSp then
21:      return originalSp
22:    end if
23:    fuelOptimalSp  $\leftarrow$  accToDecSp
24:    originalSp  $\leftarrow$  fuelOptimalSp
25:    time  $\leftarrow$  time - accToDecDuration
26:  else
27:    if crSp = originalSp and accToDecSp = originalSp then
28:      return originalSp
29:    end if
30:    if accToDecTradeOff < crTradeOff then
31:      fuelOptimalSp  $\leftarrow$  accToDecSp
32:      originalSp  $\leftarrow$  fuelOptimalSp
33:      time  $\leftarrow$  time - accToDecDuration
34:    else
35:      fuelOptimalSp  $\leftarrow$  crSp
36:      originalSp  $\leftarrow$  fuelOptimalSp
37:      time  $\leftarrow$  time - crDuration
38:    end if
39:  end if
40: end while
41: return fuelOptimalSp

```

Algorithm 2 Addition of Cruising

Require: SpeedProfile *originalSp*, *accToDecSp*, double *time*, *accToDecDuration*
Ensure: SpeedProfile *accToDecSp*, double *accToDecDuration*,
accToDecTradeOff

- 1: *accToDecTradeOff* $\leftarrow -1$
- 2: *tempSpeed* $\leftarrow 0.1$
- 3: **if** *tempSpeed* > *time* **then**
- 4: *tempSpeed* $\leftarrow time$
- 5: **end if**
- 6: *newSp* $\leftarrow$ Result of Algorithm 4 with *tempTime*
- 7: **if** *newSp* $\neq$ *originalSp* **then**
- 8: *newTradeOff* $\leftarrow \frac{\text{newSp's Fuel Consumption}}{\text{newSp's Total Duration}}$
- 9: **if** *accToDecTradeOff* < *newTradeOff* **then**
- 10: *accToDecTradeOff* $\leftarrow newTradeOff$
- 11: *accToDecSp* $\leftarrow newSp$
- 12: *accToDecDuration* $\leftarrow timeUsedAccToDec$
- 13: **end if**
- 14: **end if**

Algorithm 2 computes a speed profile with a modification of an "Acceleration and Deceleration" instance. In the case of multiple instances, it chooses the speed profile with the maximum time-fuel trade-off. Its inputs are SpeedProfile *originalSp*; the initialized speed profile *accToDecSp*, representing the adjusted *originalSp*; *accToDecDuration*, denoting the overtime of *accToDecSp* over *originalSp*; and the extra available time (*time*). Its outputs are the updated *accToDecSp* and *accToDecDuration* variables and *accToDecTradeOff* representing the time-fuel trade-off of *accToDecSp*. First, the algorithm initializes the variables *tempTime*, representing the time used in the modification, and *accToDecTradeOff* to 0.1. If *time* is less than 0.1, *tempSpeed* is set to *time*, which means that all the available time is going to be utilized. The variable *newSp* is set to the result of algorithm 4, which completely uses the time denoted by *tempTime* on a single "Acceleration and Deceleration" instance. If the algorithm's result equals *originalSp*, it means that no modification was made and the algorithm terminates. Otherwise, the algorithm calculates the time-fuel trade-off of *newSp* (*newTradeOff*), which is equal to the fraction of *newSp*'s consumption over its duration, and if *newTradeOff* is greater than *accToDecTradeOff*, it updates *accToDecTradeOff*, *accToDecSp*, and *accToDecDuration* to the results of the computations.

Algorithm 3 takes SpeedProfile *originalSp*, representing the original speed profile; SpeedProfile *crSp*, representing the modified speed profile, initialized to *originalSp*; the extra available time (*time*); and the initialized *crDuration*, representing the extra duration of *crSp* over the duration of *originalSp*, as inputs. Its outputs are the updated *crSp* and *crDuration* and the time-fuel trade-off of *crSp* (*crTradeOff*). The algorithm stores in the variable *timeUsedCr* the time used in the adjustments and initializes it to *time*. To acquire the states representing "Acceleration, Cruising and Deceleration" instances, algorithm 5 is invoked and its result is saved in *crStates*. Additionally, it initializes *crTradeOff* to -1 . To compute the trade-off of every feasible modification, the algorithm initiates a loop that iterates through all the indexes of the states in *crStates*. Each iteration executes algorithm 6, detailed thoroughly

Algorithm 3 Slower Constant Speed**Require:** SpeedProfile *originalSp*, *crSp*, double *time*, *crDuration***Ensure:** SpeedProfile *crSp*, double *crDuration*, *crTradeOff*

```

1:  $timeUsedCr \leftarrow time$ 
2:  $crStates \leftarrow$  Result of Algorithm 5
3:  $crTradeOff \leftarrow -1$ 
4: for each index in crStates do
5:   Execute Algorithm 6
6:    $newTradeOff \leftarrow \frac{newSp's \text{ Fuel Consumption}}{newSp's \text{ Total Duration}}$ 
7:   if  $crTradeOff < newTradeOff$  then
8:      $crTradeOff \leftarrow newTradeOff$ 
9:      $crSp \leftarrow newSp$ 
10:     $crDuration \leftarrow timeUsedCr$ 
11:   end if
12: end for

```

afterward, to obtain the necessary values to construct the resulting modified speed profile and calculate its trade-off. Finally, before proceeding to the next iteration, in the event that *newTradeOff* is greater than *crTradeOff*, it sets *newTradeOff* equal to the fraction of the consumption over the time, and updates *crTradeOff*, *crSp*, and *crDuration* to the results of the computation.

Algorithm 4 computes a speed profile with a modification of a single "Acceleration and Deceleration" instance utilizing all its given time. In the case of multiple instances, it chooses the modified speed profile with the maximum time-fuel trade-off. Its inputs are SpeedProfile *originalSp*; the initialized speed profile *accToDecSp*, representing the adjusted *originalSp*; *accToDecDuration*, denoting the overtime of *accToDecSp* over *originalSp*; and the extra available time (*time*). Its outputs are the updated *accToDecSp* and *accToDecDuration* variables and *accToDecTradeOff*, representing the time-fuel trade-off of *accToDecSp*. The algorithm is executed only if *time* is positive where algorithm 7 is invoked to acquire the states at the end of an acceleration that is followed by deceleration (*accToDecStates*). In the case where there are no "Acceleration and Deceleration" instances, the algorithm returns *originalSp*. Otherwise, the algorithm proceeds by initializing the variables *timeUsedAccToDec*, representing the extra time used by the modification of *originalSp*, to *time*, and *accToDecTradeOff* to -1 . Next, for each state of *accToDecStates*, it invokes algorithm 8, which initializes all the variables needed to execute the computational part of the algorithm. The computational part of algorithm 4 implements a divide and conquer methodology to determine the cruising speed (*crSpeed*). It consists of an infinite loop with two break points. Inside the loop, the algorithm computes the distance the locomotive travels while cruising at *crSpeed* for $timeUsedAccToDec + b$ seconds and stores it in the variable *dTraveled*. The variable *totalDistance* represents the total distance the locomotive travels until the end of the deceleration. If *totalDistance* equals the distance of the instance in *originalSp* (*de*), the loop breaks. Next, it checks whether the limits have merged into each other. If *maxSpeed* equals *minSpeed*, then the algorithm invokes itself with less time. Otherwise, if *totalDistance* is greater than *de*, meaning that the locomotive travels farther than it must, it updates *maxSpeed* (i.e., the upper limit of *crSpeed*) to *crSpeed*. Conversely, if *totalDistance* is less than *de*, which means that the locomotive does not travel far enough, it updates *minSpeed* (i.e., the lower limit of

Algorithm 4 Computations of Algorithm 2**Require:** SpeedProfile *originalSp*, *accToDecSp*, double *time*, *accToDecDuration***Ensure:** SpeedProfile *accToDecSp*, double *timeUsedAccToDec*

```

1: if time > 0 then
2:   accToDecStates  $\leftarrow$  Result of Algorithm 7
3:   if accToDecStates then
4:     return originalSp
5:   end if
6:   timeUsedAccToDec  $\leftarrow$  time
7:   accToDecTradeOff  $\leftarrow$  -1
8:   for each index in accToDecStates do
9:     Execute Algorithm 8
10:    while true do
11:      dTraveled  $\leftarrow$  crSpeed  $\cdot$  (timeUsedAccToDec + b)
12:      totalDistance  $\leftarrow$  ds + dTraveled
13:      if totalDistance = de then
14:        break
15:      end if
16:      if maxSpeed = minSpeed then
17:        tempTime  $\leftarrow$  time - 0.1
18:        return Result of algorithm 4 with tempTime
19:      end if
20:      if totalDistance > de then
21:        maxSpeed  $\leftarrow$  crSpeed
22:      else
23:        minSpeed  $\leftarrow$  crSpeed
24:      end if
25:      crSpeed  $\leftarrow$   $\frac{\text{maxSpeed} + \text{minSpeed}}{2}$ 
26:      crSpeedTimeAcc  $\leftarrow$   $\frac{\text{crSpeed} - \text{targetSpeed}}{\text{acc}}$ 
27:      ds  $\leftarrow$  targetStateDistance + targetSpeed  $\cdot$  crSpeedTimeAcc +  $0.5 \cdot \text{acc} \cdot \text{crSpeedTimeAcc}^2$ 
28:      crSpeedTimeDec  $\leftarrow$   $\frac{\text{crSpeed} - \text{peakSpeed}}{\text{dec}}$ 
29:      de  $\leftarrow$  accToDecStateDistance + peakSpeed  $\cdot$  crSpeedTimeDec +  $0.5 \cdot \text{dec} \cdot \text{crSpeedTimeDec}^2$ 
30:      b  $\leftarrow$   $\frac{\text{peakSpeed} - \text{crSpeed}}{\text{acc}} + \frac{\text{crSpeed} - \text{peakSpeed}}{\text{dec}}$ 
31:    end while
32:    Execute Algorithm 9
33:    newTradeOff  $\leftarrow$   $\frac{\text{newSp's Fuel Consumption}}{\text{newSp's Total Duration}}$ 
34:    if accToDecTradeOff < newTradeOff then
35:      accToDecTradeOff  $\leftarrow$  newTradeOff
36:      accToDecSp  $\leftarrow$  newSp
37:    end if
38:  end for
39: end if

```

crSpeed) to *crSpeed*. It then proceeds to the subsequent iteration by updating *crSpeed*, *crSpeedTimeAcc* (representing the start time of the new cruising), *ds* (representing the distance covered by the train while accelerating to *crSpeed*), *crSpeedTimeDec* (representing the duration of the deceleration from *maxSpeed* to *crSpeed*), *de* and *b* (representing the duration of the acceleration and deceleration between *peakSpeed* and *crSpeed* in *originalSp*), according to the corresponding calculations in algorithm 8. After completing its computations, it invokes algorithm 9 to construct *newSp*, sets *newTradeOff* equal to the fraction of *newSp*'s consumption over its duration, and in case it is greater than *accToDecTradeOff*, it updates *accToDecTradeOff* and *accToDecSp* to the results of the computation.

Algorithm 5 Acquire States at the End of Cruising

Require: SpeedProfile *originalSp*

Ensure: Map *crStates*

```

1: startSpeed  $\leftarrow$  first state of originalSp
2: for  $i \leftarrow 1$  to  $nOfEntries - 2$  do
3:   aState  $\leftarrow$  state at position  $i$  in originalSp
4:   endSpeed  $\leftarrow$  train's speed at aState
5:   if endSpeed = startSpeed then
6:     prevAcc  $\leftarrow$  false
7:     for  $j \leftarrow i - 1$  to 0 do
8:       if (train's speed at state at position  $j$  in originalSp) < endSpeed then
9:         prevAcc  $\leftarrow$  true
10:      break
11:    else if (train's speed at state at position  $j$  in originalSp) > endSpeed
12:      then
13:        prevAcc  $\leftarrow$  false
14:      break
15:    end if
16:  end for
17:  nextSpeed  $\leftarrow$  speed at position  $i + 1$  in originalSp
18:  if nextSpeed < endSpeed and prevAcc then
19:    Append crState with key  $i$  in crStates
20:  end if
21:  startSpeed  $\leftarrow$  endSpeed
22: end for
23: return crStates

```

Algorithm 5 takes the SpeedProfile *originalSp* as its single input and outputs the Map *crStates*, consisting of the states at the end of cruising in the "Acceleration, Cruising and Deceleration" instances and their positions in *originalSp*. The variable *startSpeed* denotes the train's speed at the state preceding the one currently under consideration and is initialized to the train's speed at the first state of *originalSp*. The algorithm iterates through all the positions of states in *originalSp*, from the second to the second-to-last. The variable *aState* is the state under consideration and is set to the state at position i in *originalSp*, while the variable *endSpeed* denotes the locomotive's speed at *aState*. A state describes an instance only if the train's speed at that state is greater than both the speed at the following state and the speed at the first state

before the cruising. If *startSpeed* is not equal to *endSpeed*, the algorithm advances to the next iteration by updating *startSpeed* to *endSpeed*. Otherwise, the algorithm initializes *prevAcc* to **false**. The variable *prevAcc* is a boolean variable indicating whether the locomotive's prior throttle setting was acceleration. This is determined by an iteration loop, which breaks at the first state where the train's speed is not equal to *endSpeed* (i.e., the cruising speed) and updates *prevAcc* accordingly. If *prevAcc* is **true** and *nextSpeed* (representing the train's speed at the state following *aState*) is less than *endSpeed*, it appends *aState* to *crStates*. Finally, once the loop terminates, the algorithm returns *crStates*.

Algorithm 6 Computations of Algorithm 3

Require: SpeedProfile *originalSp*, int *index*

Ensure: SpeedProfile *newSp*, double *timeUsedCr*

```

1: crState  $\leftarrow$  state at position index in originalSp
2: crSpeed  $\leftarrow$  train's speed at crState
3: Execute Algorithm 11
4: targetIndex  $\leftarrow$  index of targetState
5: startCrIndex  $\leftarrow$  index of startCrState
6: targetSpeed  $\leftarrow$  train's speed at targetState
7: acc  $\leftarrow$  train's max acceleration rate
8: dec  $\leftarrow$  train's max deceleration rate  $\cdot (-1)$ 
9: newSpeed  $\leftarrow$  crSpeed  $- 0.1$ 
10: if newSpeed  $>$  targetSpeed then
11:   if newSpeed  $<$  (train's speed at state after crState) then
12:     newSpeed  $\leftarrow$  (speed at state after crState)
13:   end if
14:   Execute Algorithm 12
15: else
16:   Execute Algorithm 13
17: end if
```

Algorithm 6 details the main functionality of algorithm 3. Its inputs are the original speed profile (*originalSp*) and the position of the state that denotes the "Acceleration, Cruising and Deceleration" instance (*index*). The outputs are the modified original speed profile (*newSp*) and the extra time utilized (*timeUsedCr*). The algorithm initializes *crState* to the state at position *index* in *originalSp* and sets the variable *crSpeed* to the locomotive's cruising speed at *crState*. Before proceeding with any calculations, it invokes algorithm 11 to acquire *targetState*, which represents the state at the start of the acceleration. It then retrieves the positions of *targetState* and *startCrState* (representing the state at the start of the cruising) in *originalSp*, as well as the train's speed at *targetState* (*targetSpeed*). Next, the algorithm calculates the acceleration and deceleration rates and uses them to determine the characteristics of the "Acceleration, Cruising and Deceleration" instance in *originalSp*. The fuel-optimal speed profile is adjusted in decrements of *crSpeed* by 0.1m/s. The following computations of the algorithm depend on the new cruising speed, denoted by variable *newSpeed*. If *newSpeed* is greater than *targetSpeed*, the modification regards a normal case. Normal cases refer to events where the modification does not transform the "Acceleration, Cruising and Deceleration" instances to other scenarios. Normal cases are addressed by algorithm 12. It is noteworthy that if in a normal case *newSpeed* is less than the speed at the

end of the deceleration, *newSpeed* is set equal to the latter. On the other hand, if *newSpeed* is greater than or equal to *targetSpeed*, the instance regards an alternative case. Unlike normal cases, alternative cases transform the "Acceleration, Cruising and Deceleration" instances into either "Deceleration, Cruising and Deceleration" or "Acceleration, Cruising and Acceleration" instances. These cases are handled by algorithm 13. The algorithm terminates after the execution of the necessary algorithm.

Algorithm 7 Identify "Acceleration and Deceleration" Instances

Require: SpeedProfile *sp*

Ensure: Map *accToDecStates*

```

1: startSpeed  $\leftarrow$  train's speed at the first state of sp
2: for i  $\leftarrow$  1 to nOfEntries - 2 do
3:   aState  $\leftarrow$  state at position i in sp
4:   endSpeed  $\leftarrow$  train's speed at aState
5:   if endSpeed > startSpeed then
6:     nextState  $\leftarrow$  state at position i + 1 in sp
7:     nextSpeed  $\leftarrow$  train's speed at nextState
8:     if nextSpeed < endSpeed then
9:       Append state at position i in sp to accToDecStates
10:    end if
11:  end if
12:  startSpeed  $\leftarrow$  endSpeed
13: end for
14: return accToDecStates

```

Algorithm 7 constructs a Map containing all the states at the end of accelerations that are immediately followed by deceleration and their positions in the given speed profile. Its input is a speed profile (*sp*) and its output is represented by *accToDecStates*. The variable *startSpeed* denotes the locomotive's speed at the state before the one currently under consideration and is initialized to the speed of the first state in *sp*. The algorithm then iterates through the positions of *sp*, from the second to the second-to-last. The variable *aState* is the state under consideration, while the variable *endSpeed* represents the locomotive's speed at *aState*. A state describes an instance if the train's speed is higher at that state than it is at the states adjacent to it. Consequently, *startSpeed* must be less than *endSpeed*; otherwise, since there is no acceleration, the algorithm updates *startSpeed* to *endSpeed* and moves on to the next iteration. If *startSpeed* is less than *endSpeed*, it proceeds by comparing the next state's speed (*nextSpeed*) with *endSpeed*. If *endSpeed* is greater than *nextSpeed*, it identifies an appropriate instance of the "Acceleration and Deceleration" scenario and appends it to *accToDecStates*. Otherwise, it proceeds to the next iteration. Finally, once the loop terminates, the algorithm returns *accToDecStates*.

Algorithm 8 initializes the variables that describe the new states of *newSp*. Its inputs are the SpeedProfile *originalSp* and the position of the state describing the instance under modification (*index*). Its outputs are the initialized variables needed for the execution of the computations of algorithm 3. First, the algorithm acquires the state at position *index* in *originalSp* (*accToDecState*), calculates the acceleration rate between *accToDecState* and its predecessor (*a*), initializes the variable *peakSpeed* to the train's speed at *accToDecState*, and invokes algorithm 10, obtaining

Algorithm 8 Initialize the "Addition of Cruising" Method Computations**Require:** SpeedProfile *originalSp*, int *index*

Ensure: double *crSpeed*, *crSpeedTimeAcc*, *crSpeedTimeDec*, *ds*, *de*, *b*, *dec*, *acc*, *accToDecStateDistance*, *minSpeed*, *maxSpeed*, *targetSpeed*

- 1: *accToDecState* $\leftarrow$ state at position *index* in *originalSp*
- 2: *peakSpeed* $\leftarrow$ train's speed at *accToDecState*
- 3: $a \leftarrow \frac{\text{peakSpeed} - (\text{train's speed at state at position } index-1 \text{ in } originalSp)}{(\text{end time at } accToDecState - (\text{end time at state at position } index-1 \text{ in } originalSp))}$
- 4: *targetState* $\leftarrow$ Result of Algorithm 10
- 5: *maxSpeed* $\leftarrow$ *peakSpeed*
- 6: *targetSpeed* $\leftarrow$ speed of *targetState*
- 7: *minSpeed* $\leftarrow$ max(*targetSpeed*, speed of state *index* + 1 of *originalSp*)
- 8: *crSpeed* $\leftarrow \frac{\text{maxSpeed} + \text{minSpeed}}{2}$
- 9: *targetStateDistance* $\leftarrow$ distance traveled up to *targetState*
- 10: *deltaT* $\leftarrow$ (end time of *accToDecState*) – (end time of *targetState*)
- 11: *acc* $\leftarrow \frac{\text{peakSpeed} - \text{targetSpeed}}{\text{deltaT}}$
- 12: *crSpeedTimeAcc* $\leftarrow \frac{\text{crSpeed} - \text{targetSpeed}}{acc}$
- 13: *ds* $\leftarrow$ *targetStateDistance* + *targetSpeed* · *crSpeedTimeAcc* + 0.5 · *acc* · *crSpeedTimeAcc*²
- 14: *accToDecStateDistance* $\leftarrow$ distance traveled up to *accToDecState*
- 15: *deltaT* $\leftarrow$ (end time of state *index* + 1 of *originalSp*) – (end time of *accToDecState*)
- 16: *dec* $\leftarrow \frac{(\text{speed of state } index+1 \text{ of } originalSp) - \text{peakSpeed}}{\text{deltaT}}$
- 17: *crSpeedTimeDec* $\leftarrow \frac{\text{crSpeed} - \text{peakSpeed}}{dec}$
- 18: *de* $\leftarrow$ *accToDecStateDistance* + *peakSpeed* · *crSpeedTimeDec* + 0.5 · *dec* · *crSpeedTimeDec*²
- 19: *b* $\leftarrow \frac{\text{peakSpeed} - \text{crSpeed}}{acc} + \frac{\text{crSpeed} - \text{peakSpeed}}{dec}$

the state *targetState*, where the acceleration begins. Next, it stores the train's speed at *targetState* in *targetSpeed*. Algorithm 2 calculates *crSpeed* using a divide and conquer methodology; consequently *crSpeed* has an upper and a lower limit. Its upper limit (*maxSpeed*) is initialized to *peakSpeed*, while its lower limit (*minSpeed*) is initialized to the maximum value between *targetSpeed* and the speed of the train at the end of the deceleration (i.e., the speed of the train at the next state after *accToDecState*). The variable *crSpeed* is initialized as the mean of those two values. Moving on, the algorithm determines the distance traveled until *targetState* (*targetStateDistance*) and calculates the instance's acceleration rate (*acc*); the duration of the acceleration until it reaches *crSpeed* in *originalSp* (*crSpeedTimeAcc*); and the distance traveled until that moment (*ds*). Similarly, it computes *accToDecStateDistance*, which represents the distance covered by the locomotive until *accToDecState*; the deceleration rate (*dec*); the duration of the deceleration starting from *crSpeed* (*crSpeedTimeDec*); the distance traveled until that moment (*newDistDec*); and *de*, which represents the distance covered while decelerating to *crSpeed* in *originalSp*. Finally, it initializes the variable *b* as the sum of the acceleration and deceleration durations between *crSpeed* and *maxSpeed*. Those *b* seconds are used in the cruising mode.

Algorithm 9 constructs the new speed profile (*newSp*) as part of the "Addition of Cruising" method. Its inputs are the original speed profile (*originalSp*); the state at the start of the acceleration (*targetState*); the position of *targetState* in *originalSp*

Algorithm 9 Construct the Speed Profile of "Addition of Cruising" Method

Require: SpeedProfile *originalSp*, State *targetState*, double *crSpeed*, *targetSpeed*, *crSpeedTimeAcc*, *timeUsedAccToDec*, *b*, int *targetIndex*, *index*

Ensure: State *newSp*, double *timeUsedAccToDec*

- 1: *newSp* $\leftarrow$ first *targetIndex* states of *originalSp*
- 2: *newTimeAcc* $\leftarrow$ (end time of *targetState*) + *crSpeedTimeAcc*
- 3: *newDistAcc* $\leftarrow$ (dist of *targetState*) – (*targetSpeed* · *crSpeedTimeAcc* + $0.5 \cdot acc \cdot crSpeedTimeAcc^2$)
- 4: Add Start of Cruising State in *newSp*
- 5: *newTimeDec* $\leftarrow$ *newTimeAcc* + *timeUsedAccToDec*
- 6: *newDistDec* $\leftarrow$ (distance traveled up to state at position *index* + 1) – *de*
- 7: Add End of Cruising State in *newSp*
- 8: *newIndex* $\leftarrow$ *index*
- 9: **if** *crSpeed* = speed at the state at position *index* + 1 **then**
- 10: *newIndex* $\leftarrow$ *newIndex* + 1
- 11: **end if**
- 12: **for** *i* $\leftarrow$ *newIndex* + 1 **to** *nOfEntries* – 1 **do**
- 13: Append state at position *i* in *originalSp* delayed by *timeUsedAccToDec* in *newSp*
- 14: **end for**

(*targetIndex*); the position of the state at the peak of the acceleration in *originalSp* (*index*); the cruising speed (*crSpeed*); the train's speed at *targetState* (*targetSpeed*); the extra time utilized by the modifications (*timeUsedAccToDec*); the duration of the acceleration until it reaches *crSpeed* in *originalSp* (*crSpeedTimeAcc*); and the variable *b* which represents the sum of the acceleration and deceleration durations between *crSpeed* and the maximum speed of the instance in *originalSp*. The algorithm sets *newSp* equal to the first *targetIndex* states of *originalSp* and calculates the variable *newTimeAcc*, which represents the start time of the cruising, as the end time of *targetState* increased by the duration of the new acceleration (*crSpeedTimeAcc*). The variable *newDistAcc*, representing the distance from the front of the train to the end of the current segment, is set to the corresponding distance in *originalSp* subtracted by the distance covered while accelerating from the new cruising speed to *crSpeed*. These values describe the state corresponding to the start of the cruising in *newSp*. On the other hand, *newTimeDec* represents the end time of the new deceleration and is equal to the sum of the duration of *newTimeAcc*, *timeUsedAccToDec*, and *b*, while *newDistDec* represents the distance of the train's front from the end of the current segment at the start of the deceleration. These variables describe the state at the end of the cruising in *newSp*. Finally, the algorithm appends *newSp*'s remaining states, which are identical to those of *originalSp*, delayed by *timeUsedAccToDec*. If the speed at the end of the deceleration is equal to *newSpeed*, meaning that the last added state and the first of the remaining states are identical, it skips the latter.

Algorithm 10 locates the state at the beginning of the acceleration in an "Acceleration and Deceleration" instance. Its inputs are the SpeedProfile *originalSp*; the position of the state at the end of the acceleration in *originalSp* (*index*); and the instance's acceleration rate between the state at *index* and its prior (*maxAcc*). Its output is represented by the variable *targetState*. First, the algorithm initializes *targetState* to the first state of *originalSp*. Next, it iterates backwards from position *index* – 1 to 1. In

Algorithm 10 Identify the Start of an "Acceleration and Deceleration" Instance**Require:** SpeedProfile *originalSp*, int *index*, double *maxAcc***Ensure:** State *targetState*

```

1: targetState  $\leftarrow$  first state of originalSp
2: for  $i \leftarrow index - 1$  to 1 do
3:    $prevAcc \leftarrow \frac{(\text{speed of state at position } i \text{ in } originalSp) - (\text{speed of state at position } i-1 \text{ in } originalSp)}{(\text{end time at state at position } i \text{ in } originalSp) - (\text{end time at state at position } i-1 \text{ in } originalSp)}$ 

4:   if  $maxAcc = prevAcc$  then
5:     return State at position  $i$  in originalSp
6:   end if
7: end for
8: return targetState

```

this loop, it calculates the acceleration rate of the previous pair of states (*prevAcc*) and compares it to *maxAcc*. Since the algorithm regards an "Acceleration and Deceleration" instance, *maxAcc* cannot be less than *prevAcc*. If *maxAcc* is greater than *prevAcc*, then *targetState* is set to the state at position i in *originalSp*; otherwise, the algorithm proceeds to the next iteration. If the loop does not break, then *targetState* remains equal to the first state of *originalSp*. Finally, the algorithm returns *targetState*.

Algorithm 11 takes the SpeedProfile *originalSp*; the integer *index*, indicating the position of an "Acceleration, Cruising and Deceleration" instance in *originalSp*; and the cruising speed (*crSpeed*), as inputs. Its outputs are the state at the start of the instance (*targetState*) and the state at the start of the cruising (*startCrState*). The algorithm begins by initializing *targetState* to the first state of *originalSp*, *startCrState* to *targetState*, and *tempSpeed* to *crSpeed*. The variable *tempSpeed* represents the train's speed at the state currently under consideration. The main functionality of the algorithm is nested in a loop that iterates backwards from the state at position $index - 1$ to the first state of *originalSp*. The variable *prevSpeed* represents the locomotive's speed at the state at position *index* in *originalSp*. *prevSpeed* must be less than *tempSpeed*; otherwise, the algorithm updates *tempSpeed* to be equal to *prevSpeed* and advances to the next iteration. This condition guarantees that *targetState* is not a state within the cruising. If *prevSpeed* corresponds to the state at position 0, then the algorithm terminates and *targetState* is equal to that state. Otherwise, the algorithm calculates the acceleration *acc* of the train between the states at positions $i + 1$ and i and identifies the start of the acceleration with rate *acc* via a loop. Specifically, it iterates backwards from the previous state (at position $i - 1$) to the first state (at position 0) and stops when it detects a different acceleration rate. When this occurs, *targetState* is set to the state last considered before the current one (i.e., the state at position $j + 1$, where j is the index of the inner loop).

Algorithm 11 Identify the State of an "Acceleration, Cruising and Deceleration" Instance

Require: SpeedProfile *originalSp*, double *crSpeed*, int *index*

Ensure: State *targetState*, *startCrState*

```

1: targetState  $\leftarrow$  first state of originalSp
2: startCrState  $\leftarrow$  targetState
3: tempSpeed  $\leftarrow$  crSpeed
4: for i  $\leftarrow$  index - 1 to 0 do
5:   prevSpeed  $\leftarrow$  train's speed at state at position i in originalSp
6:   if prevSpeed < tempSpeed then
7:     startCrState  $\leftarrow$  state at position i + 1 in originalSp
8:     if i = 0 then
9:       break
10:    end if
11:    deltaT  $\leftarrow$  (end time of state i + 1 in originalSp) - (end time at state i)
12:    acc  $\leftarrow$   $\frac{\text{tempSpeed} - \text{prevSpeed}}{\text{deltaT}}$ 
13:    for j  $\leftarrow$  i - 1 to 0 do
14:      deltaT  $\leftarrow$  (end time of state at position j + 1 in originalSp) - (end time
        of state at position j in originalSp)
15:      deltaV  $\leftarrow$  (speed of state at position j + 1 in originalSp) - (speed of state
        at position j in originalSp)
16:      tempAcc  $\leftarrow$   $\frac{\text{deltaV}}{\text{deltaT}}$ 
17:      if acc  $\neq$  tempAcc then
18:        targetState  $\leftarrow$  state at position j + 1 in originalSp
19:        break
20:      end if
21:    end for
22:    break
23:  end if
24:  tempSpeed  $\leftarrow$  prevSpeed
25: end for

```

Algorithm 12 Construct the Speed Profile in Normal Case

Require: SpeedProfile *originalSp*, State *crState*, startCrState, targetState, double *crSpeed*, targetSpeed, *acc*, *dec*, *time*, newSpeed, int *index*,

Ensure: SpeedProfile *newSp*, double *timeUsedCr*

```

1:  $oldAccTime \leftarrow \frac{crSpeed - targetSpeed}{acc}$ 
2:  $oldCruiseTime \leftarrow (\text{end time at } crState) - (\text{end time at } startCrState)$ 
3:  $oldDecTime \leftarrow \frac{(\text{train's speed at state after } crState \text{ in } originalSp) - crSpeed}{dec}$ 
4:  $oldTime \leftarrow oldAccTime + oldCruiseTime + oldDecTime$ 
5:  $newAccTime \leftarrow \frac{newSpeed - targetSpeed}{acc}$ 
6:  $newDecTime \leftarrow \frac{(\text{speed at state at position } index+1 \text{ in } originalSp) - newSpeed}{dec}$ 
7:  $totalDistance \leftarrow newSpeed \cdot (oldAccTime - newAccTime) + 0.5 \cdot acc \cdot (oldAccTime - newAccTime)^2 + (\text{distance traveled at } crState) - (\text{distance traveled at } startCrState) + crSpeed \cdot (oldDecTime - newDecTime) + 0.5 \cdot dec \cdot (oldDecTime - newDecTime)^2$ 
8:  $newCruiseTime \leftarrow \frac{totalDistance}{newSpeed}$ 
9:  $newTime \leftarrow newAccTime + newCruiseTime + newDecTime$ 
10:  $timeUsedCr \leftarrow newTime - oldTime$ 
11: if  $timeUsedCr > time$  then
12:    $newSp \leftarrow \text{Result of Algorithm 14}$ 
13:    $timeUsedCr \leftarrow (\text{Duration of } newSp) - (\text{Duration of } originalSp)$ 
14: else
15:    $newSp \leftarrow \text{all states in } originalSp \text{ up to } targetState$ 
16:   Add new State at Start of New Cruising in  $newSp$ 
17:   Add new State at End of New Cruising in  $newSp$ 
18:    $newIndex \leftarrow index$ 
19:   if  $newSpeed = \text{speed at state after } crState$  then
20:      $newIndex \leftarrow newIndex + 1$ 
21:   end if
22:   for  $i \leftarrow newIndex + 1$  to  $nOfEntries - 1$  do
23:     Append state at position  $i$  in  $originalSp$  delayed by  $timeUsedCr$  in  $newSp$ 
24:   end for
25: end if

```

Algorithm 12 depicts the construction of the modified speed profile in a normal case. Its inputs are the original speed profile (*originalSp*); the extra available time (*time*); and the variables *targetState*, *startCrState*, and *crState* representing the states at the start of the acceleration, the start of the cruising, and the end of the cruising, respectively. Additionally, it requires the train's speed at *targetState* (*targetSpeed*); the cruising speed of the instance in *originalSp* (*crSpeed*); the new cruising speed (*newSpeed*); the locomotive's acceleration and deceleration rates (*acc* and *dec*, respectively); and the position of *crState* in *originalSp* (*index*). Its outputs are the modified speed profile (*newSp*) and the extra time utilized by the algorithm (*timeUsedCr*). The algorithm begins by calculating the duration of the "Acceleration, Cruising and Deceleration" instance (*oldTime*) as the sum of *oldAccTime*, *oldCruiseTime*, and *oldDecTime*, representing the duration of the acceleration, cruising, and deceleration, respectively. It calculates the duration of the acceleration in the new instance (*newAccTime*), which peaks at *newSpeed*, and the duration of the deceleration from

$crSpeed$ to $newSpeed$ ($newDecTime$). Next, it calculates the distance the locomotive must travel while cruising at $newSpeed$ in the new instance ($totalDistance$). The variable $totalDistance$ is equal to the sum of the distance traveled during acceleration from $newSpeed$ to $crSpeed$, during cruising at $crSpeed$ in $originalSp$, and during deceleration from $crSpeed$ to $newSpeed$. The duration of the new cruising ($newCruiseTime$) is equal to the fraction of $totalDistance$ over $newSpeed$. The variable $newTime$ represents the duration of the instance in the new speed profile and is equal to the sum of $newAccTime$, $newCruiseTime$, and $newDecTime$. The variable $timeUsedCr$ is equal to the difference between $newTime$ and $oldTime$. If $timeUsedCr$ is greater than $time$, then $newSp$ is computed by algorithm 14, which uses all the remaining available time on a single instance, and the algorithm terminates. Otherwise, $newSp$ consists of the $originalSp$'s states up to and including $targetState$, followed by two extra states marking the start and the end of the new cruising, described by the previously calculated values, and the remaining states of $originalSp$ that are subsequent to $crState$ delayed by $timeUsedCr$. If the speed at the end of the deceleration is equal to $newSpeed$, meaning that the last added state and the first of the remaining states are identical, it skips the latter.

Algorithm 13 Construct the Speed Profile in Alternative Case

Require: SpeedProfile $originalSp$, State $crState$, double $targetSpeed$, int $targetIndex$

Ensure: SpeedProfile $newSp$, double $timeUsedCr$

```

1:  $newTargetIndex \leftarrow 0$ 
2: for  $j \leftarrow targetIndex - 1$  to 1 do
3:   if speed of state at position  $j$  in  $originalSp \neq targetSpeed$  then
4:      $newTargetIndex \leftarrow j$ 
5:   break
6:   end if
7: end for
8:  $newTargetState \leftarrow$  state at position  $newTargetIndex$  in  $originalSp$ 
9:  $newTargetSpeed \leftarrow$  speed at  $newTargetState$ 
10:  $newSpeed \leftarrow \max(targetSpeed, \text{train's speed after } crState)$ 
11: if  $newTargetSpeed > newSpeed$  then
12:   Execute Algorithm 15
13: else
14:   Execute Algorithm 16
15: end if
```

Algorithm 13 illustrates the construction of the fuel-optimal speed profile in the alternative case. Its inputs are the SpeedProfile $originalSp$; the state at the end of the cruising ($crState$); the speed at the start of the acceleration ($targetSpeed$); and the position of the state at the start of the acceleration in $originalSp$ ($targetIndex$). Its outputs are the modified speed profile ($newSp$) and the time it utilizes ($timeUsedCr$). First, the algorithm determines the position of the state before the start of the cruising that ends at the state at position $targetIndex$ ($newTargetIndex$). The variable $newTargetIndex$ is initialized to zero. Its value may change via a loop that iterates backwards from $targetIndex$ to 1. If the speed at the state at position j (i.e., the index of the iteration) in $originalSp$ is different than $targetSpeed$, it indicates that at that point the locomotive did not cruise, and the algorithm updates $newTargetIndex$

to j and breaks the loop. Next, it acquires the state at position $newTargetIndex$ ($newTargetState$), the train's speed at $newTargetState$, and sets $newSpeed$ to the maximum value between $targetSpeed$ and the locomotive's speed after $crState$. Finally, if the train was decelerating prior to the aforementioned cruising, it computes $newSp$ and $timeUsedCr$ by executing algorithm 15; otherwise, it computes them via algorithm 16.

Algorithm 14 produces a fuel-optimal speed profile by employing the "Slower Constant Speed" technique on a single instance, utilizing all the available time. If there are multiple instances, it modifies the one with the optimal time-fuel trade-off. Its inputs are the SpeedProfile $originalSp$ and the extra available time ($time$), while its output is the fuel-optimal speed profile ($crSp$). First, the variables $newSp$, representing a temporary speed profile used in the algorithm's computations, and $crSp$ are initialized to $originalSp$. The variable $timeUsedCr$ is initialized to $time$ and $maxTradeOff$, representing the time-fuel trade-off of $crSp$, is initialized to -1 . The algorithm constructs a loop that iterates backwards through the states in $crStates$, which consists of the states representing the instances. These states are acquired by algorithm 10. The algorithm sets $crState$ to the state at position $index$ (i.e., the index of the iteration) in $originalSp$ and sets the variable $crSpeed$ to the locomotive's cruising speed at $crState$. Next, it proceeds by initializing the necessary variables for the main computations by invoking algorithm 17. Algorithm 18 utilizes these initializations and produces the necessary variables to construct $newSp$. The variable $newSp$ consists of all the $originalSp$'s states up to and including $targetState$, followed by two extra states marking the start and the end of the new cruising, described by the prior calculated values, and the remaining states of $originalSp$ that are subsequent to $crState$ delayed by $timeUsedCr$. It then calculates the trade-off of $newSp$ ($newTradeOff$) and if it is greater than $maxTradeOff$, it updates $maxTradeOff$ and $crSp$ to $newTradeOff$ and $newSp$, respectively. If the speed at the end of the deceleration is equal to $speed$, meaning that the last added state and the first of the remaining states are identical, it skips the latter. Finally, after the completion of the loop, the algorithm returns $crSp$.

Algorithm 14 Employ "Slower Constant Speed" Method Utilizing All Extra Time**Require:** SpeedProfile *originalSp*, double *time***Ensure:** SpeedProfile *crSp*

```

1: crSp  $\leftarrow$  originalSp
2: newSp  $\leftarrow$  originalSp
3: timeUsedCr  $\leftarrow$  time
4: crStates  $\leftarrow$  Result of Algorithm 10
5: maxTradeOff  $\leftarrow$  -1
6: for each index in crStates do
7:   crState  $\leftarrow$  state at position index in originalSp
8:   crSpeed  $\leftarrow$  train's speed at crState
9:   Execute Algorithm 11
10:  targetIndex  $\leftarrow$  position of targetState
11:  startCrIndex  $\leftarrow$  position of startCrState
12:  Execute Algorithm 17
13:  Execute Algorithm 18
14:  newSp  $\leftarrow$  first targetIndex states of originalSp
15:  Add new State at Start of Cruising
16:  Add new State at End of Cruising
17:  newIndex  $\leftarrow$  index
18:  if speed = speed at state after crState then
19:    newIndex  $\leftarrow$  newIndex + 1
20:  end if
21:  for i  $\leftarrow$  newIndex + 1 to nOfEntries - 1 do
22:    Append state at position i in originalSp delayed by timeUsedCr in newSp
23:  end for
24:  newTradeOff  $\leftarrow$   $\frac{\text{newSp's Fuel Consumption}}{\text{newSp's Total Duration}}$ 
25:  if maxTradeOff < newTradeOff then
26:    maxTradeOff  $\leftarrow$  newTradeOff
27:    crSp  $\leftarrow$  newSp
28:  end if
29: end for
30: return crSp

```

Algorithm 15 computes the fuel-optimal speed profile in the alternative case where the last non-cruising throttle setting prior to the instance is deceleration. Its inputs are the original speed profile (*originalSp*); the state at the start of the instance's cruising (*startCrState*); the state at the end of the instance's cruising (*crState*); the state at the start of the instance's acceleration (*targetState*, which is at position *targetIndex* in *originalSp*); the state before the start of the cruising that ends at *targetState* (*newTargetState*); the position of the state at the end of the instance's cruising (*index*); the instance's cruising speed *originalSp* (*crSpeed*); the cruising speed before the instance, which is equal to the speed at *targetState* (*targetSpeed*); the new locomotive's maximum acceleration and deceleration rates (*acc* and *dec*, respectively); and the extra available time (*time*). Its outputs are the altered speed profile (*newSp*) and the extra time it utilizes (*crTradeOff*). The algorithm starts by calculating the overall duration from *newTargetState* until the train reaches the state

at position $index + 1$ ($oldTime$). This duration is equal to the sum of the deceleration from $newTargetSpeed$ to $targetSpeed$ ($oldDecTime1$), the duration of the cruising prior to the state ($oldCruiseTime1$), the duration of the acceleration from $targetSpeed$ to $crSpeed$ in $originalSp$ ($oldAccTime$), the duration of the original cruising ($oldCruiseTime2$), and the duration of the deceleration from $startCrState$ to the state at position $index + 1$ in $originalSp$. Next, it calculates the duration of the decelerations from $newTargetSpeed$ to $newSpeed$ ($newDecTime1$) and from $newSpeed$ to the locomotive's speed at the state at position $index + 1$ ($newDecTime2$). The variable $totalDistance$ represents the distance the locomotive must travel while cruising at $newSpeed$. The variable $newCruiseTime$ represents the duration of the new cruising and the variable $newTime$ represents the duration of the new instance. It is noteworthy that the instance in $originalSp$ is transformed into "Deceleration, Cruising and Deceleration". The variable $timeUsedCr$ is updated to the difference between $newTime$ and $oldTime$. If $timeUsedCr$ exceeds $time$, $newSp$ is computed by algorithm 14, which uses all the remaining available time on a single instance. Otherwise, $newSp$ consists of all the $originalSp$'s states up to and including $targetState$, followed by two extra states marking the start and the end of the new cruising, described by the previously calculated values, and the remaining states of $originalSp$ that are subsequent to $crState$ delayed by $timeUsedCr$. If the speed at the end of the deceleration is equal to $newSpeed$, meaning that the last added state and the first of the remaining states are identical, it skips the latter. Finally, the algorithm returns the calculated $newSp$ and $timeUsedCr$.

Algorithm 16 computes the fuel-optimal speed profile in the alternative case where the last non-cruising throttle setting prior to the instance is acceleration. Its inputs are the SpeedProfile $originalSp$; the state at the start of the instance's acceleration ($targetState$); the train's speed at $targetState$ ($targetSpeed$); the position of $targetState$ in $originalSp$ ($targetIndex$); the position of the state before the start of the cruising that ends at the state at position $targetIndex$ ($newTargetIndex$); the locomotive's speed at the state at position $newTargetIndex$ ($newTargetSpeed$); the position of the state at the end of the instance's cruising ($index$); the instance's cruising speed in $originalSp$ ($crSpeed$); the new cruising speed ($newSpeed$); and the acceleration and deceleration rates (acc and dec , respectively). Its outputs are the modified speed profile ($newSp$) and the extra time it utilizes $timeUsedCr$. First, the algorithm calculates the durations of the consecutive throttle settings from the state at position $newTargetIndex$ to the state at position $index + 1$. It calculates the duration of the acceleration from $newTargetSpeed$ to $targetSpeed$ ($oldAccTime1$), the duration of the cruising that ends at $targetState$ ($oldCruiseTime1$), the duration of the acceleration from $targetSpeed$ to $crSpeed$ ($oldAccTime2$), the duration of the original cruising ($oldCruiseTime2$), and the duration of the deceleration from $crSpeed$ to the train's speed at the end of the instance ($oldDecTime$). The variable $oldTime$ is the sum of the calculated durations, representing the overall duration from $newTargetState$ to the end of the instance in $originalSp$. Next, the algorithm calculates the duration of the new acceleration from $newTargetSpeed$ to $newSpeed$ ($newAccTime$) and the duration of the new deceleration from $newSpeed$ to the speed at the end of the deceleration ($newDecTime$). The variable $totalDistance$ represents the distance the locomotive must travel while cruising at $newSpeed$. It calculates the duration of the new cruising ($newCruiseTime$) and computes the duration of the new instance ($newTime$) as the sum of the new durations. The variable $timeUsedCr$ is updated to the difference be-

tween *newTime* and *oldTime*. If *timeUsedCr* exceeds *time*, *newSp* is computed by algorithm 14, which uses all the remaining available time on a single instance. Otherwise, *newSp* consists of all the *originalSp*'s states until *targetState*, followed by two extra states marking the start and the end of the new cruising, described by the previously calculated values, and the remaining states of *originalSp* that are subsequent to *crState* delayed by *timeUsedCr*. If the speed at the end of the deceleration is equal to *newSpeed*, meaning that the last added state and the first of the remaining states are identical, it skips the latter. Finally, the algorithm returns the calculated *newSp* and *timeUsedCr*.

Algorithm 15 Construct the Speed Profile for Decelerating Alternative Case

Require: SpeedProfile *originalSp*, State *targetState*, *startCrState*, *crState*, *newTargetState*, double *newTargetSpeed*, *crSpeed*, *targetSpeed*, *acc*, *dec*, *time*, int *index*

Ensure: SpeedProfile *newSp*, double *timeUsedCr*

```

1: oldDecTime1  $\leftarrow \frac{\text{targetSpeed} - \text{newTargetSpeed}}{\text{dec}}$ 
2: oldCruiseTime1  $\leftarrow (\text{end time at targetState}) - (\text{end time at state at position } \text{targetIndex} + 1 \text{ in } \text{originalSp})$ 
3: oldAccTime  $\leftarrow \frac{\text{crSpeed} - \text{targetSpeed}}{\text{acc}}$ 
4: oldCruiseTime2  $\leftarrow (\text{end time at state at position } \text{index} \text{ in } \text{originalSp}) - (\text{end time at } \text{startCrState})$ 
5: oldDecTime2  $\leftarrow \frac{(\text{speed at state at position } \text{index} + 1 \text{ in } \text{originalSp}) - \text{crSpeed}}{\text{dec}}$ 
6: oldTime  $\leftarrow \text{oldDecTime1} + \text{oldCruiseTime1} + \text{oldAccTime} + \text{oldCruiseTime2} + \text{oldDecTime2}$ 
7: newDecTime1  $\leftarrow \frac{\text{newSpeed} - \text{newTargetSpeed}}{\text{dec}}$ 
8: newDecTime2  $\leftarrow \frac{(\text{speed at state at position } \text{index} + 1 \text{ in } \text{originalSp}) - \text{newSpeed}}{\text{dec}}$ 
9: totalDistance  $\leftarrow \text{newTargetSpeed} \cdot (\text{oldDecTime1} - \text{newDecTime1}) + 0.5 \cdot \text{dec} \cdot (\text{oldDecTime1} - \text{newDecTime1})^2 + \text{distance traveled at state at position } \text{index} \text{ in } \text{originalSp} - \text{distance traveled at state after } \text{newTargetState} + \text{crSpeed} \cdot (\text{oldDecTime2} - \text{newDecTime2}) + 0.5 \cdot \text{dec} \cdot (\text{oldDecTime2} - \text{newDecTime2})^2$ 
10: newCruiseTime  $\leftarrow \frac{\text{totalDistance}}{\text{newSpeed}}$ 
11: newTime  $\leftarrow \text{newDecTime1} + \text{newCruiseTime} + \text{newDecTime2}$ 
12: timeUsedCr  $\leftarrow \text{newTime} - \text{oldTime}$ 
13: if timeUsedCr > time then
14:   newSp  $\leftarrow$  Result of Algorithm 14
15:   timeUsedCr  $\leftarrow (\text{Duration of } \text{newSp}) - (\text{Duration of } \text{originalSp})$ 
16: else
17:   newSp  $\leftarrow$  first targetIndex states of originalSp
18:   Add new State at Start of Cruising
19:   Add new State at End of Cruising
20:   newIndex  $\leftarrow \text{index}$ 
21:   if newSpeed = speed at state after crState then
22:     newIndex  $\leftarrow \text{newIndex} + 1$ 
23:   end if
24:   for i  $\leftarrow \text{newIndex} + 1$  to nOfEntries - 1 do
25:     Append state at position i in originalSp delayed by timeUsedCr in newSp
26:   end for
27: end if

```

Algorithm 16 Construct the Speed Profile for Accelerating Alternative Case

Require: SpeedProfile *originalSp*, State *targetState*, int *newTargetIndex*, *targetIndex*, *index*, double *crSpeed*, *newSpeed*, *newTargetSpeed*, *targetSpeed*, *acc*, *dec*

Ensure: SpeedProfile *newSp*, double *timeUsedCr*

```

1: oldAccTime1  $\leftarrow \frac{(\text{speed at state at position } \text{newTargetIndex}+1 \text{ in } \text{originalSp}) - \text{newTargetSpeed}}{\text{acc}}$ 
2: oldCruiseTime1  $\leftarrow (\text{end time at } \text{targetState}) - (\text{end time at state at position } \text{newTargetIndex} + 1 \text{ in } \text{originalSp})$ 
3: oldAccTime2  $\leftarrow \frac{\text{crSpeed} - \text{targetSpeed}}{\text{acc}}$ 
4: oldCruiseTime2  $\leftarrow (\text{end time at state at position } \text{index} \text{ in } \text{originalSp}) - (\text{end time at } \text{startCrState})$ 
5: oldDecTime  $\leftarrow \frac{(\text{speed at state at position } \text{index}+1 \text{ in } \text{originalSp}) - \text{crSpeed}}{\text{dec}}$ 
6: oldTime  $\leftarrow \text{oldAccTime1} + \text{oldCruiseTime1} + \text{oldAccTime2} + \text{oldCruiseTime2} + \text{oldDecTime}$ 
7: newAccTime  $\leftarrow \frac{\text{newSpeed} - \text{newTargetSpeed}}{\text{acc}}$ 
8: newDecTime  $\leftarrow \frac{(\text{speed at state at position } \text{index}+1 \text{ in } \text{originalSp}) - \text{newSpeed}}{\text{dec}}$ 
9: totalDistance  $\leftarrow \text{newSpeed} \cdot (\text{oldAccTime1} - \text{newAccTime}) + 0.5 \cdot \text{acc} \cdot (\text{oldAccTime1} - \text{newAccTime})^2 + \text{distance traveled at state at position } \text{index}) - (\text{distance traveled at state at position } \text{newTargetIndex} + 1) + \text{crSpeed} \cdot (\text{oldDecTime} - \text{newDecTime}) + 0.5 \cdot \text{dec} \cdot (\text{oldDecTime} - \text{newDecTime})^2$ 
10: newCruiseTime  $\leftarrow \frac{\text{totalDistance}}{\text{newSpeed}}$ 
11: newTime  $\leftarrow \text{newAccTime} + \text{newCruiseTime} + \text{newDecTime}$ 
12: timeUsedCr  $\leftarrow \text{newTime} - \text{oldTime}$ 
13: if timeUsedCr > time then
14:   newSp  $\leftarrow$  Result of Algorithm 14
15:   originalSp  $\leftarrow$  newSp
16:   timeUsedCr  $\leftarrow$  (Duration of newSp) – (Duration of originalSp)
17: else
18:   newSp  $\leftarrow$  first targetIndex states of originalSp
19:   Add new State at Start of Cruising
20:   Add new State at End of Cruising
21:   newIndex  $\leftarrow$  index
22:   if newSpeed = speed at state after crState then
23:     newIndex  $\leftarrow$  newIndex + 1
24:   end if
25:   for i  $\leftarrow$  newIndex + 1 to nOfEntries – 1 do
26:     Append state at position i in originalSp delayed by timeUsedCr in newSp
27:   end for
28: end if

```

Algorithm 17 initializes the variables required for the execution of algorithm 18. It requires the original speed profile (*originalSp*); the state at the start of the acceleration (*targetState*); the state at the start of the cruising (*startCrState*); and the cruising speed in *originalSp* (*crSpeed*). Algorithm 18 applies a divide and conquer methodology to determine the new cruising speed of the modified *originalSp* (*speed*). The upper limit of *speed* is represented by *maxSpeed* and is initialized to *crSpeed*, while the lower limit is denoted by *minSpeed* and initialized to the train's speed at *targetState*

Algorithm 17 Initialize the Computations of Algorithm 18**Require:** SpeedProfile *originalSp*, State *targetState*, *startCrState*, double *crSpeed***Ensure:** double *maxSpeed*, *targetSpeed*, *minSpeed*, *speed*, *targetStateDistance*, *acc*, *accTime*, *ds*, *dec*, *decTime*, *de*

- 1: $maxSpeed \leftarrow crSpeed$
- 2: $targetSpeed \leftarrow \text{speed of } targetState$
- 3: $minSpeed \leftarrow \max(targetSpeed, \text{speed of state } index + 1 \text{ of } originalSp)$
- 4: $speed \leftarrow \frac{maxSpeed + targetSpeed}{2}$
- 5: $targetStateDistance \leftarrow \text{distance traveled until } targetState$
- 6: $deltaT \leftarrow (\text{end time at } startCrState) - (\text{end time at } targetState)$
- 7: $acc \leftarrow \frac{crSpeed - targetSpeed}{deltaT}$
- 8: $accTime \leftarrow \frac{deltaT \cdot speed - targetSpeed}{acc}$
- 9: $ds \leftarrow targetStateDistance + targetSpeed \cdot accTime + 0.5 \cdot acc \cdot accTime^2$
- 10: $deltaT \leftarrow (\text{end time at state at position } index + 1 \text{ in } originalSp) - (\text{end time at state at position } index \text{ in } originalSp)$
- 11: $dec \leftarrow \frac{(\text{speed at state at position } index + 1 \text{ in } originalSp) - crSpeed}{deltaT}$
- 12: $decTime \leftarrow \frac{speed - crSpeed}{dec}$
- 13: $de \leftarrow \text{distance traveled until state at position } index + 1 \text{ in } originalSp$

(*targetSpeed*). The variable *speed* is set equal to the mean of these two limits. The variable *targetStateDistance* represents the distance the locomotive has traveled until *targetState*. Next, the algorithm calculates the acceleration rate (*acc*); the time needed for the train to accelerate from *targetSpeed* to *speed* (*accTime*); and the distance traveled when it reaches *speed* during acceleration (*ds*). Similarly, it calculates the deceleration rate (*dec*); the time required for the train to decelerate from *crSpeed* to *speed* (*decTime*); and the distance covered when it reaches *speed* during deceleration (*de*).

Algorithm 18 depicts the divide and conquer approach of algorithm 14 to determine the new cruising speed. Its inputs are the SpeedProfile *originalSp*; the cruising speed in *originalSp* (*crSpeed*); the initialized new cruising speed (*speed*); the acceleration and deceleration rates (*acc* and *dec*, respectively); the durations of acceleration and deceleration in *originalSp* (*accTime* and *decTime*, respectively); the upper and lower limits of *speed* (*maxSpeed* and *minSpeed*, respectively); the distance the locomotive travels when it reaches *speed* during acceleration (*ds*); the distance the locomotive travels when it reaches *speed* during deceleration (*de*); the position of the state at the end of the cruising (*index*); and the extra available time (*timeUsedCr*). Its outputs are the updated *speed* and *accTime*. The functionality of the algorithm is nested in an infinite loop with a single breaking point. Each iteration calculates *totalDistance*, representing the total distance traveled until the end of the instance with a cruising speed of *speed*. If *totalDistance* equals *de*, the loop breaks, and the algorithm terminates. Otherwise, if *totalDistance* is greater than *de*, indicating that the locomotive travels farther than it must, it updates *maxSpeed* to *speed*. Conversely, if *totalDistance* is less than *de*, meaning the locomotive does not travel far enough, it updates *minSpeed* to *speed*. Finally, the algorithm proceeds to the subsequent iteration by updating *speed*, *accTime*, *decTime*, and *ds* according to the corresponding calculations in algorithm 17.

Algorithm 18 Computations of Algorithm 14

Require: SpeedProfile *originalSp*, double *crSpeed*, *speed*, *acc*, *dec*, *timeUsedCr*, *decTime*, *ds*, *de*, *maxSpeed*, *minSpeed*, int *index*

Ensure: double *speed*, *accTime*

```

1: while true do
2:    $\delta T \leftarrow ((\text{end time at state at position } index \text{ in } originalSp) - (\text{end time at state at } startCrState)) + \frac{crSpeed - speed}{acc} + \frac{speed - crSpeed}{dec} + timeUsedCr$ 
3:    $dTraveled \leftarrow speed \cdot \delta T + speed \cdot decTime + 0.5 \cdot dec \cdot decTime^2$ 
4:    $totalDistance \leftarrow ds + dTraveled$ 
5:   if  $totalDistance = de$  then
6:     break
7:   end if
8:   if  $totalDistance > de$  then
9:      $maxSpeed \leftarrow speed$ 
10:  else
11:     $minSpeed \leftarrow speed$ 
12:  end if
13:   $speed \leftarrow \frac{maxSpeed + minSpeed}{2}$ 
14:   $accTime \leftarrow \frac{speed - targetSpeed}{acc}$ 
15:   $decTime \leftarrow \frac{(speed \text{ at state at position } index+1 \text{ in } originalSp) - speed}{dec}$ 
16:   $ds \leftarrow targetStateDistance + targetSpeed \cdot accTime + 0.5 \cdot acc \cdot accTime^2$ 
17: end while

```

Chapter 5

Results

In this chapter, we present the outcomes of our research, demonstrating the functionality of our proposed solution through detailed examples. The following sections present diverse examples to better illustrate the behaviour of our algorithms. These examples use the train defined by table 5.1. The first three examples showcase the algorithm's behaviour against "Acceleration, Cruising and Deceleration" instances, while the fourth and fifth examples demonstrate the algorithm's behaviour against "Acceleration and Deceleration" instances. The last example presents a versatile time-optimal speed profile to better comprehend the nature of our approach. For a more graspable demonstration of the algorithms' behaviour, its accuracy will be limited to five decimal points.

Parameter	Value
Length	150 m
Maximum Speed	75 m/s
Maximum Acceleration Rate	1.5 m/s ²
Maximum Deceleration Rate	1 m/s ²

Table 5.1: Train 2

5.1 Example 1

5.1.1 Description

This example illustrates a scenario where the time-optimal speed profile consists of two consecutive "Acceleration, Cruising and Deceleration" instances, with the locomotive cruising for the same duration but at different speeds. The problem of this example is described in table 5.2. Figure 5.2 demonstrates the fuel consumption over time for the time-optimal speed profile. The total fuel consumption of the speed profile is 4678.833 units.

Figure 5.1 depicts the time-optimal speed profile along with the distance it travels.

Parameter	Value
Train	Train 3 (Table 5.1)
Path Speed	Path 5 (Table 5.3)
Initial State	(3570.833, 0, [1]))
Goal State	(0, 0, [3]))
Original Speed Profile	Speed Profile 5 (Figure 5.1)
Extra Time	20 sec

Table 5.2: Problem 5

Segment	Length (m)	Speed Limit (m/s)
1	3720.833	65
2	1541.667	50
3	5504.167	92

Table 5.3: Path 5 Layout

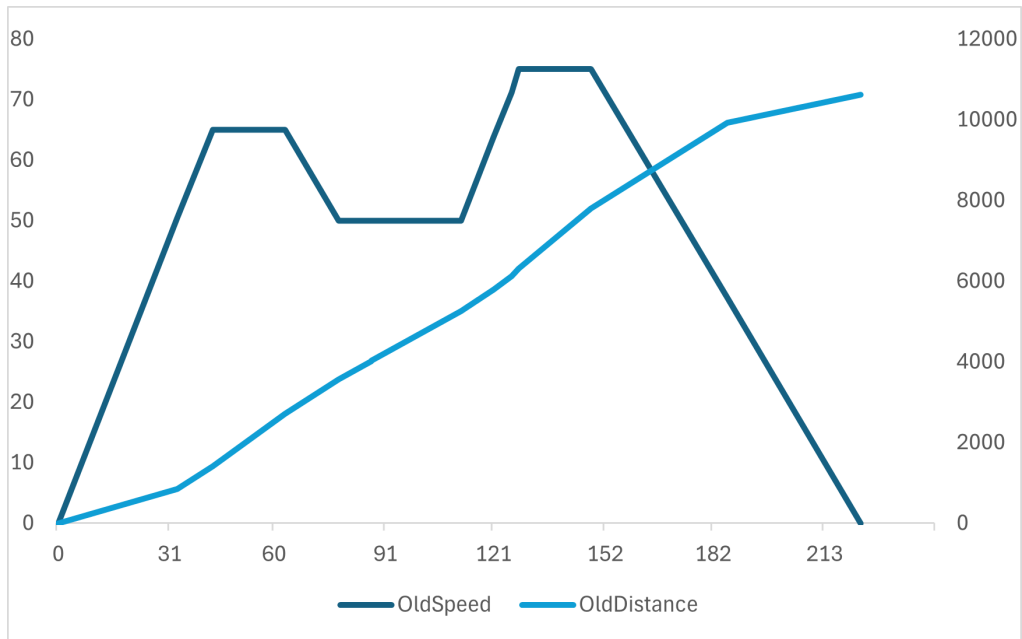


Figure 5.1: Time-Optimal Speed Profile of Example 1

Both cruising instances last 20 seconds, while the train cruises at 65m/s in the first instance and at 75m/s in the second. It is apparent that the speed profile consists of two "Acceleration, Cruising and Deceleration" instances joined by a "Deceleration, Cruising and Acceleration" instance. The speed profile's duration is 223.833 seconds, covering 10616.667 meters. The algorithm is given an extra 20 seconds to apply the fuel-optimal adjustments.

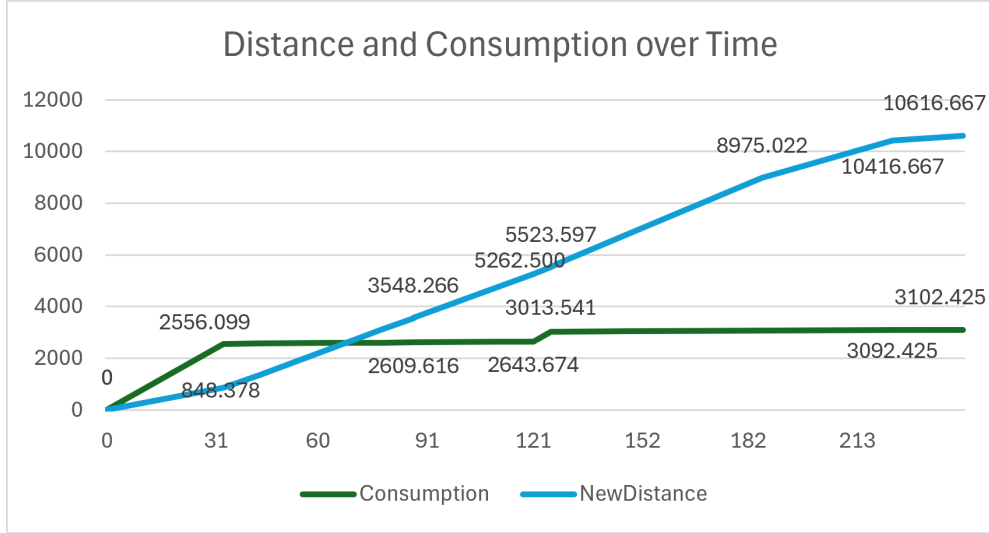


Figure 5.2: Fuel Consumption of Time-Optimal Speed Profile of Example 1

5.1.2 Analysis

Due to the form of the time-optimal speed profile, algorithm 2 will not be executed, meaning that the variable *accToDecTradeOff* will always equal -1 . On the other hand, algorithm 3 will be executed on every iteration. The following analysis and discussion examine the behaviour of only the latter algorithm.

After the necessary initializations described in the previous chapter, algorithm 5 acquires the states at the end of each cruising. To simplify things, since the numbering in Java starts from zero, the position of the first state corresponds to number 0, the position of the second state corresponds to number 1, etc. Based on that, the states at the end of the cruising are at positions 2 and 8. Next, follows the calculation of the time-fuel trade-offs of the modified speed profiles.

Regarding the first "Acceleration, Cruising and Deceleration" instance, the variable *targetState* is set to the state at position 0, where the acceleration starts. The cruising speed in *originalSp* is 65m/s, meaning that the new cruising speed (*newSpeed*) is set to $65 - 0.1 = 64.9$ m/s. This value is higher than both the speed at the start and at the end of the instance, indicating that the modification refers to a normal case. The durations of the acceleration, cruising and deceleration in *originalSp* are calculated to be 43.33333, 20 and 15 seconds, respectively, meaning that the instance lasts for 78.33333 seconds. The new acceleration ends at 64.9, so its duration is 43.26667 seconds, while the new deceleration starts from 64.9m/s, making its duration 14.9 seconds. In the modified speed profile, the locomotive must cruise until it travels the distance it covers in *originalSp* when it reaches 64.9m/s during deceleration. This distance is denoted by *totalDistance* and is calculated to be 1310.8245 meters. As a result, the cruising duration is $\frac{1310.8245}{64.9} = 20.19761$ seconds, making the total duration of the modified instance 78.36428 seconds. The variable *timUsedCr* is updated to the extra time this modification requires, which is equal to $78.36428 - 78.33333 = 0.03095$ seconds. Since the required time is less than the available extra time, the construction of *newSp* proceeds with the discussed values. The fuel consumption of *newSp* is calculated to be 4673.91428 units and its total duration is 223.86428 seconds. Thus, its trade-off is $\frac{4673.91428}{223.86428} = 20.87834$, which is larger than *crTradeOff*. As a result, *crTradeOff* is

updated to 20.87834, $crSp$ to $newSp$, and $crTime$ to 0.03095. Next, the algorithm proceeds to the modification of the second instance.

At the next iteration of algorithm 3, the state under consideration is at position 6. In this instance, the cruising speed in $originalSp$ is 75m/s, so the new cruising speed ($newSpeed$) is set to $75 - 0.1 = 74.9$ m/s. Similarly to the modification of the first instance, $newSpeed$ is higher than the speeds at the start and at the end of the instance, so it is also a normal case. After computing the respective calculations, $timeUsedCr$ is updated to 0.02681 seconds. Once again, $timeUsedCr$ is less than $time$, so algorithm 14 is not executed and $newSp$ is constructed according to the calculated values. The fuel consumption of $newSp$ is calculated to be 4673.91015 units and its total duration is 223.86015 seconds. Thus, its trade-off is $\frac{4673.91015}{223.86015} = 20.87871$, which is larger than the current value of $crTradeOff$ (20.87834). As a result, $crTradeOff$ is updated to 20.87871, $crSp$ to $newSp$, and $crTime$ to 0.02681. Since there is not another "Acceleration, Cruising and Deceleration" instance, the algorithm updates $fuelOptimalSp$ and $originalSp$ to $crSp$ and reduces $time$ by $crTime$ making it equal to 19.97319 seconds. The remaining iterations consist only of normal cases, where the described behaviour repeats. The only difference, apart from the changes in the inferior and superior trade-offs between the two modified instances, is the invocation of algorithm 14.

Algorithm 14 is invoked if $time$ is not sufficient for the modification of an instance, which happens only at an instance's final modification. In this example, the cruising speeds have been reduced to 50.5 and 57.3, respectively. The remaining extra available time is 0.05366 seconds, yet the modification of the first instance, which lowers the cruising speed to 50.4 requires 0.10608 seconds. Algorithm 14 provides the best speed profile as a modification of a single instance, using all the remaining time. Once it initializes its variables and acquires the position of the states at the end of the first and second cruising, it proceeds to its loop.

At the first iteration of algorithm 14, the algorithm adjusts the instance described by the state at position 2 in $originalSp$. It computes the position of $targetState$ and initializes the variables used in its computational part. Its computational part is an implementation of divide and conquer, where it calculates the new cruising speed ($speed$) by converging its upper and lower limits ($maxSpeed$ and $minSpeed$, respectively) together according to the satisfaction of an equation. This equation ensures that the distance covered at the end of the original instance and the distance of the modified instance are equal. These values are represented by variables de and $totalDistance$, respectively. de is equal to 3570.83333 meters. $maxSpeed$ is initialized to the instance's current cruising speed (50.5m/s), while $minSpeed$ is initialized to the train's speed at the end of the deceleration (50m/s). Consequently, $speed$ is initialized to $\frac{50.5+50}{2} = 50.25$. At that speed, $totalDistance$ is 3560.13302 meters, making it less than de . Since the locomotive cannot travel the necessary distance at 50.25m/s, its lower limit is increased to 50.25 and $speed$ is updated to $\frac{50.5+50.25}{2} = 50.375$. This procedure continues until $speed$ is set to 50.44932, where $totalDistance$ equals de . $newSp$ is constructed according to these values having a consumption of 3102.42489 fuel units and lasting 243.83333 seconds. Its trade-off is $\frac{3102.42489}{243.83333} = 12.72355$, which is greater than the initialized value of $maxTradeOff$, so $maxTradeOff$ and $crSp$ are updated accordingly.

On the next iteration, the algorithm adjusts the second "Acceleration, Cruising and Deceleration" instance. For this instance de is equal to 10616.66667 meters, $maxSpeed$ is initialized to 57.3m/s, $minSpeed$ is initialized to 50m/s, and as a result $speed$ is set to $\frac{57.3+50}{2} = 53.65$. At that speed, $totalDistance$ is 9138.5883, which is less than 10616.66667, $speed$'s lower limit is increased to 53.65m/s and $speed$ is updated to $\frac{57.3+53.65}{2} = 55.475$ m/s. This procedure continues until $speed$ is set to 57.24903m/s, where $totalDistance$ equals de . $newSp$ is then constructed according to these values, having a consumption of 3102.41056 fuel units and lasting 243.83333 seconds. Its trade-off is $\frac{3102.41056}{243.83333} = 12.72349$, which is less than $maxTradeOff$. Therefore, $maxTradeOff$ and $crSp$ do not get updated, and the algorithm returns the speed profile with the first instance modified.

At that moment, $crTradeOff$ is equal to its initialized value (-1), so $crSp$, $crTradeOff$, and $crTime$ are updated. Consequently, algorithm 14 is invoked once more, because the modification of the second instance also requires more extra time than the remaining. Since the algorithm is invoked with the same data, it produces the same speed profile with the same trade-off. Finally, $fuelOptimalSp$ is updated to the calculated $crSp$.

Having utilized all the available time, algorithm 1 returns the $fuelOptimalSp$ depicted in figure 5.3. In particular, figure 5.7 compares the time-optimal and the fuel-optimal speed profiles, along with the distance covered over time. The algorithm reduces both cruising speeds with a slight preference for the initially higher cruising speed. Specifically, the second cruising speed was reduced by 1.55% more. Additionally, figure 5.4 demonstrates the fuel consumption over time for the fuel-optimal speed profile. Its overall fuel consumption is 3102.42489 units over 243.83333 seconds.

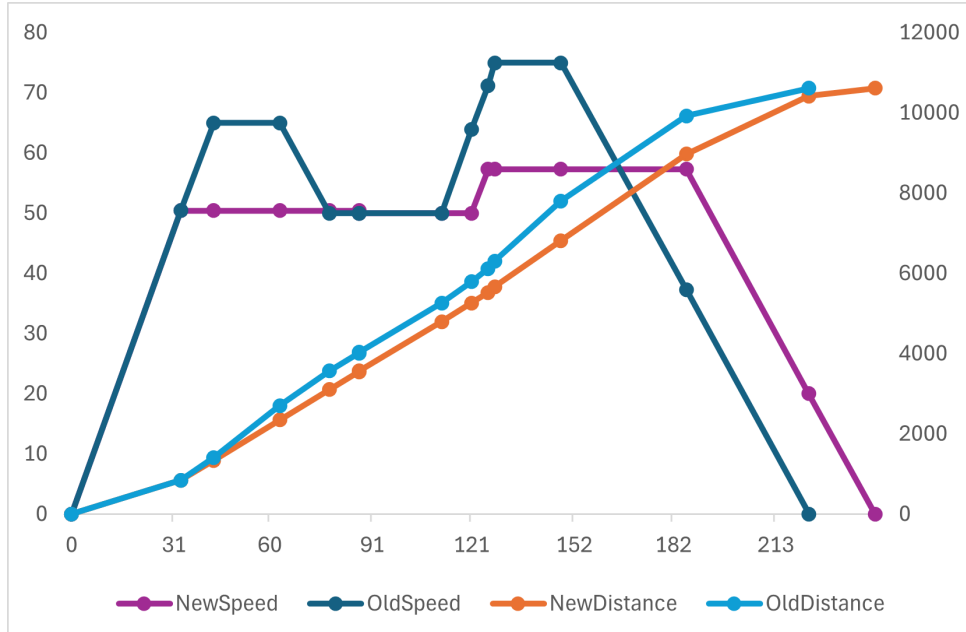


Figure 5.3: Fuel-Optimal Speed Profile of Example 1

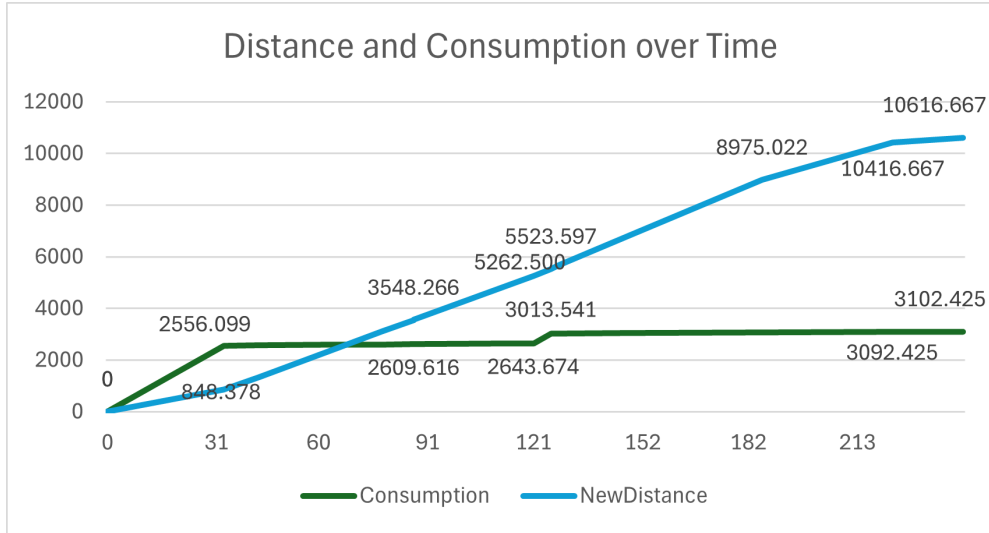


Figure 5.4: Fuel Consumption of Fuel-Optimal Speed Profile of Example 1

5.2 Example 2

5.2.1 Description

This example demonstrates a time-optimal speed profile that consists of two consecutive "Acceleration, Cruising and Deceleration" instances where the locomotive cruises at the same speed, but for different durations in each instance. The problem of this example is detailed in table 5.4. Figure 5.6 showcases the fuel consumption over time for the time-optimal speed profile. The total fuel consumption of the speed profile is 5240.278 units.

Parameter	Value
Train	Train 3 (Table 5.1)
Path Speed	Path 6 (Table 5.5)
Initial State	(1000, 0, [1]))
Goal State	(0, 0, [4]))
Original Speed Profile	Speed Profile 6 (Figure 5.5)
Extra Time	10 sec

Table 5.4: Problem 6

Segment	Length (m)	Speed Limit (m/s)
1	1150	100
2	3025	76
3	2975	50
4	8000	100

Table 5.5: Path 6 Layout

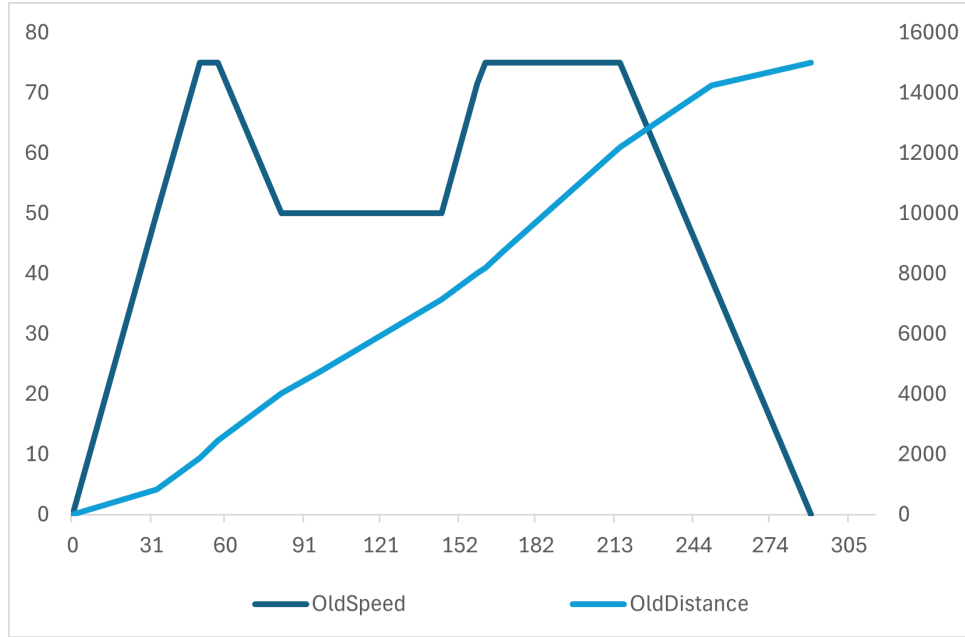


Figure 5.5: Time-Optimal Speed Profile of Example 2

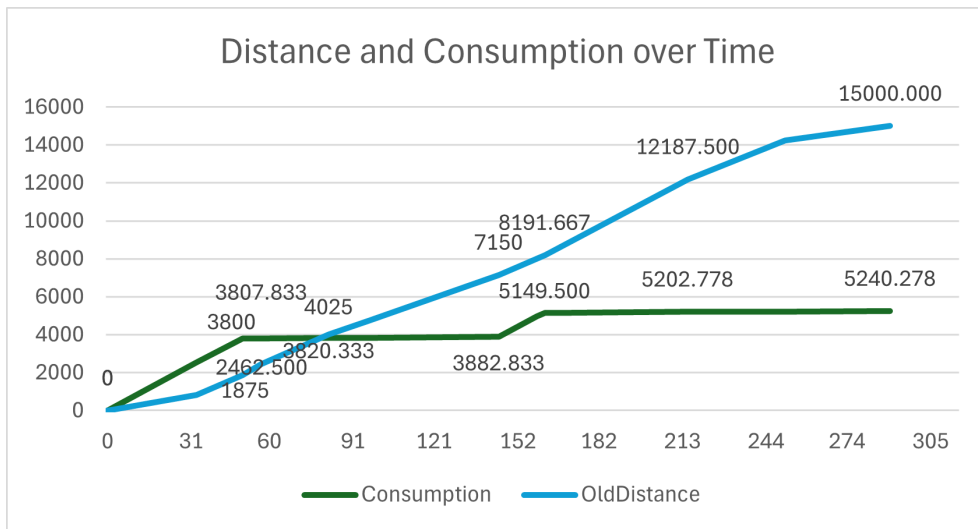


Figure 5.6: Fuel Consumption of Time-Optimal Speed Profile of Example 2

Figure 5.5 illustrates the time-optimal speed profile along with the distance it travels. In both cruising, the train travels at 75m/s. The first cruising lasts only 7.833 seconds, while the second lasts 53.278 seconds. Similarly to the time-optimal speed profile of the previous example, this one consists of two "Acceleration, Cruising and Deceleration" instances joined by a "Deceleration, Cruising and Acceleration" instance. The speed profile's total duration is 290.278 seconds, covering 15000 meters. The algorithm must utilize up to 25 extra seconds to minimize its fuel consumption.

5.2.2 Analysis

Similarly to example 1, algorithm 2 will not be detailed in this example because its time-optimal speed profile does not contain any "Acceleration and Deceleration" instances. On the other hand, algorithm 3 will be executed on every iteration. The following analysis and discussion examine the behaviour of algorithm 3 against problem 6.

First, algorithm 3 initializes the variables *timeUsedCr* and *crTradeOff*, representing the time it utilizes and the modified speed profile's time-fuel trade-off, and acquires the states at the end of the cruising of the two instances. It first examines the instance with the short cruising. Its cruising speed is 75m/s, so the algorithm sets *newSpeed* to $75 - 0.1 = 74.9$ m/s. Since *newSpeed* is greater than the locomotive's speed at the start (0m/s) and at the end (50m/s) of the instance, it proceeds to handle it as a normal case. Once it has computed the duration of the original instance (82.83333 seconds), it calculates the duration of the new acceleration, which ends at 74.9m/s (49.93333 seconds) and the duration of the new deceleration, which starts from 74.9m/s (24.9 seconds). The distance the train must cover is the distance it travels in *originalSp* when it reaches 74.9m/s during deceleration. This distance is equal to 599.99167 meters and is represented by *totalDistance*. Since the train must travel 599.99167 meters while cruising at 74.9m/s it must cruise for $\frac{599.992}{74.9} = 8.01057$ seconds. The total duration of the modified instance, computed as the sum of the new durations is 82.8439 seconds, meaning that the overtime of the modification is $82.8439 - 82.83333 = 0.01057$ seconds. Since the remaining extra time exceeds the time the modification requires, *newSp* is constructed according to the calculated values. The fuel consumption of *newSp* is calculated to be 5235.33835 units and its total duration is 290.28835 seconds. Thus, its trade-off is $\frac{5235.33835}{290.28835} = 18.03496$, which is larger than *crTradeOff*. As a result, *crTradeOff* is updated to 18.03496, *crSp* to *newSp*, and *crTime* to 0.01057. Next, the algorithm proceeds to the modification of the second instance.

The steps for modifying the second instance are identical to those for the previous instance. The algorithm acquires *targetState* and initializes the necessary variables. The cruising speed in *originalSp* is also 75m/s, so *newSpeed* is again set to 74.9m/s. Once again, *newSpeed* is greater than the speeds at the start and at the end of the instance, so it is a normal case. After computing the respective calculations, *timeUsedCr* is updated to 0.07124 seconds. Since *timeUsedCr* is less than *time*, *newSp* is constructed as previously demonstrated. The fuel consumption of *newSp* is calculated to be 5235.39902 units and its total duration is 290.34902 seconds. Thus, its trade-off is $\frac{5235.39902}{290.34902} = 18.0314$, which is not greater than 18.03496 (i.e., the current value of *crTradeOff* set in the previous iteration of algorithm 3). Since there is not another "Acceleration, Cruising and Deceleration" instance in *originalSp*, the algorithm terminates. *fuelOptimalSp* and *originalSp* are updated to *crSp* and *time* gets reduced by *crTime* making it equal to 24.98943 seconds.

The subsequent iterations consist only of normal cases, where the algorithm repeats the previously described steps. The main difference, apart from the changes in the inferior and superior trade-offs between the two modified instances, is the invocation of algorithm 14. This algorithm is invoked only once to adjust the second "Acceleration, Cruising and Deceleration" instance. Handled as a normal case, the modification requires 0.12833 seconds, which is greater than the remaining extra time (i.e., 0.06236 seconds). At this iteration, *de* is equal to 15000 meters. The variable *maxSpeed* is initialized to 64.1m/s, while *minSpeed* is initialized to 50m/s. The variable *speed* is initialized to the mean of its limits, i.e., $\frac{64.1+50}{2} = 57.05$. At that speed, *totalDistance* is 13133.69514 meters, which is less than *de*. Since the locomotive cannot travel the necessary distance at the current value of *speed*, its lower limit is increased to 57.05m/s and *speed* is

updated to $\frac{64.1+57.05}{2} = 60.575\text{m/s}$. This procedure continues until *speed* is set to 64.05134m/s , where *totalDistance* equals 15000 meters. The new speed profile is constructed according to these values, with a fuel consumption of 3485.81927 units and a duration of 315.27778 seconds. Its trade-off is $\frac{3485.81927}{315.27778} = 11.05634$, which is greater than -1 . Therefore, *maxTradeOff* and *crSp* are updated accordingly.

At that moment, *crTradeOff* is also equal to its initialized value (-1), so *crSp*, *crTradeOff*, and *crTime* are updated accordingly. Consequently, *fuelOptimalSp* is set equal to *crSp* as well. Having utilized all the available time, algorithm 1 returns the *fuelOptimalSp* illustrated in figure 5.7. Specifically, it compares the time-optimal and the fuel-optimal speed profiles, along with the distance covered in this example over time. The algorithm reduces both cruising speeds, with a preference for the initially shorter cruising. The reduction transforms the first "Acceleration, Cruising and Deceleration" instance into an "Acceleration, Cruising and Acceleration" instance. Additionally, figure 5.8 demonstrates the fuel consumption over time for the fuel-optimal speed profile. Its overall fuel consumption is 3485.819 units over 315.278 seconds.

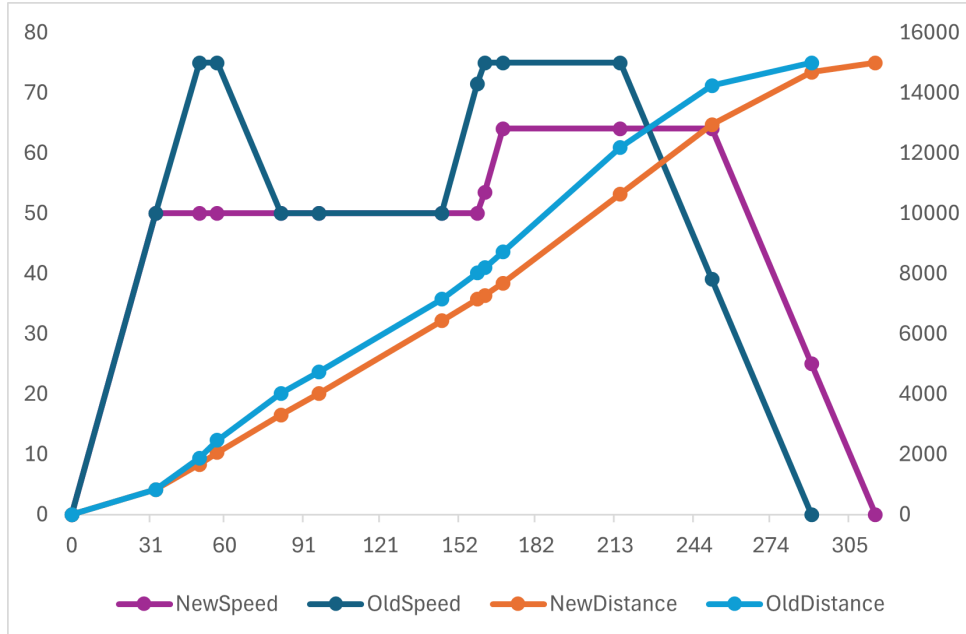


Figure 5.7: Fuel-Optimal Speed Profile of Example 2

5.3 Example 3

5.3.1 Description

This example delves into another aspect of algorithm 3. The time-optimal speed profile is a middle ground between examples 1 and 2, as it contains two consecutive "Acceleration, Cruising and Deceleration" instances where the locomotive cruises for different durations and at a different speeds. The problem of this example is described in table 5.6. Figure 5.10 demonstrates the fuel consumption over time for the time-optimal speed profile. The total fuel consumption of the speed profile is 4176.485 units.

Figure 5.9 depicts the time-optimal speed profile along with the distance it travels.

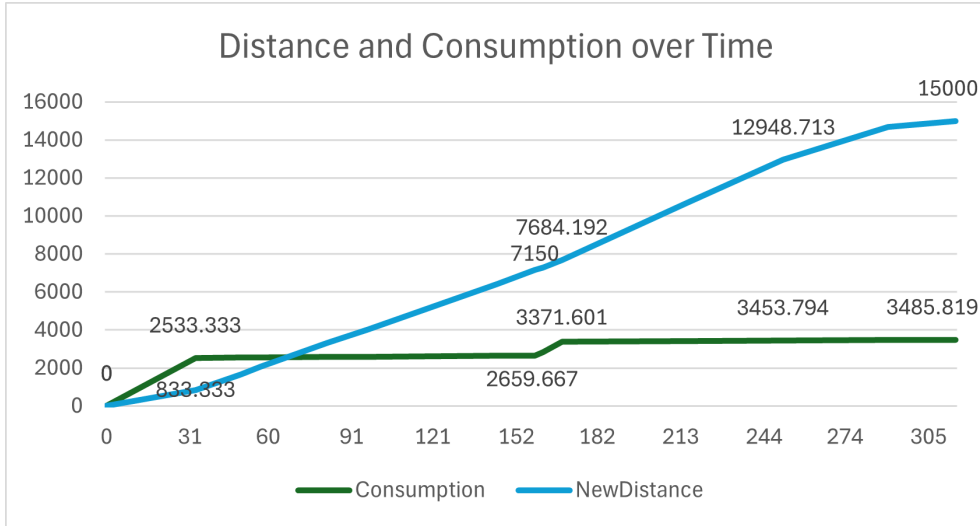


Figure 5.8: Fuel Consumption of Fuel-Optimal Speed Profile of Example 2

Parameter	Value
Train	Train 3 (Table 5.1)
Path Speed	Path 7 (Table 5.7)
Initial State	(5000, 0, [1]))
Goal State	(0, 0, [3]))
Original Speed Profile	Speed Profile 7 (Figure 5.9)
Extra Time	20 sec

Table 5.6: Problem 7

Segment	Length (m)	Speed Limit (m/s)
1	5150	100
2	1000	50
3	4000	55

Table 5.7: Path 7 Layout

The first cruising lasts 20.833 seconds with the locomotive's speed is 75m/s, while in the second instance the train cruises at 55m/s for 39.318 seconds. It is apparent that the speed profile consists of two "Acceleration, Cruising and Deceleration" instances joined by a "Deceleration, Cruising and Acceleration" instance. The speed profile's duration is 216.485 seconds and it covers 10000 meters. The algorithm is given an extra 15 seconds to apply the fuel-optimal adjustments.

5.3.2 Analysis

Due to the form of the time-optimal speed profile, algorithm 2 will not be executed, meaning that the variable *accToDecTradeOff* will always equal -1 . On the other hand, algorithm 3 will be executed in every iteration. Since most functionalities of the

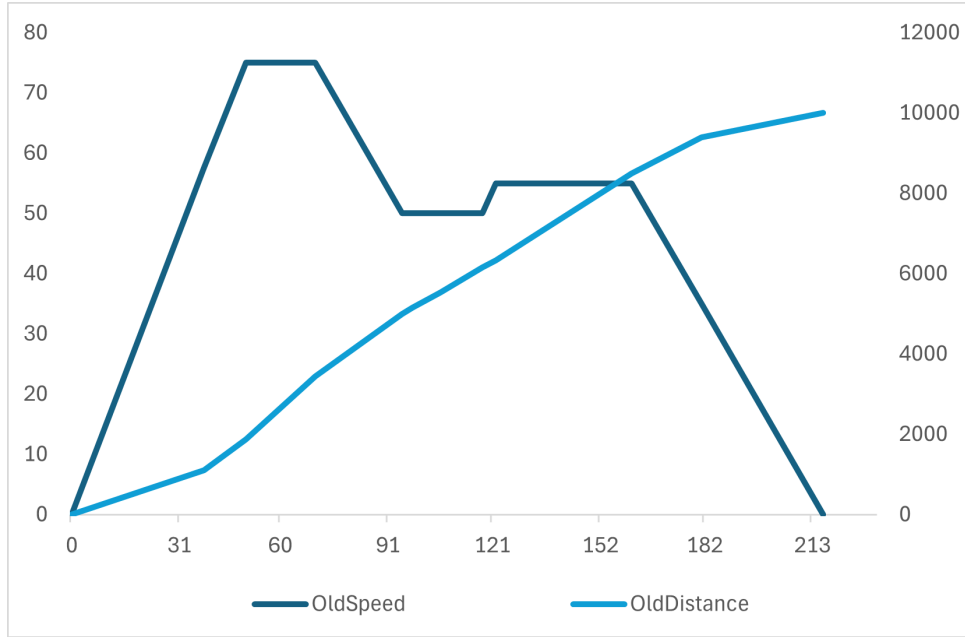


Figure 5.9: Time-Optimal Speed Profile of Example 3

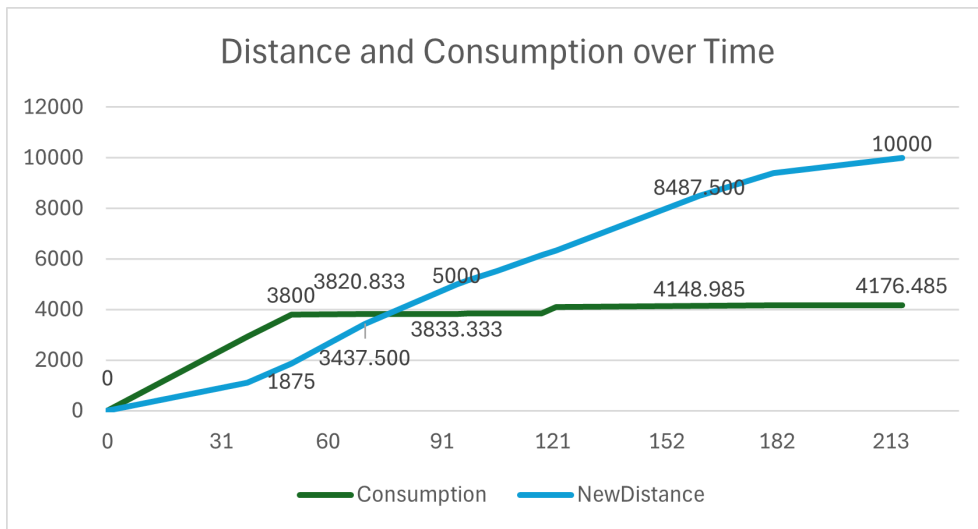


Figure 5.10: Fuel Consumption of Time-Optimal Speed Profile of Example 3

latter algorithm have been demonstrated, this analysis will detail some of the functionalities that were not demonstrated on examples 1 and 2.

After 220 iterations of algorithm 1, the cruising speed of the second instance has been reduced to a little less than 50.1m/s and time is set to 0.39374 seconds. In the following iteration, *newSpeed* is set to almost 50m/s, yet less than the locomotive's speed at the start of the instance. Additionally, *newSpeed* is also less than 57.9m/s, which is the locomotive's speed before the cruising that is prior to the "Acceleration, Cruising and Deceleration" instance. As a result, the modification refers to an alternative case where the previous non-cruising throttle setting is deceleration and *newSpeed* gets updated to exactly 50m/s. These cases are handled by algorithm 15.

Algorithm 15 proceeds by calculating the difference in time between the aforementioned state where the train's speed is 57.9m/s (*newTargetSpeed*) and the end of the instance's deceleration. This duration (represented by *oldTime*) is equal to the sum of

the deceleration from 57.9m/s to 50m/s ($\frac{40-57.9}{-1} = 7.9$ seconds), the duration of the cruising in *originalSp* (23 seconds), the duration of the acceleration from 50m/s to 50.1m/s ($\frac{50.1-50}{1.5} = 0.06667$ seconds), the duration of the instance's cruising (51.72971 seconds), and the duration of the deceleration from 50.1m/s to the locomotive's speed at the state after *crState* ($\frac{0-50.1}{-1} = 50.1$ seconds). In the modified speed profile, the first deceleration does not change, while the second deceleration starts from 50m/s, making its duration equal to 50 seconds. The locomotive must cruise until it covers the distance it travels in *originalSp* when it reaches 50m/s during deceleration. This distance is denoted by *totalDistance* and is calculated to be 3750 meters. As a result, its duration is $\frac{3750}{50} = 75$ seconds, making the total duration of the modified instance 132.9 seconds. The variable *timUsedCr* is updated to the extra time the modification requires, which is equal to $132.9 - 132.7963739188296 = 0.10363$ seconds. Since the modification's extra time is less than the available extra time (i.e., 0.39374 seconds), *newSp* is constructed according to the computed values. The fuel consumption of *newSp* is calculated to be 3097.24473 units and its total duration is 231.19473 seconds. Thus, its trade-off is $\frac{3097.24473}{231.19473} = 13.39669$, which is less than the time-fuel trade-off of the previously calculated "Slower Constant Speed" modification (13.3967 seconds). As a result, the modification is not applied in this iteration. Instead, it is applied in the next one where its trade-off is higher than the rest. Specifically, *crTradeOff* is updated to 13.36975, *crSp* to *newSp*, and *crTime* to 0.10363.

The remaining iterations involve functionalities that have already been demonstrated in examples 1 and 2. Once it has utilized all the available time, algorithm 1 returns the *fuelOptimalSp* illustrated in figure 5.11. Figure 5.11 compares the time-optimal and the fuel-optimal speed profiles along with the distance covered over time. The algorithm reduces both cruising speeds to the point where it transforms the second "Acceleration, Cruising and Deceleration" instance into an unalterable "Deceleration, Cruising and Deceleration". Figure 5.12 demonstrates the fuel consumption over time for the fuel-optimal speed profile. Its overall fuel consumption is 3083.727 units over 231.485 seconds

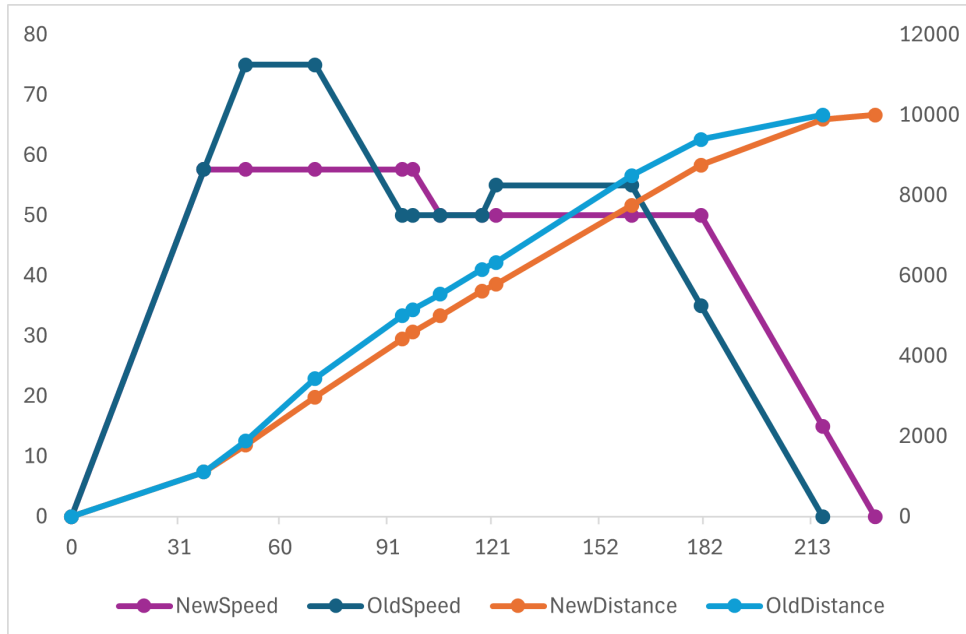


Figure 5.11: Fuel-Optimal Speed Profile of Example 3

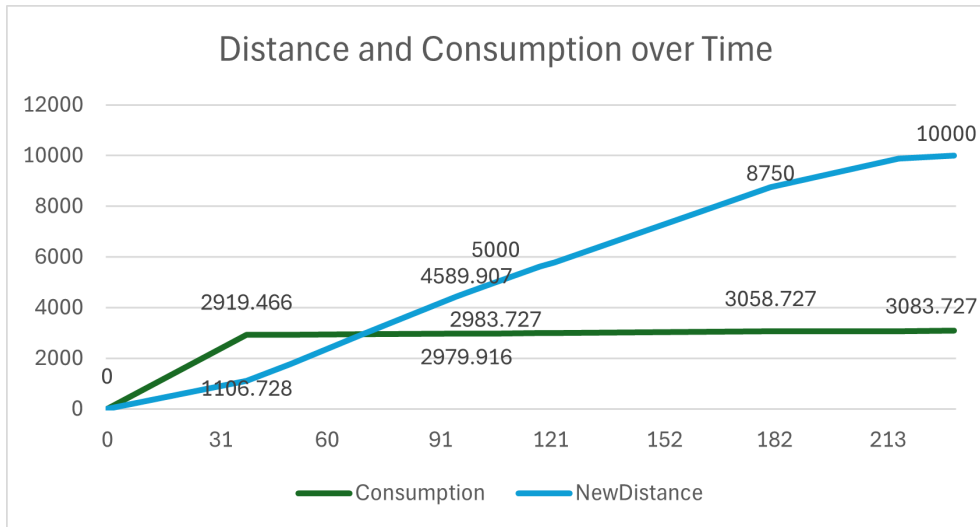


Figure 5.12: Fuel Consumption of Fuel-Optimal Speed Profile of Example 3

5.4 Example 4

5.4.1 Description

This example provides a time-optimal speed profile that gradually accelerates into two "Acceleration and Deceleration" instances, ending with a sharp deceleration. The problem of this example is detailed in table 5.8. Figure 5.14 illustrates the fuel consumption over time for the time-optimal speed profile. The total fuel consumption of the speed profile is 2887.981 units.

Parameter	Value
Train	Train 3 (Table 5.1)
Path Speed	Path 8 (Table 5.9)
Initial State	(150, 0, [1]))
Goal State	(0, 0, [7]))
Original Speed Profile	Speed Profile 8 (Figure 5.13)
Extra Time	10 sec

Table 5.8: Problem 8

Figure 5.13 depicts the time-optimal speed profile along with the distance it travels. The highest speeds of the two "Acceleration and Deceleration" instances are 42.19005m/s and 53.47897m/s, respectively. The speed profile's duration is 132.365 seconds and it covers 3800 meters.

5.4.2 Analysis

The nature of the time-optimal speed profile necessitates the execution of only algorithm 2 in the initial steps of algorithm 1. Once both "Acceleration and Deceleration" instances are transformed into "Acceleration, Cruising and Deceleration", algorithm 3 will be

Segment	Length (m)	Speed Limit (m/s)
1	300	20
2	150	20
3	700	80
4	800	40
5	1000	85
6	200	80
7	600	100
8	200	20

Table 5.9: Path 8 Layout

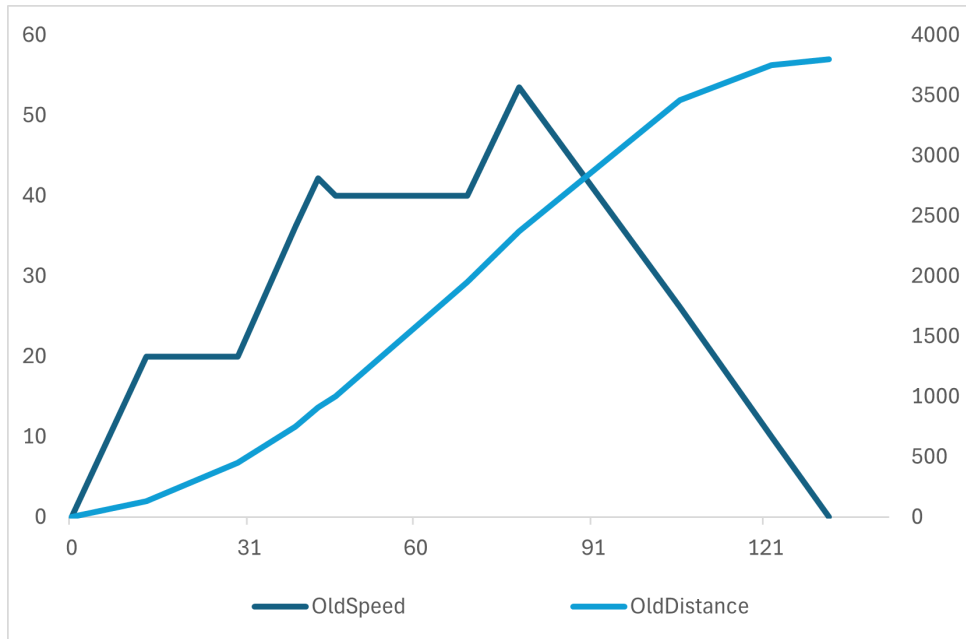


Figure 5.13: Time-Optimal Speed Profile of Example 4

executed in every iteration. The following analysis and discussion examine focus primarily on the behaviour of the former algorithm.

First, the algorithm invokes algorithm 7 to obtain the states at the peak of the "Acceleration and Deceleration" instances. The state at the end of the acceleration of the first instance is at position 5, while the second one is at position 10 in *originalSp*. Moving on, it computes the modified speed profile for both instances on each invocation.

At the first iteration, the algorithm invokes algorithm 4 with 0.1 seconds to transform a state using as much time as possible. Algorithm 4 first adjusts the "Acceleration and Deceleration" instance described by the state at position 5. The algorithm computes the position of *targetState* and initializes the variables used in its computational part. This computational part implements a divide and conquer methodology to calculate *crSpeed* by iteratively adjusting its upper and lower limits (*maxSpeed* and *minSpeed*,

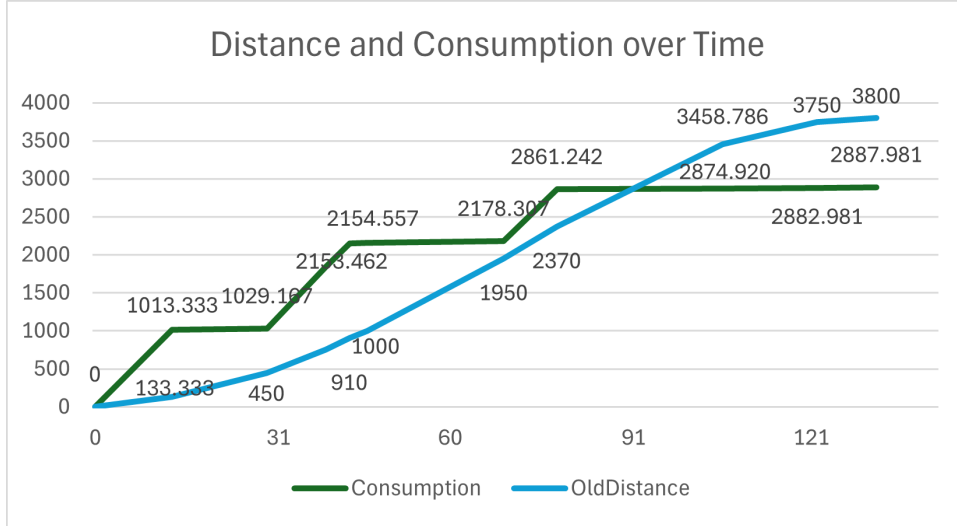


Figure 5.14: Fuel Consumption of Time-Optimal Speed Profile of Example 4

respectively) until a specific equation is satisfied. The equation ensures that the distance traveled at the end of the cruising in the modified speed profile is equal to the distance the locomotive covers when it reaches $crSpeed$ during deceleration in $originalSp$. These distances are represented by variables de and $totalDistance$. $maxSpeed$ is initialized to the instance's highest speed (42.19005m/s), while $minSpeed$ is initialized to the train's speed at the end of the deceleration (40m/s). Consequently, $crSpeed$ is initialized to $\frac{42.19005+40}{2} = 41.09502$ m/s and de is set to 955.59954 meters. At that speed, $totalDistance$ is 958.70981 meters, making it greater than de . Because the locomotive travels farther than the necessary distance at 41.09502m/s, the upper limit is decreased to 41.09502 and $crSpeed$ is updated to $\frac{41.09502+40}{2} = 40.54751$ m/s. This procedure continues until $crSpeed$'s limits converge to 40m/s. At that point, $totalDistance$ is 1000.00308 meters, while de is 1000 meters, indicating that the algorithm cannot utilize all the remaining extra time on this instance, so it invokes itself with $0.1 - 0.01 = 0.09$ seconds. Executing the same steps, $crSpeed$ ends up equal to 40.10876m/s, where both $totalDistance$ and de equal 995.64363 meters and the cruising lasts for 3.55881 seconds. The fuel consumption of this modified speed profile, constructed according to the calculated values, is 2785.04762 units and its duration is 132.45502 seconds. Its trade-off is $\frac{2785.04762}{132.45502} = 21.02636$, which is more than -1 , so $accToDecTradeOff$, $accToDecSp$, and $accToDecTime$ are updated accordingly.

At the next iteration, the algorithm adjusts the second "Acceleration and Deceleration" instance with $time$ being 0.09. On this instance, the upper limit is initialized to 53.47897m/s, the lower limit is initialized to 40m/s, and as a result, $crSpeed$ is set to $\frac{53.47897+40}{2} = 46.73948$. At that speed, $totalDistance$ is 2674.06634 meters, which is less than the distance the locomotive covers when it reaches 46.73948m/s during deceleration in $originalSp$ (de). At this point, de equals 2707.71032. Since $totalDistance$ is less than de , $minSpeed$ is increased to 46.73948m/s and $crSpeed$ is updated to $\frac{53.47897+46.73948}{2} = 50.10923$ m/s. This procedure continues until $crSpeed$ is set to 51.12909m/s, where both $totalDistance$ and de equal 2492.90825 meters and the locomotive cruises for 4.22331 seconds. The fuel consumption of the modified speed profile, constructed according to the calculated values, is 2771.75210 units and its duration is 132.45502 seconds. Its time-fuel trade-off is $\frac{2771.75210}{132.45502} = 20.92599$, which is less than the current value of $accToDecTradeOff$. Consequently, $accToDecTradeOff$,

$accToDecSp$, and $accToDecTime$ are not updated. Since the algorithm iterates through all the "Acceleration and Deceleration" instances, it returns the computed $accToDecSp$, which then updates the variable $accToDecSp$ in algorithm 2.

On this iteration of algorithm 1, $crTradeOff$ equals its initialized value, so the variable $fuelOptimalSp$ is updated to $accToDecSp$ and $time$ to 9.91. The next iteration of algorithm 1 contains both an "Acceleration, Cruising and Deceleration" and an "Acceleration and Deceleration" instance. That iteration transforms the first "Acceleration and Deceleration" instance into an "Acceleration, Cruising and Deceleration" instance as demonstrated.

The algorithm proceeds by adjusting the "Acceleration, Cruising and Deceleration" instances as described in the previous examples until the cruising speed of the second instance gets reduced to 40.079m/s and $time$ gets reduced to 6.26654 seconds. On the subsequent iteration, $newSpeed$ is set to $40.02909 - 0.1 = 39.92909$ m/s, which is less than the locomotive's speed at the start of the instance. Consequently, $newSpeed$ is set to 40)m/s. Additionally, 40m/s is greater than 20m/s, which is the locomotive's speed before the cruising that is prior to the "Acceleration, Cruising and Deceleration" instance. As a result, it is an alternative case where the previous non-cruising throttle setting is acceleration. These cases are handled by algorithm 16.

Algorithm 16 calculates the difference in time between the aforementioned state where the train's speed is 40m/s ($newTargetSpeed$) and the end of the instance's deceleration. This duration (represented by $oldTime$) is equal to the sum of the acceleration from 20m/s to 40m/s ($\frac{40-20}{1.5} = 13.33333$ seconds), the duration of the cruising in $originalSp$ (27.5 seconds), the duration of the acceleration from 40m/s to 40.02909m/s ($\frac{40.02909-40}{1.5} = 0.01939$ seconds), the duration of the instance's cruising (26.18247 seconds), and the duration of the deceleration from 40.02909m/s to the locomotive's speed at the state after $crState$ ($\frac{20-40.02909}{-1} = 20.02909$ seconds). In the modified speed profile, the acceleration does not change, while the deceleration starts from 40m/s, making its duration 20 seconds. The locomotive must cruise until it covers the distance the locomotive travels in $originalSp$ at 40m/s during deceleration. This distance is stored in $totalDistance$ and is calculated to 2150 meters. As a result, its duration is $\frac{2150}{40} = 53.75$, making the total duration of the modified instance 87.08333 seconds. The variable $timUsedCr$ is updated to the extra time this modification requires, which is equal to $87.08333 - 87.06428 = 0.01906$ seconds. Since the modification's extra time is less than the available extra time, $newSp$ is constructed with the computed values. The fuel consumption of $newSp$ is calculated to be 2116.68287 units and its total duration is 136.24918 seconds. Thus, its trade-off is $\frac{2116.25}{136.25} = 15.53211$, which is larger than the initialized value of $crTradeOff$. As a result, $crTradeOff$ is updated to 15.53211, $crSp$ to $newSp$, and $crTime$ to 0.01906.

At that point, the remaining extra available time is 6.11584 seconds. The speed profile contains only one modifiable instance, an "Acceleration, Cruising and Deceleration", so the following iterations are handled as normal cases until all the extra available time is used. These functionalities have already been demonstrated in examples 1, 2 and 3. Finally, algorithm 1 returns the $fuelOptimalSp$ depicted in figure 5.15. The figure compares the time-optimal speed profile to the fuel-optimal speed profile along with the

distance covered over time. It is apparent that the algorithm completely transforms the speed profile into an "Acceleration, Cruising and Acceleration" instance followed by a single "Acceleration, Cruising and Deceleration" instance. Figure 5.16 demonstrates the fuel consumption over time for the fuel-optimal speed profile. Its overall fuel consumption is 1930.4697 units over 142.365 seconds.

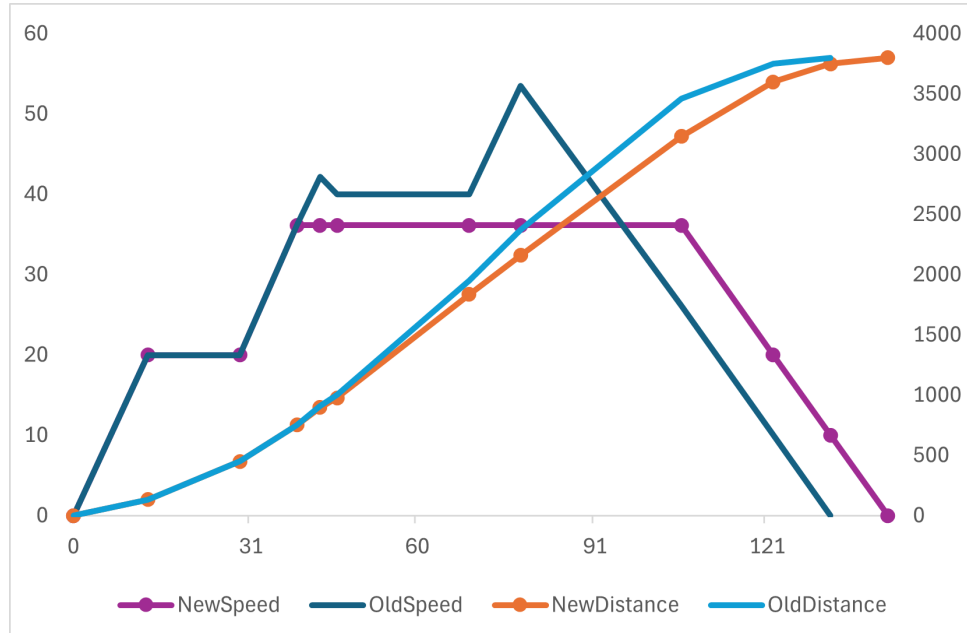


Figure 5.15: Fuel-Optimal Speed Profile of Example 4

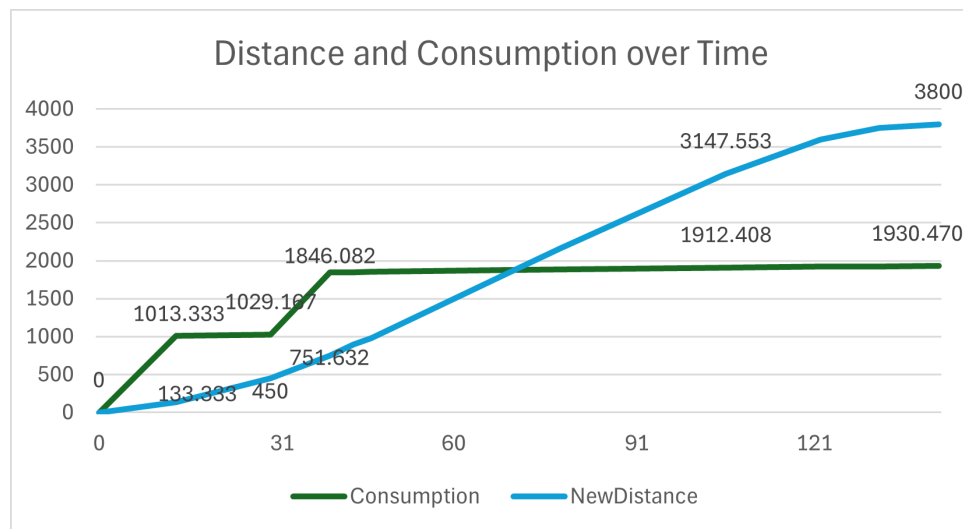


Figure 5.16: Fuel Consumption of Fuel-Optimal Speed Profile of Example 4

5.5 Example 5

5.5.1 Description

Example 5 depicts a time-optimal speed profile consisting of two "Acceleration and Deceleration" instances, similar to the previous example. The differentiation arises from the fluctuations of the throttle settings. The problem of this example is described in table 5.10. Figure 5.18 demonstrates the fuel consumption over time for the time-optimal speed profile. The total fuel consumption of the speed profile is 3783.362 units.

Parameter	Value
Train	Train 3 (Table 5.1)
Path Speed	Path 9 (Table 5.11)
Initial State	(150, 0, [1]))
Goal State	(0, 0, [12]))
Original Speed Profile	Speed Profile 9 (Figure 5.17)
Extra Time	10 sec

Table 5.10: Problem 9

Segment	Length (m)	Speed Limit (m/s)
1	300	20
2	150	40
3	700	50
4	800	80
5	1000	85
6	200	40
7	300	35
8	200	30
9	650	70
10	150	55
11	200	65
12	200	20

Table 5.11: Path 9 Layout

Figure 5.17 depicts the time-optimal speed profile along with the distance the locomotive covers. It is apparent that there are only two modifiable instances, both of which are "Acceleration and Deceleration". The cruising instances of these speed profiles are not modifiable. The highest speed of the first instance is 62.769m/s and the second instance's is 40.249m/s. The speed profile's duration is 168.9388 seconds and it covers 5100 meters. Additionally, it is worth mentioning that the diverse nature of the original speed profile corresponds to a high number of states.

5.5.2 Analysis

The form of the time-optimal speed profile necessitates the execution of only algorithm 2 in the initial steps of algorithm 1. Once both "Acceleration and Deceleration" instances are transformed into "Acceleration, Cruising and Deceleration", algorithm 3 will be executed on every iteration. The following analysis and discussion examine only the

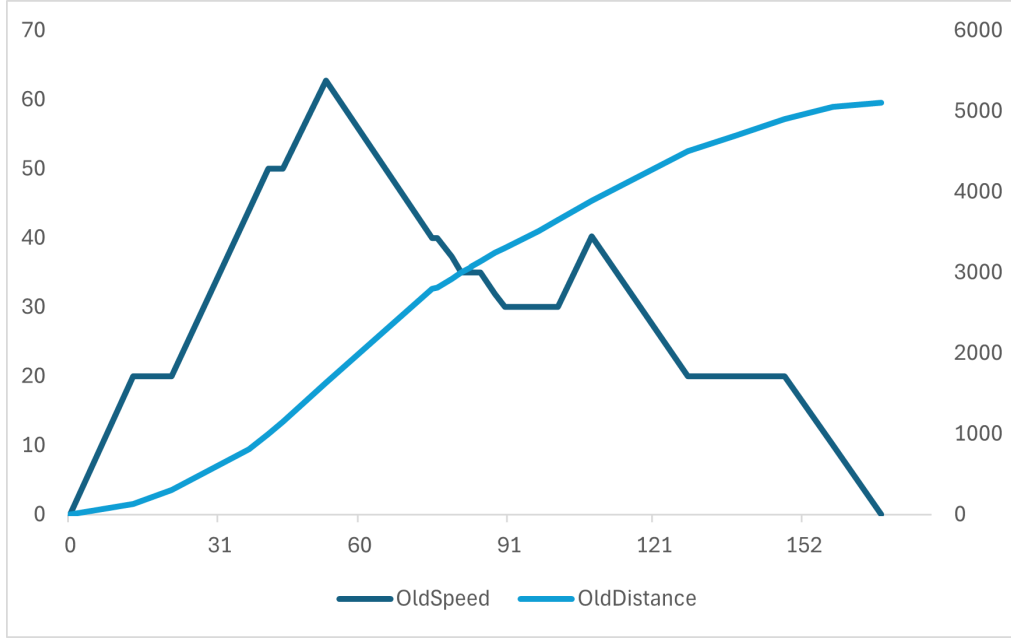


Figure 5.17: Time-Optimal Speed Profile of Example 5

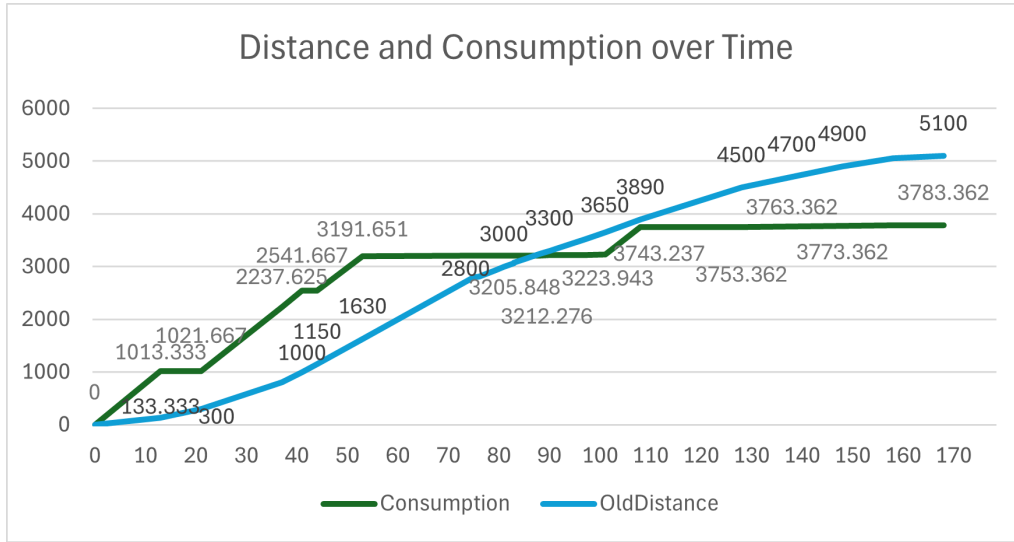


Figure 5.18: Fuel Consumption of Time-Optimal Speed Profile of Example 5 behaviour of the former algorithm.

At the first iteration, algorithm 3 invokes algorithm 4 with 0.1 seconds to transform a state using as much time as possible. The algorithm computes the position of *targetState* and initializes the variables used in its computational part. Regarding the second instance, *maxSpeed* is initialized to 40.24922m/s, while *minSpeed* is initialized to the train's speed at the start of the acceleration (30m/s). *crSpeed* is initialized to the mean of its limits ($\frac{40.24922+30}{2} = 35.12461$ m/s) and *de* is equal to 4083.13082 meters. At that speed, *totalDistance* is 4064.75857 meters which is less than *de*. Since the locomotive cannot travel the necessary distance at 40.24922m/s, the lower limit is increased to 35.12461m/s and *crSpeed* is updated to $\frac{40.24922+35.12461}{2} = 37.68692$ m/s. This procedure continues until *crSpeed* is set to 38.1107m/s, where both *totalDistance* and *de* equal 3973.78728 meters, while the cruising lasts for 3.66421 seconds. Its fuel consumption is 3677.60471 units and its duration is 169.03881 seconds, so its trade-off

is $\frac{3677.60471}{169.03881} = 21.75598$. As a result, the algorithm constructs *newSp* and updates *accToDecTradeOff*, *accToDecSp*, and *accToDecTime* based on the constructed speed profile.

At the subsequent iteration, the algorithm adjusts the first "Acceleration and Deceleration" instance with *time* being 0.1. For this instance, the upper limit is initialized to 62.76942m/s, the lower limit is initialized to 50m/s, and as a result *crSpeed* is set to $\frac{62.76942+50}{2} = 56.38471$ m/s. At that speed, *totalDistance* is 1982.0503 meters, which is less than 2010.38226 meters (the current value of *de* at 56.38471m/s). Since *totalDistance* is less than *de*, *minSpeed* is increased to 56.38471m/s and *crSpeed* is updated to $\frac{62.76942+56.38471}{2} = 59.57706$ m/s. This procedure continues until *crSpeed* is set to 60.08425m/s, where *totalDistance* and *de* are both 1794.94119 meters and the cruising's duration is 4.57527 seconds. The fuel consumption of the modified speed profile, constructed according to the calculated values, is 3650.54597 units and its duration is 169.03881 seconds. Its time-fuel trade-off is $\frac{3650.54597}{169.03881} = 21.5959$, which is less than *accToDecTradeOff*, meaning that *accToDecTradeOff*, *accToDecSp*, and *accToDecTime* do not get updated. Since there are no more "Acceleration and Deceleration" instances, the algorithm returns the computed *accToDecSp*, which then updates the variable *accToDecSp* in algorithm 2. The original speed profile does not contain any "Acceleration, Cruising and Deceleration" instances, so *fuelOptimalSp* is also set to *accToDecSp*.

At the next iteration of algorithm 1, *originalSp* contains both an "Acceleration, Cruising and Deceleration" and an "Acceleration and Deceleration" instance. That iteration transforms the second "Acceleration and Deceleration" instance into an "Acceleration, Cruising and Deceleration" instance, computing the same calculations as demonstrated previously. After the completion of the iteration, *fuelOptimalSp* has a consumption of 3544.78904 units and lasts for 169.13881 seconds.

At that point, the remaining extra available time is 9.8 seconds. The following iterations involve only "Acceleration, Cruising and Deceleration" instances, which are handled as shown by examples 1, 2 and 3. Finally, once *time* gets reduced to zero, algorithm 1 returns the *fuelOptimalSp* depicted in figure 5.19. Figure 5.19 compares the time-optimal speed profile to the fuel-optimal speed profile along with the distance covered over time. The fuel-optimal speed profile consists of a sequence of "Acceleration, Cruising and Acceleration", "Acceleration, Cruising and Deceleration", and three consecutive "Deceleration, Cruising and Deceleration" instances. Figure 5.20 demonstrates the fuel consumption over time for the fuel-optimal speed profile. Its overall fuel consumption is 2356.89835 units over 178.939 seconds.

5.6 Example 6

5.6.1 Description

Example 6 showcases a highly versatile time-optimal speed profile. It includes both "Acceleration, Cruising and Deceleration" and "Acceleration and Deceleration" instances, effectively illustrating our algorithm's behaviour against diverse instances and speed lim-

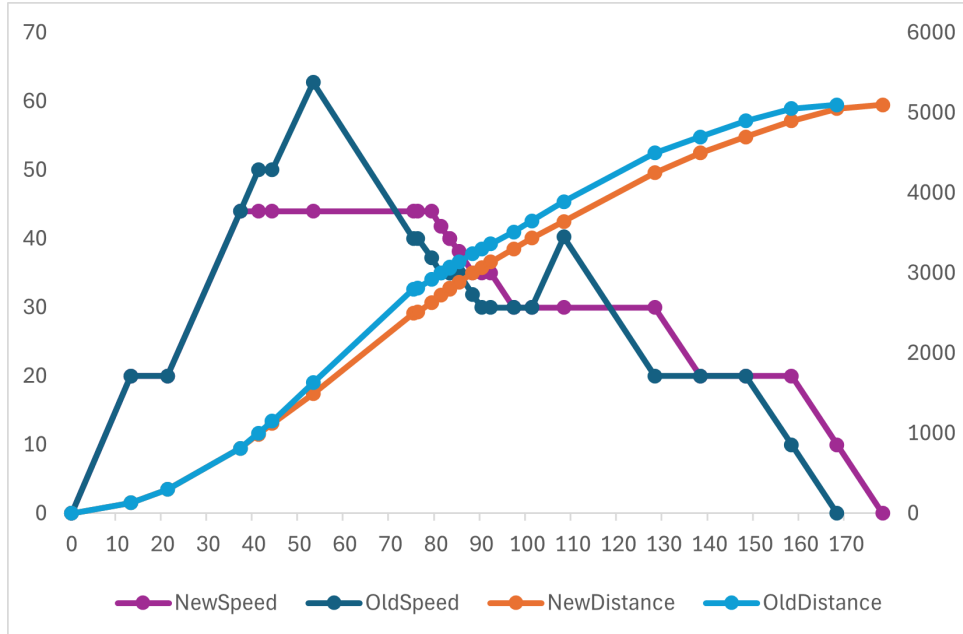


Figure 5.19: Fuel-Optimal Speed Profile of Example 5

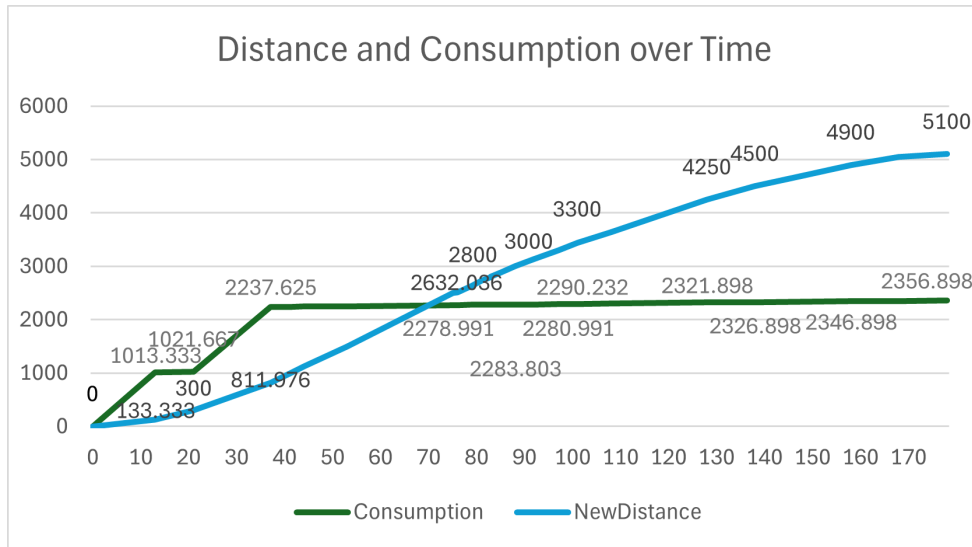


Figure 5.20: Fuel Consumption of Fuel-Optimal Speed Profile of Example 5 its. The problem is detailed in table 5.12. Figure 5.22 demonstrates the fuel consumption over time for the time-optimal speed profile, with a total fuel consumption of 5047.34291 units.

Figure 5.21 depicts the time-optimal speed profile along with the distance the locomotive travels. Due to the polymorphic nature of the speed profile, this example can be considered a simple simulation of a real-world implementation. The speed profile's duration is 217.33935 seconds, covering 8850 meters. Additionally, similar to the previous example, the speed profile consists of numerous states.

5.6.2 Analysis

This example's analysis does not delve into the algorithm's calculations, as these have already been detailed in the other examples. Instead, it aims to demonstrate its func-

Parameter	Value
Train	Train 3 (Table 5.1)
Path Speed	Path 10 (Table 5.13)
Initial State	(2850, 0, [1]))
Goal State	(0, 0, [12]))
Original Speed Profile	Speed Profile 10 (Figure 5.21)
Extra Time	10 sec

Table 5.12: Problem 10

Segment	Length (m)	Speed Limit (m/s)
1	3000	60
2	250	50
3	200	83
4	1500	73
5	150	45
6	200	70
7	100	40
8	550	45
9	600	53
10	250	40
11	2000	60
12	200	20

Table 5.13: Path 10 Layout

tionality on a more abstract level to better present its modifications. First, the algorithm transforms the first "Acceleration and Deceleration" instance, where the train's speed peaks at 63.83573m/s, into an "Acceleration, Cruising and Deceleration" instance, where the train cruises at 61.12735m/s for 4.61395 seconds. Next, it transforms the third "Acceleration and Deceleration" instance, where the train reaches 55.67764m/s, by adding a cruising phase at 53.15212m/s for 4.3092 seconds. The algorithm then transforms the remaining "Acceleration and Deceleration" instance, the locomotive's speed peaks at 48.06246m/s, into an "Acceleration, Cruising and Deceleration" instance by adding a cruising at 45.72015m/s, lasting for 4.00385 seconds. After these modifications, the total duration of the speed profile increased by 0.3 seconds (i.e., 0.1 seconds for each adjustment). It can be assumed that, regardless of the time-optimal speed profile, the algorithm always starts by transforming the "Acceleration and Deceleration". Specifically, it first applies the transformation where the cruising in the modified speed profile has the lowest duration among the other applications of the "Addition of Cruising" method,

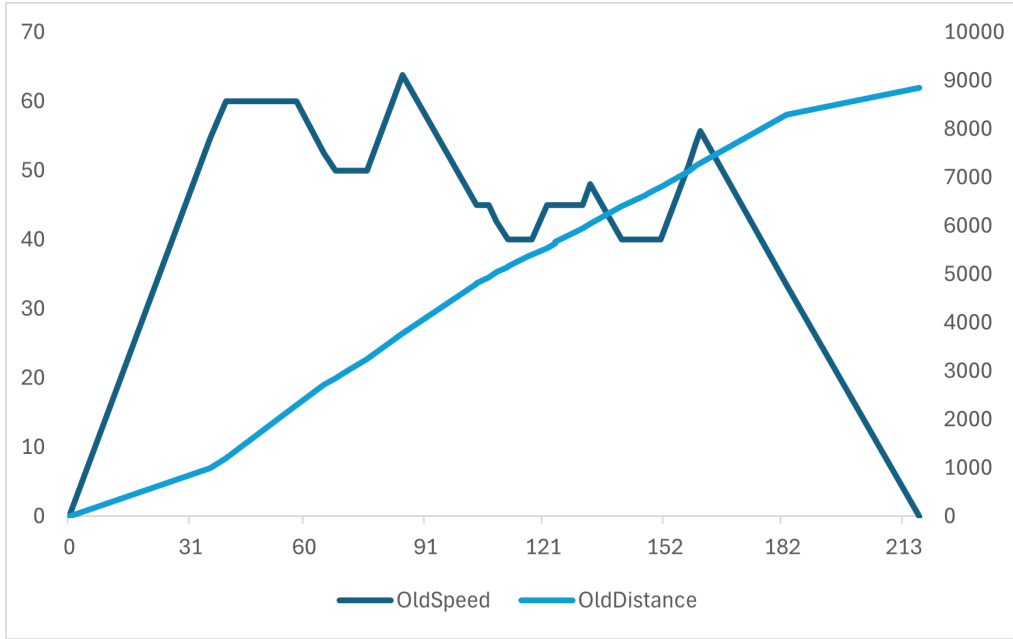


Figure 5.21: Time-Optimal Speed Profile of Example 6

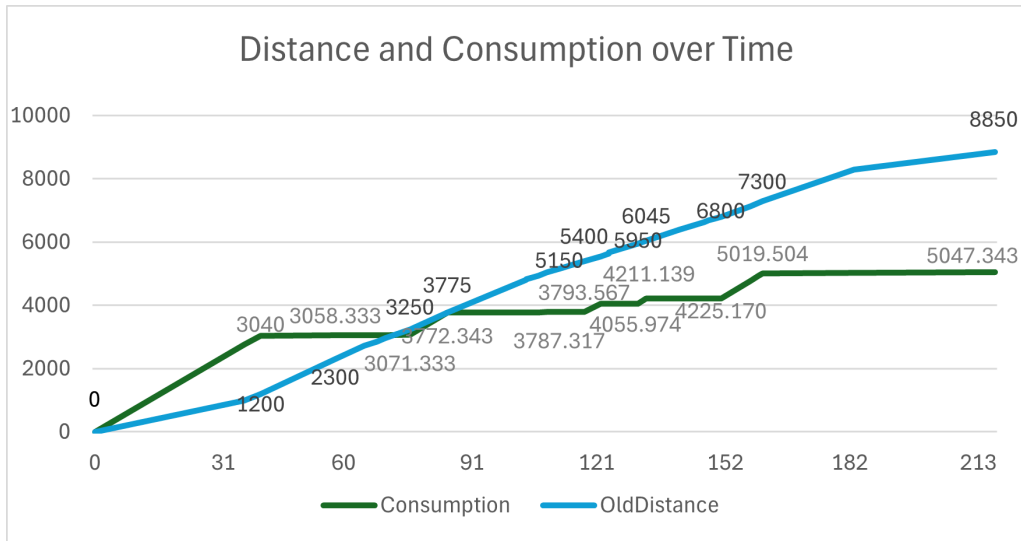


Figure 5.22: Fuel Consumption of Time-Optimal Speed Profile of Example 6 as evidenced by both this and examples 4 and 5.

At this point, there are only four modifiable instances in the speed profile, all of which are "Acceleration, Cruising and Deceleration". We observed that the algorithm modifies the instance where the fraction $\frac{\text{Cruising Speed}}{\text{Cruising Duration}}$ of the original speed profile is the greatest. The algorithm iterates over 300 times, so we will not analyze each iteration. Instead, we will mention that as the cruising speed of the third "Acceleration, Cruising and Deceleration" instance reduces, after only 33 iterations, it unifies its prior "Acceleration, Cruising and Acceleration" into itself, creating a longer "Acceleration, Cruising and Deceleration" instance. On the other hand, at the 303rd iteration, the reduction of the second instance's cruising speed transforms it into "Deceleration, Cruising and Deceleration".

Finally, algorithm 1 terminates and returns the *fuelOptimalSp* depicted in 5.23.

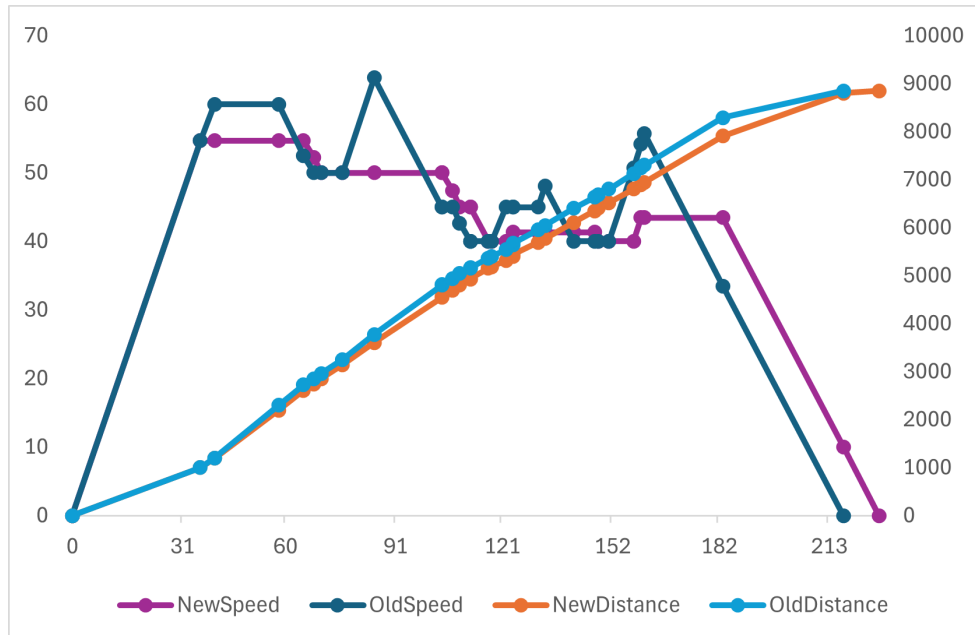


Figure 5.23: Fuel-Optimal Speed Profile of Example 6

Figure 5.24 demonstrates the fuel consumption over time for the fuel-optimal speed profile. Its overall fuel consumption is 3169.93685 units over 227.33935 seconds.

Figure 5.25 depicts the algorithm's result when 130 seconds of extra time is provided. Essentially, it transforms the entire speed profile into a single "Acceleration, Cruising and Deceleration" instance. This transformation is plausible for every time-optimal speed profile in which the locomotive is immobile. In this case, the overall fuel consumption is 1696.83818 units over 347.33935 seconds.

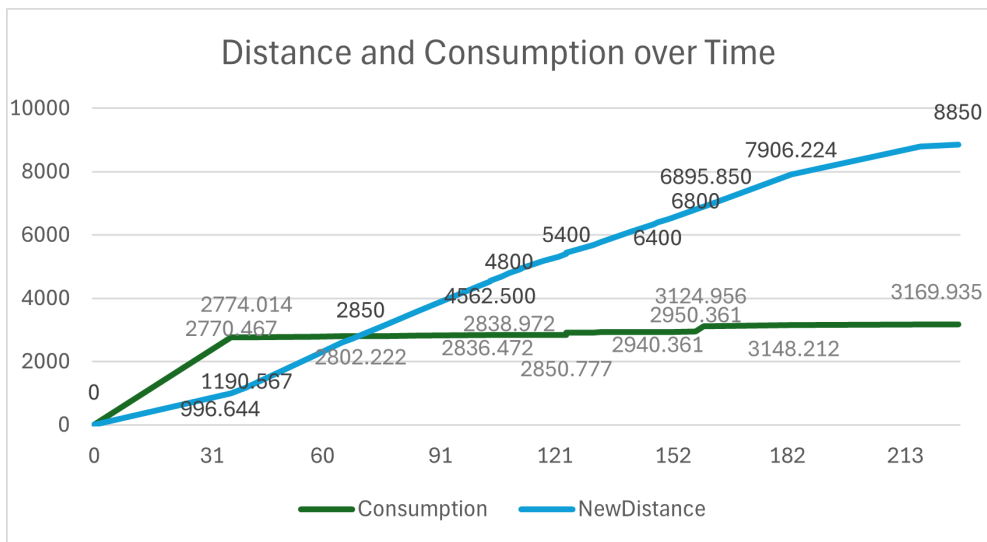


Figure 5.24: Fuel Consumption of Fuel-Optimal Speed Profile of Example 6

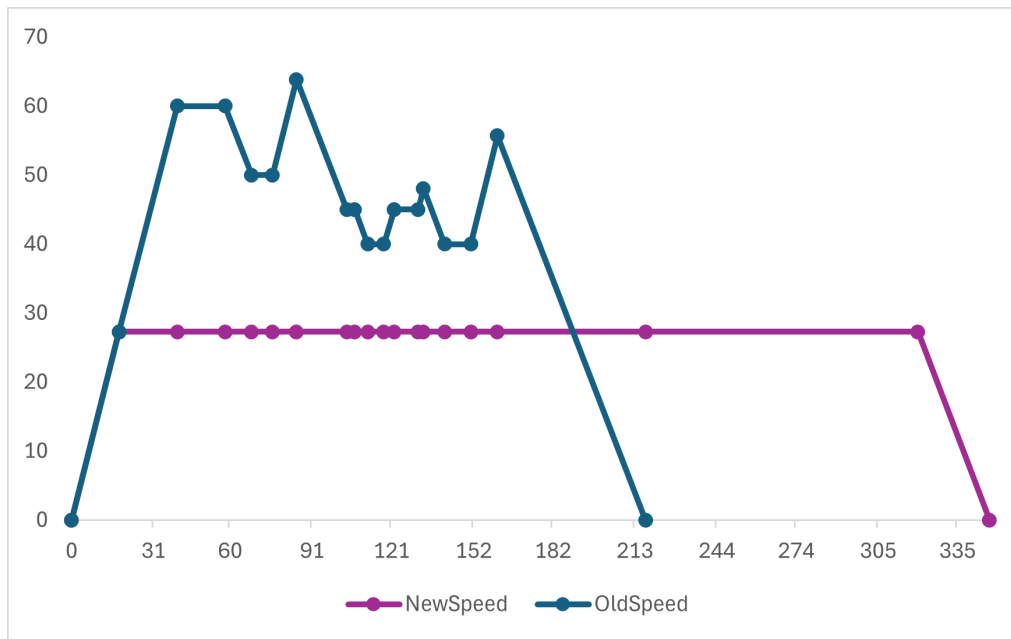


Figure 5.25: Fuel-Optimal Speed Profile of Example 6

Chapter 6

Conclusion and Future Work

This study addressed the transformation of a time-optimal speed profile to fuel-optimal, given a specific amount of extra time, by adjusting throttle setting sequences. The distinctive characteristic of the path is its segmentation into sections of varying lengths and speed limits. To tackle this challenge, we provided a realistic problem formulation and developed a linear fuel consumption model. By examining all possible throttle setting sequences, this study established that neither the acceleration nor the deceleration rates should be altered. Instead, it is more effective to either lower the cruising speed of an "Acceleration, Cruising and Deceleration" instance or to create such an instance whenever possible.

Specifically, we concluded that the best approach is to first transform "Acceleration and Deceleration" instances into "Acceleration, Cruising and Deceleration" and then lower the cruising speed of the latter instances. We discovered that it is optimal to choose the instance with the highest speed for "Acceleration and Deceleration" instances, while for "Acceleration, Cruising and Deceleration" instances, it is best to select the one where the fraction $\frac{\text{Cruising Speed}}{\text{Cruising Duration}}$ is greater. We envision our approach being implemented into a train scheduling model that operates over a railway network to maximize the use of delays, created to avoid collisions at intersections, to reduce the total fuel consumption.

Despite the progress made, there are several avenues for future research that can extend the scope and impact of this study. Our work ought to extend into considering more physical characteristics of the problem, such as train load, track geometry, and wind resistance. Additionally, our linear model should be modified to more efficiently capture the different rates of fuel consumption at various speeds. It would be of great interest to study the behaviour of our approach within a train scheduling model. Exploring real-world data about railways could provide valuable insights to develop a fully functional product. By addressing these future research directions, this study could serve as a foundation for further advancements in minimizing locomotive fuel consumption.

References

- Abbey, W. W. (2022). *The diesel that did it: General motors' ft locomotive*. Indiana University Press.
- Aceves, S. M., Flowers, D. L., Martinez-Frias, J., Smith, J., Dibble, R., Au, M., & Girard, J. (2001). *Hcci combustion: Analysis and experiments* (tech. rep.). Lawrence Berkeley National Lab.(LBNL), Berkeley, CA (United States).
- Agasucci, V., Grani, G., & Lamorgese, L. (2023). Solving the train dispatching problem via deep reinforcement learning. *Journal of Rail Transport Planning & Management*, 26, 100394.
- Ahn, K. (1998). *Microscopic fuel consumption and emission modeling* [Doctoral dissertation, Virginia Tech].
- Ahn, K. (2002). *Modeling light duty vehicle emissions based on instantaneous speed and acceleration levels* [Doctoral dissertation, Virginia Polytechnic Institute and State University].
- Ahn, K., Rakha, H., Trani, A., & Van Aerde, M. (2002). Estimating vehicle fuel consumption and emissions based on instantaneous speed and acceleration levels. *Journal of transportation engineering*, 128(2), 182–190.
- Akcelik, R. (1985). An interpretation of the parameters in the simple average travel speed model of fuel consumption. *Australian Road Research*, 15(1).
- Amsden, A., Ramshaw, J., O'rourke, P., & Dukowicz, J. (1985). *Kiva: A computer program for two-and three-dimensional fluid flows with chemical reactions and fuel sprays* (tech. rep.). Los Alamos National Lab., NM (USA).
- Assanis, D. N., & Heywood, J. B. (1986). Development and use of a computer simulation of the turbocompounded diesel system for engine performance and component heat transfer studies. *SAE transactions*, 451–476.
- Austen, A., & Lyn, W. (1962). Some steps toward calculating diesel engine behavior. *SAE Transactions*, 70, 526–550.
- Bi, X., Yang, M., Han, S., & Ma, Z. (1999). *A multi-zone model for diesel spray combustion* (tech. rep.). SAE Technical Paper.
- Biggs, D., & Akcelik, R. (1987). Estimating effect of vehicle characteristics on fuel consumption. *Journal of transportation engineering*, 113(1), 101–106.
- Bożejko, W., Grymin, R., & Pempera, J. (2017). Scheduling and routing algorithms for rail freight transportation. *Procedia Engineering*, 178, 206–212.
- Bracco, F. (1985). Modeling of engine sprays. *SAE transactions*, 144–167.
- Brandstätter, W., Johns, R., & Wigley, G. (1985). The effect of inlet port geometry on in-cylinder flow structure. *SAE transactions*, 680–700.

- Carrizosa Priego, E. J., Harbering, J., & Schöbel, A. (2016). Minimizing the passengers' traveling time in the stop location problem. *Journal of the Operational Research Society*, 67 (10), 1325-1337.
- Chang, D. J., & Morlok, E. K. (2005). Vehicle speed profiles to minimize work and fuel consumption. *Journal of transportation engineering*, 131(3), 173-182.
- Chang, M.-F., & Herman, R. (1981). Trip time versus stop time and fuel consumption characteristics in cities. *Transportation Science*, 15(3), 183-209.
- Commission, E., for Mobility, D.-G., & Transport. (2011). *White paper on transport: Roadmap to a single european transport area: Towards a competitive and resource-efficient transport system*. Publications office of the European Union.
- Commission, E., for Mobility, D.-G., & Transport. (2022). *Eu transport in figures – statistical pocketbook 2022*. Publications Office of the European Union. <https://doi.org/doi/10.2832/216553>
- Cook, J. A., Sun, J., Buckland, J. H., Kolmanovsky, I. V., Peng, H., & Grizzle, J. W. (2006). Automotive powertrain control—a survey. *Asian Journal of Control*, 8(3), 237-260.
- Corman, F., D'Ariano, A., Pacciarelli, D., & Pranzo, M. (2010). A tabu search algorithm for rerouting trains during rail operations. *Transportation Research Part B: Methodological*, 44(1), 175-192.
- D'ariano, A., Pacciarelli, D., & Pranzo, M. (2007). A branch and bound algorithm for scheduling trains in a railway network. *European journal of operational research*, 183(2), 643-657.
- Daw, C., Kennel, M., Finney, C., & Connolly, F. (1998). Observing and modeling nonlinear dynamics in an internal combustion engine. *Physical Review E*, 57(3), 2811.
- Delhay, B., & Taklanti, A. (1999). Multi-dimensional modeling of the aerodynamic and combustion in diesel engines. *Oil & Gas Science and Technology*, 54(2), 271-277.
- Ding, Y. (2000). *Quantifying the impact of traffic-related and driver-related factors on vehicle fuel consumption and emissions* [Doctoral dissertation, Virginia Tech].
- Ding, Y., & Rakha, H. (2002). Trip-based explanatory variables for estimating vehicle fuel consumption and emission rates. *Water, Air and Soil Pollution: Focus*, 2, 61-77.
- Dündar, S., & Şahin, İ. (2013). Train re-scheduling with genetic algorithms and artificial neural networks for single-track railways. *Transportation Research Part C: Emerging Technologies*, 27, 1-15.
- Eaton, J., Yang, S., & Gongora, M. (2017). Ant colony optimization for simulated dynamic multi-objective railway junction rescheduling. *IEEE Transactions on Intelligent Transportation Systems*, 18(11), 2980-2992.
- Economic, E. C., & Papers, F. A. E. E. O. (2014, December). *Infrastructure in the eu: Developments and impact on growth* (tech. rep.) (ISBN 978-92-79-38818-7 (online), ISBN 978-92-79-38819-4 (print)). European Commission. Brussels, Belgium. [https://doi.org/10.2765/85301\(online\),10.2765/85337\(print\)](https://doi.org/10.2765/85301(online),10.2765/85337(print))
- Evans, L., & Herman, R. (1978). Automobile fuel economy on fixed urban driving schedules. *Transportation Science*, 12(2), 137-152.
- Evans, L., Herman, R., & Lam, T. (1976). Multivariate analysis of traffic factors related to fuel consumption in urban driving. *Transportation Science*, 10(2), 205-215.
- Everall, P. (1968). *The effects of road and traffic conditions on fuel consumption*. Road Research Laboratory.

- Facts, E. (2021). Us transportation sector greenhouse gas emissions, 1990–2019. *Office of Transportation and Air Quality: Washington DC, USA*.
- Filipi, Z. S., & Assanis, D. N. (2001). A nonlinear, transient, single-cylinder diesel engine simulation for predictions of instantaneous engine speed and torque. *J. Eng. Gas Turbines Power*, 123(4), 951–959.
- Foster, D. E. (1985). An overview of zero-dimensional thermodynamic models for ic engine data analysis. *SAE transactions*, 436–449.
- Frey, H. C., Roupail, N. M., Zhai, H., Farias, T. L., & Gonçalves, G. A. (2007). Comparing real-world fuel consumption for diesel-and hydrogen-fueled transit buses and implication for emissions. *Transportation Research Part D: Transport and Environment*, 12(4), 281–291.
- Gao, D. W., Mi, C., & Emadi, A. (2007). Modeling and simulation of electric and hybrid vehicles. *Proceedings of the IEEE*, 95(4), 729–745.
- Gattermann, P., Harbering, J., & Schöbel, A. (2015). Generation of line pools. *Planning a Public Transportation System with a View Towards Passengers' Convenience*, 91.
- Genta, G. (1997). *Motor vehicle dynamics: Modeling and simulation* (Vol. 43). World scientific.
- Gong, I., Oh, S., & Min, Y. (2020). Train scheduling with deep q-network: A feasibility test. *Applied Sciences*, 10(23), 8367.
- González-Gil, A., Palacin, R., Batty, P., & Powell, J. (2014). A systems approach to reduce urban rail energy consumption. *Energy Conversion and Management*, 80, 509–524.
- Guo, S., & Chen, L. (2019). Can urban rail transit systems alleviate air pollution? empirical evidence from beijing. *Growth and Change*, 50(1), 130–144.
- Guo, X., Sun, W., Yao, S., & Zheng, S. (2020). Does high-speed railway reduce air pollution along highways?—evidence from china. *Transportation Research Part D: Transport and Environment*, 89, 102607.
- Guzzella, L., & Amstutz, A. (1998). Control of diesel engines. *IEEE Control Systems Magazine*, 18(5), 53–71.
- Harbering, J. (2017). Delay resistant line planning with a view towards passenger transfers. *Top*, 25(3), 467–496.
- Harbering, J., Ranade, A., Schmidt, M., & Sinnen, O. (2015). Single track train scheduling. *Planning a Public Transportation System with a View Towards Passengers' Convenience*, 151.
- Harris, H., & Pearce, F. (1990). A universal mathematical model of diesel engine performance. *Journal of agricultural engineering research*, 47, 165–176.
- Hendricks, E. (1986). A compact, comprehensive model of large turbocharged, two-stroke diesel engines. *SAE transactions*, 820–834.
- Hendricks, E. (1997). Engine modelling for control applications: A critical survey. *Mechanica*, 32, 387–396.
- Hendricks, E. (2001). Isothermal vs. adiabatic mean value si engine models. *IFAC Proceedings Volumes*, 34(1), 363–368.
- Hendricks, E., & Sorenson, S. C. (1990). Mean value modelling of spark ignition engines. *SAE transactions*, 1359–1373.
- Heywood, J. B. (1988). Internal combustion engine fundamentals. (*No Title*).
- Hrovat, D., & Sun, J. (1997). Models and control methodologies for ic engine idle speed control design. *Control Engineering Practice*, 5(8), 1093–1100.

- Hung, W.-T., Tong, H.-Y., & Cheung, C.-S. (2005). A modal approach to vehicular emissions and fuel consumption model development. *Journal of the Air & Waste Management Association*, 55(10), 1431–1440.
- IEA. (2023, July). Rail. <https://www.iea.org/energy-system/transport/rail>
- Jung, D., & Assanis, D. N. (2001). Multi-zone di diesel spray combustion model for cycle simulation studies of engine performance and emissions. *SAE transactions*, 1510–1532.
- Kao, M., & Moskwa, J. J. (1995). Turbocharged diesel engine modeling for nonlinear engine control and state estimation.
- Karmiggelt, R. (1998). Mean value modelling of a si engine.
- Khadilkar, H. (2018). A scalable reinforcement learning algorithm for scheduling railway lines. *IEEE Transactions on Intelligent Transportation Systems*, 20(2), 727–736.
- Khayyam, H., Kouzani, A., Khoshmanesh, K., & Hu, E. (2008). Power analysis and simulation of a vehicle under combined loads.
- Killingsworth, N. J., Aceves, S. M., Flowers, D. L., & Krstic, M. (2006). A simple hcci engine model for control. *2006 IEEE Conference on Computer Aided Control System Design, 2006 IEEE International Conference on Control Applications, 2006 IEEE International Symposium on Intelligent Control*, 2424–2429.
- Kim, H. M., Kokkolaras, M., Louca, L. S., Delagrammatikas, G. J., Michelena, N. F., Filipi, Z. S., Papalambros, P. Y., Stein, J. L., & Assanis, D. N. (2002). Target cascading in vehicle redesign: A class vi truck study. *International journal of vehicle design*, 29(3), 199–225.
- Komninos, N. P., Hountalas, D. T., & Kouremenos, D. (2005). *Description of in-cylinder combustion processes in hcci engines using a multi-zone model* (tech. rep.). SAE Technical paper.
- Lavoie, G., & Blumberg, P. (1980). A fundamental model for predicting fuel consumption, nox and hc emissions of the conventional spark-ignited engine. *Combustion Science and Technology*, 21(5-6), 225–258.
- Liao, T.-Y., & Machemehl, R. B. (1998). Development of an aggregate fuel consumption model for signalized intersections. *Transportation research record*, 1641(1), 9–18.
- Lin, D., Nelson, J. D., Beecroft, M., & Cui, J. (2021). An overview of recent developments in china's metro systems. *Tunnelling and Underground Space Technology*, 111, 103783.
- Liu, Z., & Chen, R. (2009). A zero-dimensional combustion model with reduced kinetics for si engine knock simulation. *Combustion science and technology*, 181(6), 828–852.
- Logan, K. G., Nelson, J. D., McLellan, B. C., & Hastings, A. (2020). Electric and hydrogen rail: Potential contribution to net zero in the uk. *Transportation Research Part D: Transport and Environment*, 87, 102523.
- Manitz, J., Harbering, J., Schmidt, M., Kneib, T., & Schöbel, A. (2015). 3.5 network-based source detection for propagation processes on complex networks with application to train delays on railway systems. *Planning a Public Transportation System with a View Towards Passengers' Convenience*, 177.
- Manolakis, D., & Refanidis, I. (2023). Finding time optimal routes for trains using basic kinematics and a. *European Conference on Artificial Intelligence*, 119–137.
- Miandoab, M. H., Ghezavati, V., & Mohammaditabar, D. (2020). Developing a simultaneous scheduling of passenger and freight trains for an inter-city railway con-

- sidering optimization of carbon emissions and waiting times. *Journal of Cleaner Production*, 248, 119303.
- Newman, P., Alimoradian, B., & Lyons, T. (1989). Estimating fleet fuel consumption for vans and small trucks. *Transportation science*, 23(1), 46–50.
- Ni, D., & Henclewood, D. (2008). Simple engine models for vii-enabled in-vehicle applications. *IEEE Transactions on Vehicular Technology*, 57(5), 2695–2702.
- Nilsson, Y., Eriksson, L., & Gunnarsson, M. (2008). Torque modelling for optimising fuel economy in variable compression engines. *International Journal of Modelling, Identification and Control*, 3(3), 327–339.
- Nirmala, G., & Ramprasad, D. (2014). A genetic algorithm based railway scheduling model. *International Journal of Science and Research (IJSR)*, 3(1), 11–14.
- Ouladsine, M., Bloch, G., & Dovifaaz, X. (2005). Neural modelling and control of a diesel engine with pollution constraints. *Journal of Intelligent and Robotic Systems*, 41, 157–171.
- Powell, B., Bailey, K., & Cikanek, S. (1998). Dynamic modeling and control of hybrid electric vehicle powertrain systems. *IEEE Control Systems Magazine*, 18(5), 17–33.
- Pu, J., & Yin, C. (2007). Optimal control of fuel economy in parallel hybrid electric vehicles. *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, 221(9), 1097–1106.
- Rakha, H., Ahn, K., Faris, W., & Moran, K. (2010). Simple vehicle powertrain model for use in traffic simulation software. *89th Transportation Research Board Annual Meeting*.
- Rakha, H., & Ahn, K. (2004). Integration modeling framework for estimating mobile source emissions. *Journal of transportation engineering*, 130(2), 183–193.
- Rakha, H., Van Aerde, M., Ahn, K., & Trani, A. (2000). Requirements for evaluating traffic signal control impacts on energy and emissions based on instantaneous speed and acceleration measurements. *Transportation Research Record*, 1738(1), 56–67.
- Richardson, A., & Akcelik, R. (1981). Fuel consumption models and data needs for the design and evaluation of urban traffic systems. *Transportation Conference 1981: Preprints of Papers*, 21–28.
- Samà, M., Pellegrini, P., D'Ariano, A., Rodriguez, J., & Pacciarelli, D. (2016). Ant colony optimization for the real-time train routing selection problem. *Transportation Research Part B: Methodological*, 85, 89–108.
- Schmitt, J., Fremovici, M., Grondin, O., & Le Berr, F. (2009). Compression ignition engine model supporting powertrain development. *IFAC Proceedings Volumes*, 42(26), 48–55.
- Sciarretta, A., Back, M., & Guzzella, L. (2004). Optimal control of parallel hybrid electric vehicles. *IEEE Transactions on control systems technology*, 12(3), 352–363.
- Šemrov, D., Marsetič, R., Žura, M., Todorovski, L., & Srdic, A. (2016). Reinforcement learning approach for train rescheduling on a single-track railway. *Transportation Research Part B: Methodological*, 86, 250–267.
- Sim, H., & Sunwoo, M. (2004). Development and validation of a dynamic model for a gaseous-fuel engine. *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, 218(7), 697–705.

- Song, G., Yu, L., & Wang, Z. (2009). Aggregate fuel consumption model of light-duty vehicles for evaluating effectiveness of traffic management strategies on fuels. *Journal of Transportation Engineering*, 135(9), 611–618.
- Streit, E. E., & Borman, G. L. (1971). Mathematical simulation of a large turbocharged two-stroke diesel engine. *SAE transactions*, 733–768.
- Taylor, N. B. (2003). The contram dynamic traffic assignment model. *Networks and spatial economics*, 3, 297–322.
- Tormos, P., Lova, A., Barber, F., Ingolotti, L., Abril, M., & Salido, M. A. (2008). A genetic algorithm for railway scheduling problems. *Metaheuristics for scheduling in industrial and manufacturing applications*, 255–276.
- Varnavas, C., & Assanis, D. N. (1996). *A high temperature and high pressure evaporation model for the kiva-3 code* (tech. rep.). SAE Technical Paper.
- Wahlström, J. (2006). *Control of egr and vgt for emission control and pumping work minimization in diesel engines* [Doctoral dissertation, Institutionen för systemteknik].
- Wang, H., Fu, L., Zhou, Y., & Li, H. (2008). Modelling of the fuel consumption for passenger cars regarding driving characteristics. *Transportation Research Part D: Transport and Environment*, 13(7), 479–482.
- Wang, M., Wang, L., Xu, X., Qin, Y., & Qin, L. (2019). Genetic algorithm-based particle swarm optimization approach to reschedule high-speed railway timetables: A case study in china. *Journal of Advanced Transportation*, 2019(1), 6090742.
- Watson, N., Pilley, A., & Marzouk, M. (1980). *A combustion correlation for diesel engine simulation* (tech. rep.). SAE Technical Paper.
- Watson, N. (1984). Dynamic turbocharged diesel engine simulator for electronic control system development.
- Whitehouse, N. D., & Sareen, B. K. (1974). *Prediction of heat release in a quiescent chamber diesel engine allowing for fuel/air mixing* (tech. rep.). SAE Technical Paper.
- Winterbone, D., Thiruarooran, C., & Wellstead, P. (1977). A wholly dynamic model of a turbocharged diesel engine for transfer function evaluation.
- Xiang, P., Xu, S., Zhao, H., Jiang, L., Ma, H., & Liu, X. (2023). Running safety analysis of a train-bridge coupled system under near-fault ground motions considering rupture directivity effects. *Structures*, 58, 105382.
- Yoon, P., Park, S., & Sunwoo, M. (2000). *A nonlinear dynamic model of si engines for designing controller* (tech. rep.). SAE Technical Paper.
- Yoon, Y. H., & Bang, Y. (2011, February). Apparatus and method for controlling speed in automatic train operation [US Patent App. 12/856,528].
- Yue, H. (2008). Mesoscopic fuel consumption and emission modeling.
- Zhao, H., Wei, B., Shao, Z., Xie, X., Jiang, L., & Xiang, P. (2023). Assessment of train running safety on railway bridges based on velocity-related indices under random near-fault ground motions. *Structures*, 57, 105244.
- Zhou, P., Zhou, S., & Clelland, D. (2006). A modified quasi-dimensional multi-zone combustion model for direct injection diesels. *International Journal of Engine Research*, 7(4), 335–345.
- Zhou, W., & Teng, H. (2016). Simultaneous passenger train routing and timetabling using an efficient train-based lagrangian relaxation decomposition. *Transportation Research Part B: Methodological*, 94, 409–439.
- Zweiri, Y., Whidborne, J., & Seneviratne, L. (2001). Detailed analytical model of a single-cylinder diesel engine in the crank angle domain. *Proceedings of the Institution*

of Mechanical Engineers, Part D: Journal of Automobile Engineering, 215(11), 1197–1216.